



Entwurf und Validierung eines Frameworks für cloudbasierte Simulationen von Datenräumen

Masterarbeit

im Studiengang Informatik M. Sc. - Schwerpunkt
'Komplexe Softwaresysteme'

Autor: Michel Otto
Matrikelnummer: 5125216

Version vom: 2. Mai 2023

1. Prüfer: Prof. Dr. Lars Braubach
Hochschule Bremen, Fakultät 4 - Elektrotechnik und Informatik

2. Prüferin: M.Sc. Linda Rülcke
Fraunhofer-Institut für Energiewirtschaft und Energiesystemtechnik

Erklärung über das eigenständige Erstellen der Arbeit

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Die Stellen der Arbeit, die anderen Werken dem Wortlaut oder dem Sinn nach entnommen wurden, sind durch Angaben der Herkunft kenntlich gemacht.

Diese Erklärung erstreckt sich auch auf in der Arbeit enthaltene Grafiken, Skizzen, bildliche Darstellungen sowie auf Quellen aus dem Internet. Die Arbeit habe ich in gleicher oder ähnlicher Form auch auszugsweise noch nicht als Bestandteil einer Prüfungs- oder Studienleistung vorgelegt.

Ich versichere, dass die eingereichte elektronische Version der Arbeit vollständig mit der Druckversion übereinstimmt.

Michel Otto

Matrikelnummer: 5125216

Bremen, den 2. Mai 2023

Unterschrift

Entwurf und Validierung eines Frameworks für cloudbasierte Simulationen von Datenräumen

Michel Otto

Zusammenfassung

Zur Förderung von digitalen Ökosystemen finden derzeit diverse europäische Projekte und Initiativen zum Aufbau von Datenräumen für einen souveränen Datenaustausch statt. Aufgrund der gegebenen Komplexität der Technologien und vielfältigen Gestaltungsmöglichkeiten bezüglich Architektur, eingesetzter Komponenten und Ablauf gibt es einen Bedarf an technischer Unterstützung zum vereinfachten Experimentieren in diesem Kontext. Diese Arbeit schlägt ein technisches Rahmenwerk vor, welches die Beschreibung, Simulation und Auswertung von Datenraumszenarien ermöglicht und damit die Entwicklung geeigneter Lösungen unterstützt. Hierfür werden zu Beginn Ziele definiert, Grundlagen beschrieben und verwandte Arbeiten analysiert. Im Hauptteil erfolgt eine umfassende Beschreibung der Referenzimplementierung des Frameworks. Die Evaluation anhand eines praxisrelevanten Projekts demonstriert die Tauglichkeit für das Anwendungsgebiet.

Abstract

To promote digital ecosystems, various European projects and initiatives are currently underway to establish data spaces for sovereign data exchange. Due to the given complexity of the technologies and diverse design options in terms of architecture, deployed components and data flow, there is a need for technical support to facilitate experimentation. This thesis proposes a technical framework that enables the description, simulation and evaluation of data space scenarios and thus supports the development of suitable solutions. For this purpose this document defines the objectives of the framework, describes the fundamentals in the required knowledge domains, and analyses related work. The main part gives a comprehensive description of the reference implementation. An evaluation based on a practical project demonstrates its suitability for the application area.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Ziel und Lösungsansatz	2
1.2	Vorgehen / Methodik	4
1.3	Aufbau der Arbeit	5
2	Grundlagen	7
2.1	Digitale Ökosysteme	7
2.1.1	Herkunft und Definition	9
2.1.2	Datensouveränität	10
2.1.3	Datenräume	11
2.1.4	Aufbau von Datenräumen	14
2.1.5	Aktueller Stand und Entwicklung	17
2.2	Simulation	19
2.3	Softwarearchitektur	20
2.3.1	Softwarearchitektur & -design	20
2.3.2	Architekturprinzipien	20
2.3.3	Architektur- und Entwurfsmuster	22
2.3.4	Framework	23
2.4	Verwendete Technologien	24
2.4.1	TypeScript	25
2.4.2	Kubernetes	25
3	Anforderungen	33
3.1	Zielformulierung	33
3.2	Anwender	34
3.3	Fachliche Anforderungen	34
4	Verwandte Arbeiten	37
5	Frameworkarchitektur	41
5.1	Systemkontext	41

5.2	Systemübersicht	42
5.3	Module	43
5.3.1	dssim-core	43
5.3.2	Scenario-Execution	44
5.3.3	Scenario-Controller	44
5.3.4	DataSpace-Component-Controller	45
5.3.5	Environment-Controller	45
5.3.6	Scenario-Monitoring	46
5.4	Konfiguration	47
5.5	Technologien	48
5.6	Versionierung	48
6	Umsetzung Framework-Module	51
6.1	Core-Modul	51
6.1.1	Architektur	52
6.1.2	Hilfsfunktionen	52
6.2	Scenario-Execution	53
6.2.1	Szenariokonfiguration	53
6.2.2	Szenariobeschreibung	55
6.2.3	Konnektivität	58
6.2.4	Skalierung	58
6.3	Scenario-Controller	59
6.3.1	Generische Factory Methoden	59
6.4	IDS Controller	61
6.4.1	Fassade	62
6.4.2	Usage Rule Mapper	64
6.4.3	Steuerungsbibliotheken	65
6.5	Kubernetes-Controller	66
6.5.1	Architektur	66
6.5.2	Bereitstellung von Dateien	70
6.5.3	Routing	70
6.5.4	Umsetzung Ressourcensteuerung	71
6.6	Scenario-Monitoring	75
6.6.1	Logging-Pipeline	76
6.6.2	Scraping-Pipeline	78
6.6.3	Auswertungsmodul	78
7	Evaluation	81
7.1	Szenario	81
7.1.1	Fachliche Beschreibung	82
7.1.2	Umsetzung im Framework	85

7.1.3	Erkenntnisse aus Szenarioaufbau und Simulation	88
7.2	Gegenüberstellung der Anforderungen & Funktionen	92
8	Zusammenfassung und Fazit	95
9	Ausblick	99
	Literatur	101
	Akronyme	111
	Abbildungsverzeichnis	113
	Quellcodeverzeichnis	115
Anhang		i
1	Klassendiagramm dssim-core	i
2	Implementierung Evaluationsszenario	ii
2.1	DSC spezifisch	ii
2.2	EDC spezifisch	x
2.3	Mit Fassade	xvii

Kapitel 1

Einleitung

Das staatliche Fördervolumen und die Anzahl namhafter Teilnehmer aus Industrie und Wissenschaft an großen Initiativen wie *GaiaX* und konkreten Projekten wie *Catena-X*¹, *Mobility Dataspaces*² oder *Health-X*³ zeugen in aller Deutlichkeit von dem hohen wirtschaftlichen und politischen Interesse digitale Ökosysteme im europäischen Raum mithilfe von praxistauglichen Datenräumen weiter auszubauen.

Ziel dieser Datenräume ist es, einen sicheren und souveränen Datenaustausch zwischen Marktteilnehmern zu ermöglichen, den Wert der Daten besser nutzbar zu machen und neue Werte durch die Zusammenführung von Daten zu schaffen. Im Gegensatz zum Ansatz der zentralen Datenplattform sind die Datenräume dezentraler organisiert und die Anbieter der Daten sollen dabei fortwährend volle Kontrolle über die Nutzung behalten. [Franklin, Halevy und Maier 2005][Curry 2015][S.51f].

Datenräume entstehen durch die Verbindung von Datenraumkomponenten, welche die notwendigen Funktionen eines Datenraums gewährleisten. Hierzu zählt beispielsweise die gesicherte Übertragung der Daten, die Auffindbarkeit von Ressourcen, die Identitätsfeststellung der Teilnehmer oder die Bereitstellung eines gemeinsamen Vokabulars [Nagel und Lycklama 2021][B. Otto, Steinbuß u. a. 2019].

¹110 Mio. € Förderung vom Bundesministerium für Wirtschaft und Energie [*Schriftliche Frage an die Bundesregierung im Monat September 2021. Frage Nr. 294* 2021], Projektbeteiligte: BITKOM e. V., BMW AG, SAP SE, Daimler AG, Deutsche Telekom, Siemens AG, Robert Bosch AG, Porsche AG u. v. m. [Catena-X Automotive Network e.V. 2022]

Videobotschaft Dr. Robert Habeck, Bundesminister für Wirtschaft und Klimaschutz [Dr. Robert Habeck 2022]

²3.9 Mio. € Förderung, davon 75 % Förderanteil durch BMDV [BMDV 2022]

³13 Mio. € Förderung [Berlin Institute of Health in der Charité (BIH) 2022]

Es stehen für diese Datenraumkomponenten unterschiedliche Ansätze und Implementierungen mit verschiedenen Reifegraden zur Verfügung. Ebenfalls ist es möglich eigene Implementierungen auf Basis gemeinsamer Protokolle zu erstellen. Die Konfiguration und die Verwendung dieser Komponenten ist kompliziert und erfordert umfangreiches Fachwissen. Auch erlauben der Datenfluss, die Zeitpunkte und Orte der Aufbereitung, Zusammenführung, Standardisierung oder Anonymisierung von Daten Variationen in der Gestaltung mit unterschiedlichen Vor- und Nachteilen.

Diesen und weiteren Gestaltungsfragen stehen Projekte bei dem Aufbau von Datenräumen gegenüber, deren Beantwortung einer sorgfältigen Analyse bedarf. Der manuelle Aufbau solcher Architekturen ist jedoch zeitaufwändig, komplex und nur mit entsprechendem Know-how und Aufwand möglich. Forschende, die mit Datenräumen experimentieren möchten, müssen Kenntnisse zur Infrastruktur mitbringen, um Umgebungen aufzusetzen, Datenraumkomponenten zu installieren und zu konfigurieren. Spezifische Anwenderkenntnisse für die gewählten Implementierungen der Komponenten sind dabei genauso notwendig, wie Infrastrukturkenntnisse der gewählten Umgebung. Der Aufwand steigert sich umso mehr, wenn verschiedene Technologien, Aufbauten und Abläufe direkt miteinander vergleichen und Unterschiede in der Qualität, Performance und Komplexität aus der konkreten Ausführung ermitteln werden sollen.

1.1 Ziel und Lösungsansatz

Im Kontext der Arbeit wird ein technisches Rahmenwerk mit dem Namen *Dataspace Scenario Simulation Framework (DSSIM)* entworfen und umgesetzt, welches eine Beschreibung von Datenraumszenarien erlaubt, diese automatisiert in einer Cloud-Infrastruktur aufbaut, ausführt und protokolliert. Auf Basis der gemachten Erfahrungen und Protokollierung sollen sich Gestaltungsempfehlungen für Anwendungsfälle ableiten lassen. Die Beschreibung umfasst den Aufbau und den Ablauf der Szenarien und erfolgt möglichst einfach und niederschwellig. Diese Niederschwelligkeit ermöglicht einem breiteren Personenkreis das Experimentieren mit Datenraumszenarien.

Es wird erforscht, wie ein gut strukturierter, technologischer Rahmen aufgebaut werden kann, welcher das Experimentieren mit Datenräumen erlaubt. Im Fokus steht dabei die Austauschbarkeit und Erweiterbarkeit von Modulen des Frameworks, um der dynamischen Entwicklung auf dem Gebiet der Datenräume gerecht zu werden. Anpassungen an weiterentwickelte Datenraumkomponenten oder auch die Unterstützung neuer Entwicklungen wird antizipiert und einfach möglich sein. Auch erfolgt die Szenariobeschreibung

unabhängig von der Datenraumtechnologie, damit diese mit verschiedenen Implementierungen simuliert und verglichen werden kann. Protokolliert werden fachliche und technische Ereignisse in dem Szenario und die Auslastung von Systemkomponenten, wie beispielsweise dem Netzwerk, CPU und Arbeitsspeicher. Die Protokollierung kann grafisch aufgearbeitet und kompakt dargestellt werden, damit mit entsprechender Fachkompetenz die Ergebnisse ausgewertet und Gestaltungsempfehlungen erarbeitet werden können.

Zur Validierung der Forschungsergebnisse und um Praxisrelevanz zu gewährleisten, wird ein vereinfachtes Szenario aus einem Projekt des Fraunhofer-Instituts herangezogen. Konkret betrifft dies eine Vorstudie zum *energy data-X* Projekt [Fraunhofer IEE 2021] und den Austausch von Leistungsdaten bei der Stromerzeugung. Bei dieser Vorstudie werden laufend aktuelle Leistungsdaten von Photovoltaikanlagen übertragen, um in Verbindung mit anderen Parametern Prognosen zur Netzlast zu ermitteln. Mithilfe des Frameworks werden Szenarien den Aufbau des verteilten Datenflusses simulieren und den Einfluss der Netzwerkqualität auf den Datentransfer analysieren.

Als ersten Lösungsansatz zur Gesamtarchitektur zeigt Abbildung 1.1 eine Skizze des Frameworks. Das Szenario-Development-Kit (*Szenario-DK*) bildet dabei den Kern und stellt diverse *Controller* zur Steuerung und Überwachung der Simulation zur Verfügung. *Environment Controller* sorgen dafür, dass die Datenraumkomponenten ausgeführt und konfiguriert werden und kontrollieren deren Rahmenbedingungen, wie Netzwerkgeschwindigkeit oder CPU Kapazität. *DataSpace Component Controller* steuern die jeweiligen Datenraumkomponenten und bieten beispielsweise Funktionen zum Veröffentlichen von Datenangeboten, durchsuchen von Datenräumen oder dem Austausch von Daten. Ein *Szenario Controller* aggregiert *Environment Controller* und *DataSpace Component Controller* Funktionen zu verwendenden Anwendungsfällen und ermöglicht eine einfache Komposition des Szenarios. In allen Modulen des Frameworks und der Infrastruktur entstehen Ereignisse und befinden sich Messpunkte, deren Ergebnisse an ein *Szenario Monitoring* übergeben und dort ausgewertet werden.

Die blau gekennzeichneten Module werden im Rahmen der Masterarbeit so implementiert, dass die ausgewählten Szenarien abgebildet und eine Evaluierung des Frameworks erfolgen kann. Eine vollständige Abbildung aller Datenraumfunktionen wird aufgrund des Umfangs für diese Arbeit nicht angestrebt. Der Umfang der Umsetzung soll ausschließlich für eine Validierung des konzeptuellen Ansatzes ausreichen.

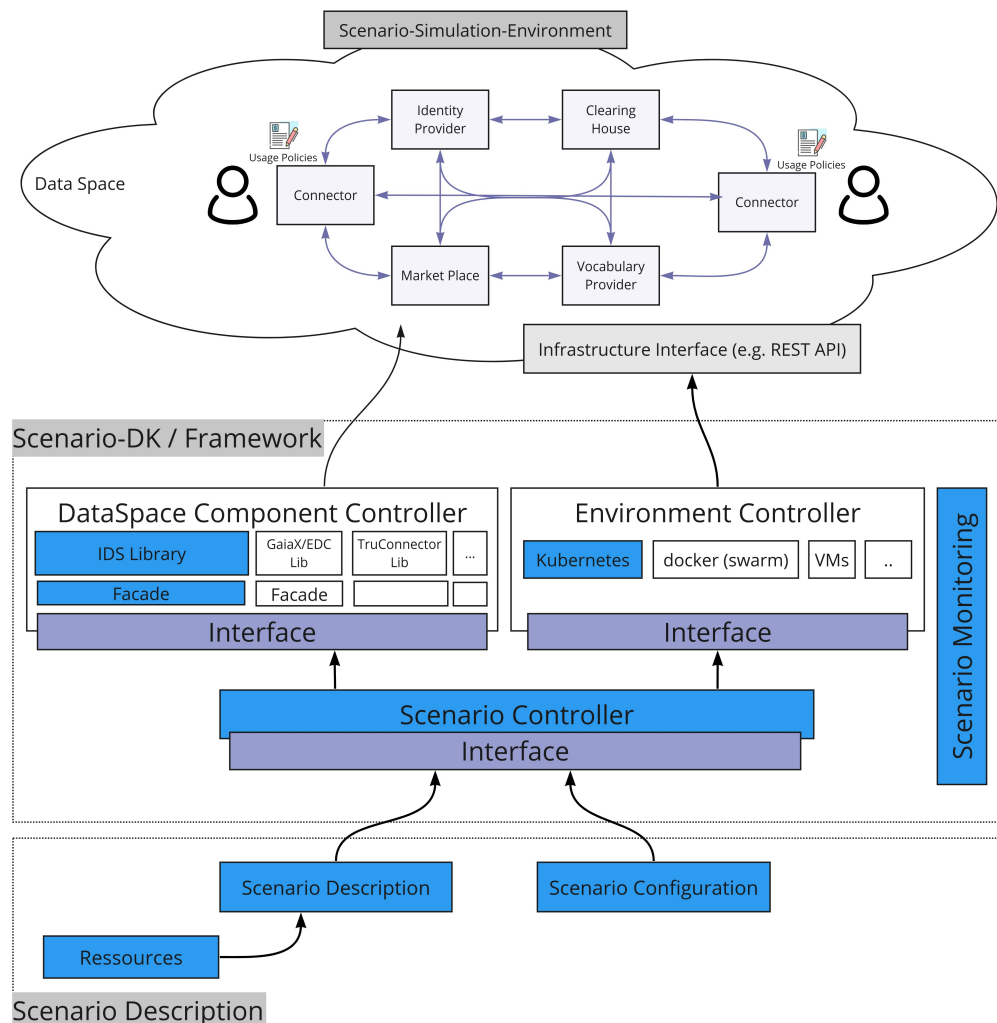


Abbildung 1.1: Lösungsansatz

1.2 Vorgehen / Methodik

Zunächst erfolgte eine grundlegende Auseinandersetzung mit dem fachlichen und technischen Kontext von Datenräumen, aktuell laufenden Projekten des Fraunhofer-Instituts in diesem Themenkontext, deren Fragestellungen und verwendeten Komponenten. Es galt zu verstehen, welche Projektziele verfolgt und wie diese erreicht werden sollen. Dazu wurde der theoretische Rahmen und die praktische Verwendung der Komponenten erlernt. Mit diesem Verständnis der Domäne wurden dann die groben fachlichen und technischen Anforderungen an das Framework erfasst und analysiert.

Hierauf aufbauend erfolgte die Konzeption der Framework Architektur. Dabei wurden die grundlegenden Framework-Module identifiziert und beschrieben, sowie die Kommunikation der Module untereinander. Zur Risikominimierung wurde die technische Machbarkeit und der Aufwand der Umsetzung von zentralen Aspekten geprüft. Hierzu zählte z. B. die Einschränkung von Netzwerkverkehr in den gewählten Technologien, die dynamische Steuerung der Infrastruktur und Datenraumkomponenten oder die kontinuierliche Erfassung von Ereignissen und Systemlast. Dann erfolgte die Implementierung der entworfenen Architektur als Systemdurchstich. Dies bedeutet, dass alle benötigten Teile der Architektur in Minimalausführung existieren und mindestens eine Funktion umgesetzt ist.

Als nächsten Schritt wurde eine erste Beschreibung der durchzuführenden Szenarien erstellt, um festzustellen, welche konkreten Funktionalitäten zur Ausführung und Validierung des Frameworks implementiert sein mussten. Hierauf folgte dann die Spezifikation der Schnittstellen, parallel zur Implementierung der Framework-Module.

Nach der Implementierung erfolgte die Ausführung und Bewertung der Szenarien. Es wurde geprüft, ob die angedachten Ziele mit dem Framework erreicht und Anforderungen umgesetzt werden konnten. Die Evaluation erfolgte an Hand von (a) der Gegenüberstellung der ermittelten funktionalen und nicht-funktionalen Anforderungen mit den tatsächlich durch das Framework bereitgestellten Funktionen und (b) durch eine Betrachtung, wie das Beschreiben und Ausführen der durchgeführten Szenarien bei der Gestaltung der Datenräume Erkenntnisse liefern konnten.

Zum Abschluss erfolgte eine Zusammenfassung und ein Fazit zur technischen Umsetzung und getroffenen Entscheidungen, sowie eine Analyse möglicher Perspektiven.

1.3 Aufbau der Arbeit

Als Einführung wird zunächst die aktuelle Relevanz, der Themenkontext und Problemstellung beschrieben, um darauf aufbauend den gewählten Lösungsansatz und die Ziele der Arbeit zu definieren.

Hierauf folgt im Grundlagenteil eine Auseinandersetzung mit wesentlichen Themen, dessen Verständnis für diese Arbeit Relevanz hat. Hierzu gehören insbesondere digitale Ökosysteme und Datenräume. Diese Grundlagen tragen dazu bei, die Relevanz des Themas und die Ergebnisse einordnen zu können. Darauf folgen Erläuterungen zu Simulationen und Softwarearchitekturen, welche dabei Helfen die Ergebnisse der Arbeit besser definieren zu

können und den Schwerpunkt der Arbeit nachzuvollziehen. Den Grundlagen-
teil schließen Erläuterungen zu den verwendenden Technologien ab.

Im dritten Kapitel werden die fachlichen und technischen Anforderungen
an das Framework beschrieben, gegen die das Framework im weiteren Verlauf
evaluiert wird.

Das vierte Kapitel *Verwandte Arbeiten* analysiert eine Auswahl an ande-
ren wissenschaftlichen Arbeiten, welche in diesem Themengebiet existieren
und prüft Überschneidungen und inhaltliche Auswirkungen auf diese Arbeit.

Hierauf folgen die beiden Hauptkapitel, die sich mit der Beschreibung
des erstellten Frameworks befassen. In Kapitel 5 wird zunächst die allgemei-
ne Architektur des Frameworks, die grundsätzlichen Aufgaben der Module
und modulübergreifende Themen beschrieben. Das Kapitel 6 beschreibt kon-
krete Umsetzungen für die Module und gewährt einen Einblick in zentrale
Problemstellungen und wie diese gelöst wurden.

Nach einer Gegenüberstellung der eingehend dargestellten Anforderun-
gen und denen, durch das Framework bereitgestellten Funktionen widmet
sich das Kapitel 7 der praktischen Anwendung anhand eines ausgewählten,
aktuellen Szenarios. Hierbei wird zunächst der fachliche Kontext erläutert,
die Umsetzung im Framework beschrieben und geprüft welche Erkenntnisse
durch eine Simulation gewonnen werden konnten.

Abschließend wird, nach einer Zusammenfassung des Inhaltes der Arbeit
und einem Fazit, ein Ausblick über eine mögliche Fortführung der Forschung
und Weiterentwicklung des Frameworks gegeben.

Kapitel 2

Grundlagen

In diesem Kapitel soll es ein klares Verständnis zentraler Themen der Arbeit erreicht werden, um auf Basis dieser Definitionen im weiteren Verlauf aufzubauen.

Zunächst wird im ersten Abschnitt 2.1 der Themenbereich der digitalen Ökosysteme näher betrachtet, um den fachlichen Kontext der Datenräume besser einordnen zu können, die Datenraumfunktionen besser zu verstehen und die aktuelle Relevanz des Frameworks aufzuzeigen. Als zentrales Ziel wird dann im Abschnitt 2.2 der Begriff *Simulation* definiert und die Wahl des Begriffes begründet. Grundlagen zu Softwarearchitekturen werden in Abschnitt 2.3 erläutert, um auf die Inhalte des Kapitels 5 *Architektur und Umsetzung* vorzubereiten. Teil der Ausarbeitung ist eine genaue Erläuterung des Frameworkbegriffs mit dem Ziel die Ergebnisse der Arbeit klar zu definieren. Abschließend erfolgt im Abschnitt 2.4 eine Erläuterung zu den verwendeten Technologien und notwendigen Grundlagen.

2.1 Digitale Ökosysteme

‘Das was aus Bestandteilen so zusammengesetzt ist, daß es ein einheitliches Ganzes bildet, nicht nach Art eines Haufens, sondern wie eine Silbe, das ist offenbar mehr als bloß die Summe seiner Bestandteile. Eine Silbe ist nicht die Summe ihrer Laute; ba ist nicht dasselbe wie b plus a, und Fleisch ist nicht dasselbe wie Feuer plus Erde. Denn zerlegt man sie, so ist das eine, das Fleisch und die Silbe, nicht mehr vorhanden, aber wohl das andere, die Laute, oder Feuer und Erde. Die Silbe ist also etwas für sich; sie ist nicht bloß ihre Laute, Vokal plus Konsonant, sondern noch etwas Weiteres, und das Fleisch ist nicht bloß Feuer und Erde oder das Warme und das Kalte, sondern noch etwas Weiteres. Wäre dieses Weitere, was hinzukommt, notwendig auch

wieder ein Element wie die anderen, oder bestände es aus anderem, so würde, falls es selbst ein Element wäre, dasselbe gelten wie vorher; das Fleisch würde aus diesem Element und aus Feuer und Erde bestehen und noch aus Weiterem, was hinzukommt, und so würde es weiter gehen ins Unendliche.' [Lasson 1907][IV. Das begriffliche Wesen]

Diese hier zitierte Einsicht hatte Aristoteles bereits zwischen 348 und 322 v. Chr., als sein Werk 'Metaphysik' entstanden ist. Wahrscheinlich hatte er dabei nicht konkret digitale Ökosysteme im Sinn, aber auch hierauf lässt sich sein bekanntes Zitat übertragen: In digitale Ökosystemen werden durch das Teilen und Verbinden von Daten neue Werte geschaffen, die über den Wert der Ursprungsdaten hinausgehen. Damit wird das Ganze mehr als die Summe seiner Teile. Außerdem entstehen dabei neue Daten, die wiederum Teil des Ökosystems werden und aus denen wieder etwas Neues entstehen kann. In diesem Abschnitt wird ein grundlegendes Verständnis zu digitalen Ökosystemen geschaffen, was Datenräume sind, wie sie zu florierenden Ökosystemen beitragen und wie diese technisch geschaffen sind.

Daten fallen heutzutage kontinuierlich in erheblicher Menge als Nebenprodukt sonstiger Geschäftsaktivitäten an. Isoliert betrachtet schaffen sie kaum Mehrwert und das Potenzial bleibt ungenutzt. Die Menge der gesammelten Daten und die Datenqualität steigt kontinuierlich an und könnte unter anderem die Grundlage komplexer Analysen für Methoden des maschinellen Lernens sein [Kaufmann, Mayer und Niemietz 2021][S.33 ff]. Erst durch den Austausch und das Zusammenführen dieser Daten können Synergien realisiert, Werte geschaffen [Koutroumpis, Leiponen und Thomas 2017][Parmar u. a. 2014][Thomas und Leiponen 2016] und für Innovationen und Gemeinwohl eingesetzt werden. [vgl. M. Otto 2021]

Die Ideen für die Nutzung dieser Daten sind vielfältig. *Gaia-X*, eine europäische Initiative mit dem erklärten Ziel der Schaffung digitaler Ökosysteme [Gaia-X European Association for Data and Cloud AISBL 2023b], listet aktuell 91 umfassende Whitepapers zu konkreten Anwendungsfällen [Gaia-X European Association for Data and Cloud AISBL 2023c]. Diese umspannen diverse Domänen vom Gesundheitssektor, über Agrar, Mobilität, Gesundheit, Bildung oder Energie. Beispielsweise geht es um den Austausch zu Daten vom Betrieb und Wartung von Produktions- oder Energieerzeugungsanlagen um bessere Vorhersageergebnisse aus den *Predictive Maintenance* Modellen zu erreichen und den Anlagenbetrieb zu optimieren oder Stromnetznutzung zu optimieren. Ein weiteres Beispiel ist der Datenaustausch über Verkehr und Mobilität, um ökologischere und effizientere Infrastrukturen zu schaffen oder individuelle Mobilitätsentscheidungen besser treffen zu können.

2.1.1 Herkunft und Definition

Die durch eine digitale Interaktion von Akteuren entstehenden soziotechnischen Systeme werden als *digitale Ökosysteme*, oder auch *digitale Datenökosysteme* bezeichnet. Angelehnt sind diese Begriffe an *Ökosysteme* der Biologie, mit dem der ständige Austausch in verschiedenster Form innerhalb eines Systems, nicht nur zwischen den Organismen, sondern zwischen organischem und anorganischem Bereich bezeichnet wird [Tansley 1935][S. 299].

Übertragen in die Ökonomie hat dies Moore [Moore 1993] mit der Etablierung des Begriffs *business ecosystems* und der Nutzung von Metaphern aus der Biologie bei der Beschreibung des Geschäftsumfeldes. Er schreibt, dass eine isolierte Betrachtung eines Unternehmens als Teil einer Industrie zu kurz greifen würde. Vielmehr wäre es eine gemeinsame Entfaltung ('co-evolve') der Unternehmen mit neuen Innovationen, kooperativer und kompetitiver Arbeiten um neue Produkte zu entwickeln und Kundenbedürfnisse zu befriedigen. Es wird also ein System beschrieben, in dem durch die Interaktion zwischen Entitäten ein Mehrwert geschaffen wird, welcher durch eine individuelle und unabhängige Handlung der Einzelnen nicht hätte entstehen können.

Nach Li et al. [Li, Badr und Biennier 2012] sind *digitale Ökosysteme* ein neues, interdisziplinäres Konzept, für welches es je nach Disziplin unterschiedliche Definitionen zu finden gäbe. Als übergreifend passende wird die von Fu [Fu 2006] formulierte Definition angeführt, welche die ökologischen Aspekte hervorhebt. Dabei ist ein digitales Ökosystem '*eine digitale Umgebung, welche durch digitale Spezies oder digitale Komponenten bevölkert wird, welche Softwarekomponenten, Anwendungen, Services, Wissen, Geschäftsprozesse und -modelle, Training-Module, Vertragsrahmenwerke, Gesetze etc. sein können. [...] Die Infrastruktur eines digitalen Ökosystems unterstützt die Beschreibung, Komposition, Evolution, Integration, die gemeinsame Nutzung und die Verteilung von digitalen Komponenten und Wissen*'. Auffällig ist an dieser Stelle, dass die Infrastruktur bei digitalen Ökosystemen so zentral zu sein scheint, dass diese direkt in einer grundlegenden Definition Berücksichtigung findet. Im Abschnitt 2.1.3 Datenräume soll hierauf nochmals eingegangen werden.

Li et al. [Li, Badr und Biennier 2012] führen neben dieser Definition noch eine ökonomische, technische und allgemeine Definition heran. Jedoch solle ein digitales Ökosystem eine gesamtheitliche und interdisziplinäre Definition haben, welche die digitalen Entitäten identifiziert. Diese Definition soll ebenfalls im Kontext dieser Arbeit verwendet werden:

Digitales Ökosystem. *Ein digitales Ökosystem ist ein selbstorganisierendes, skalierbares und nachhaltiges System, welches aus heterogenen digitalen Entitäten und deren wechselseitigen Beziehungen mit Fokus auf Interaktio-*

nen zwischen Entitäten liegt, um den Systemnutzwert zu erhöhen, Vorteile auszubauen und den Informationsaustausch, Kooperationen und Systeminnovationen zu fördern. [Li, Badr und Biennier 2012]

Zusammenfassend ist festzuhalten, dass es bei digitalen Ökosystemen um den digitalen Austausch von Informationen geht, aus dessen Potenzial einen Mehrwert geschaffen und ein gegenseitiger Nutzen erzeugt wird. Um dieses Potenzial nutzbar zu machen, müssen allerdings Hemmnisse ausgeräumt werden, die der Bereitschaft entgegenstehen Daten zur Verfügung zu stellen. Nach einer Studie von Gelhaar und Otto aus dem Jahr 2020 spielt eine wesentliche Rolle bei der Entscheidung, ob eine Entität Dritten Daten zur Verfügung stellt, die Möglichkeit souverän über die Verwendung der Daten zu entscheiden [Gelhaar und B. Otto 2020]. Hierfür etabliert sich der Begriff *Datensouveränität*.

2.1.2 Datensouveränität

Da Datensouveränität noch ein junges Forschungsfeld ist, hat sich noch keine übergreifende Definition etablieren können. Abhängig von der Anwendungsdomäne finden sich in der Literatur unterschiedliche Standpunkte und die Definitionen gehen je nach Domäne auseinander. Ein guter Einstiegspunkt bei der Begriffsanalyse ist eine Betrachtung der Datensouveränität im Sinne der informationellen Selbstbestimmung. In einer Stellungnahme des deutschen Ethikrats wird wie folgt ausgeführt: *‘vielmehr geht es wesentlich um die Befugnis, selbst zu bestimmen, mit welchen Inhalten jemand in Beziehung zu seiner Umwelt tritt und sich dadurch kommunikativ entfaltet. [...] Eine so verstandene informationelle Freiheitsgestaltung ist gekennzeichnet durch die Möglichkeit, auf Basis persönlicher Präferenzen effektiv in den Strom persönlich relevanter Daten eingreifen zu können’* [Dabrock 2017][S.252f]. Diese Definition ist zwar einschlägig und relevant, jedoch nicht ausreichend für den Kontext der Datenräume. Die Definition des deutschen Ethikrats bezieht sich ausschließlich auf personenbezogene Daten. In Datenräumen können sowohl nicht-personenbezogene Daten, als auch Daten Dritter ausgetauscht werden, an denen ein Unternehmen oder Individuum ebenfalls berechtigtes Interesse bezüglich souveräner Entscheidungen hat. Somit kann die Wahrnehmung der informationellen Selbstbestimmung ursächlich dafür sein, dass der Anbieter personenbezogene Daten nur unter bestimmten Bedingungen weitergeben und somit Datensouveränität ausüben möchte, es kann jedoch auch andere Gründe haben. Hierzu zählen beispielsweise ethische, wirtschaftliche, persönliche oder rechtliche Gründe. [vgl. M. Otto 2021]

Um hier keine weiteren Einschränkungen einzuführen, soll folgende abgeänderte Definition des Ethikrates den Begriff der Datensouveränität für ein klares Verständnis im Rahmen dieser Arbeit einordnen:

Datensouveränität. *Datensouveränität ist die selbstbestimmte Ausübung der Kontrolle über die Erhebung, Speicherung, Verbreitung und Verarbeitung von Daten im Eigentum.*

2.1.3 Datenräume

Auch wenn der Begriff Datenraum (engl. *Data Space*) bereits 2005 in Veröffentlichungen auftaucht (siehe Abschnitt 2.1.3 *Datenräume nach Franklin*), dürfte der Begriff im Wesentlichen durch das Fraunhofer-Institut im Rahmen des *InDaSpace* Projekts¹ 2015 in seiner heutigen Bedeutung geprägt worden sein. Im Jahr 2016 wurde der gemeinnützige Verein *Industrial Data Space e. V.*² gegründet, mit dem Ziel ‘Wissenschaft und Wirtschaft für nachhaltige Lösungen zu vernetzen, die Architektur des Industrial Data Space mitzugestalten sowie zentrales Organ für die Kooperation mit verwandten Initiativen zu sein’ [Fraunhofer-Gesellschaft 2022]. Die IDSA arbeitet bis heute aktiv an einer Standardisierung und Etablierung von Technologien in diesem Kontext. Bevor diese genauer betrachtet werden, soll zunächst rückblickend die initiale Idee der *Datenräume* nach Franklin betrachtet werden.

Datenräume nach Franklin

Der Begriff Datenraum ist nicht vollständig neu. Franklin, Halevy und Maier [Franklin, Halevy und Maier 2005] führen 2005 aus, dass sich über die Zeit verändere, wie Daten gehalten und verarbeitet werden. Heutzutage hätten Systeme meistens nicht mehr ein zentrales Datenbankmanagementsystem (DBMS), dass es Entwicklern erlaubt effizient und konsistent auf die Daten zuzugreifen und sich auf die applikationsspezifischen Problemstellungen zu fokussieren. Sie hätten es eher mit heterogenen, lose gekoppelten Datenquellen zu tun und müssen Lösungen für grundlegende Datenmanagementanforderungen finden. Hierzu gehöre beispielsweise das Suchen und Abfragen, Einhalten von Regeln, Integritätsbedingungen, Namenskonventionen, Zugriffskontrolle oder Transaktionssicherheit.

¹Forschungsprojekt ‘InDaSpace - Industrial Data Space: Digitale Souveränität über Daten’, <https://foerderportal.bund.de/foekat/jsp/SucheAction.do?actionMode=view&fkz=01IS15054>

²2018 umbenannt in International Data Spaces Association (IDSA) [International Data Spaces e. V. 2023]

Datenräume werden von Franklin et al. als neue Abstraktion für Datenmanagement gesehen und der Entwurf und die Entwicklung von DataSpace Support Platforms (DSSPs) werden als Schlüsselement vorgeschlagen. Diese DSSPs sollen Services zur Verfügung stellen, welche es erlauben mit den verschiedenen ungleichartig verwalteten Daten zu arbeiten, ohne sich um die damit einhergehenden, wiederkehrenden Herausforderungen kümmern zu müssen.

Als wesentlicher Unterschied zu traditionellen Datenplattformen wird beschrieben, dass die Daten für Datenräume nicht vorab semantisch integriert sein müssten. Eine semantische Integration würde bedeuten, dass ein gemeinsames Schema erarbeitet werden müsste, in dem die genaue Bedeutung der Daten, die Beziehungen zueinander eindeutig definiert wäre. Datenräume hingegen wären kein Datenintegrationsansatz und setzen diese (i. d. R. aufwändige) semantische Integration nicht voraus. Sie sollten eine Basisfunktionalitäten über alle Datenquellen anbieten, unabhängig davon, wie integriert diese sind.

Einfache Funktionalitäten wie eine Stichwortsuche über alle Inhalte könnten unabhängig von der Datenintegration implementiert werden. Für anspruchsvollere Funktionen, wie Abfragen in relationalen Datenbanken, Data-Mining oder Monitoring könnte man die Daten ausgewählter Quellen inkrementell integrieren. Die gleiche Flexibilität stellen sich Franklin et al. für andere DSSP Funktionen vor. Je größere Garantien für einen Aspekt (Konsistenz, Verfügbarkeit, Transaktionssicherheit, ...) benötigt werden, desto mehr Aufwand müsse investiert werden, um entsprechende Vereinbarungen zwischen den Dateneigentümern treffen und gemeinsame Schnittstellen zu definieren zu können.

Aktuelle Definition Datenräume

Das Positionspapier *Design Principles for Data Spaces* des IDSA definiert Datenräume als *dezentrale Infrastruktur für vertrauenswürdiges Teilen und Austauschen in Datenökosystemen auf Basis gemeinsam vereinbarter Prinzipien* [Nagel und Lycklama 2021][S. 23]. Den Gedanken dieses Positionspapiers folgend besteht ein Datenraum von einer technischen Perspektive betrachtet aus *einer Sammlung an technischen Komponenten, die einen dynamischen, sicheren und nahtlosen Fluss an Daten/Informationen zwischen den Parteien und Domänen ermöglichen. Diese Komponenten können auf unterschiedliche Art und Weise implementiert und in unterschiedliche Laufzeitumgebungen deployt werden* [ebd.][S. 44].

Aus ökonomischer Perspektive kann man Datenräume als *Enabler* für digitale Ökosysteme verstehen: Sie schaffen einen Rahmen, in dem sich Öko-

systeme entwickeln können. Dafür stellen sie eine Infrastruktur bereit und sorgen für die Souveränität der Teilnehmer. Im Gegensatz zu einer zentralen Datenplattform verbleiben Daten bis zum Nutzungszeitpunkt nicht-integriert und beim Datenanbieter.

Die im vorherigen Kapitel erläuterte Datensouveränität steht zusammen mit der Interoperabilität und Vertrauenswürdigkeit im Fokus bei der Schaffung von technischen Lösungen für Datenräume [DSBA 2022][S.6]. Darüber hinaus gibt es weitere, wesentliche Funktionen, die ein Datenraum für eine praktikable Anwendung zur Verfügung stellen muss. Hierzu zählen beispielsweise Wege ein gemeinsames Vokabular zu definieren, um Daten korrekt zu interpretieren, Möglichkeiten den Datenraum zu durchsuchen oder Identitäten der Datenraumteilnehmer gesichert festzustellen [Nagel und Lycklama 2021][S. 29ff][Curry 2015][S. 53].

Zum weiteren Verständnis von Datenräumen soll eine Einordnung zum Kontext des Terminus der *Data Lakes* beitragen. Der 2011 eingeführte Begriff *Data Lake* bezeichnet zentrale Speicherlösungen innerhalb einer Organisation, in welche jegliche Art von Daten in ihrem Ursprungsformat zum Ziel der späteren Wertgenerierung gespeichert werden können. Analog zu den Datenräumen im Sinne dieser Arbeit müssen die Daten dabei nicht zuvor integriert werden. Daher sind diverse Werkzeuge und Techniken notwendig, um Daten zu einem späteren Zeitpunkt integrieren und Wertschöpfung realisieren zu können. Es gibt keine definitiven Vorgaben welche Anforderungen erfüllt und welche Architektur umgesetzt werden muss, damit ein zentraler Speicher als *Data Lake* gelten kann. Ähnlich wie die Begriffe *Cloud* oder *Big Data* wird der Begriff vielseitig verwendet und bezeichnet ein breites Feld an Lösungen. [Woods 2023][Mathis 2017]

Wie im vorherigen Absatz beschrieben, haben *Data Lakes* eine zentrale Zusammenführung aller Daten **innerhalb einer Organisation** zum Ziel und damit stehen auch Themen wie Datensouveränität, Nutzungsbedingungen oder unternehmensübergreifende Nutzungsvereinbarungen nicht im Fokus der Lösungen. Vielmehr könnte man einzelne Inhalte oder Ergebnisse aus den Analysen eines *Data Lakes* zur weiteren Wertschöpfung in einem *Datenraum* teilen. Bereits jetzt gibt es im Kontext der *Data Lakes* den Begriff des *Connectors*, über welche externe Systeme angebunden werden können und Daten sowohl in den *Data Lake* überführt oder abgerufen werden können [ebd.][S. 292]. Denkbar wäre also, dass ein solcher *Data Lake-Connector* entsprechende Datenraumprotokolle verarbeiten kann, oder über einen *Datenraum-Connector* (siehe folgendes Kapitel) angebunden wird. Somit stellen *Datenräume* und *Data Lakes* sich ergänzende Konzepte dar.

Zusammenfassend ist festzuhalten, dass es Parallelen bei dem Verständnis von Franklin et al. zu Datenräumen gibt, was die dezentraler, nicht-

integrierter Haltung von Daten und die Bereitstellung von Funktionen für diese Datenräume betrifft. Datenräume im Sinne der aktuellen Literatur und Initiativen haben keinen Fokus auf die typischen ‘Basisfunktionen‘ eines Datenbankmanagementsystems, sondern fokussieren die Förderung von unternehmens- und domänenübergreifenden Datenaustausch und Datensouveränität. Auch bei den zu simulierenden Datenräume im Sinne dieser Arbeit handelt es sich um Datenräume nach den Ausführungen des IDSA Positionspapiers. Um eine kompakte und aussagekräftige Definition von Datenräumen im Kontext dieser Arbeit unter Einbeziehung der vorhergehenden Überlegungen zu haben, soll folgende zusammenfassende Definition gelten:

Datenraum. *Datenräume sind digitale Ökosysteme zum vertrauenswürdigen und unternehmensübergreifenden Teilen von Daten unter Beibehaltung der Souveränität der Teilnehmer. Sie stellen als dezentrale Infrastrukturen Lösungen zur Verfügung, welche den spezifischen Anforderungen in diesem Anwendungsgebiet Rechnung tragen.*

2.1.4 Aufbau von Datenräumen

Vom Fraunhofer-Institut mitentwickelt und veröffentlicht von der IDSA wurde eine Referenzarchitektur zur Gestaltung von Datenräumen mit dem Titel Reference Architecture Model (RAM). Ziel ist es eine gemeinsame Sprache und eine gemeinsame Basis für die Entwicklung von IDS-konformen Anwendungen zu haben. Diese Referenzarchitektur unterscheidet bei ihrer Betrachtung von Datenräumen folgenden Ebenen: [B. Otto, Steinbuß u. a. 2019]

Business Layer Der *Business-Layer* definiert und kategorisiert die verschiedenen Rollen, die die Teilnehmer in IDS Datenräumen annehmen können. Es spezifiziert grundlegende Interaktionsmuster, die zwischen diesen Rollen stattfinden. Obwohl die Business-Layer eine abstrakte Beschreibung der Rollen bereitstellt, kann es als Grundlage für die anderen, technischeren Schichten betrachtet werden.

Functional Layer Der *Functional-Layer* beschreibt unabhängig von Technologien und Anwendungen die funktionalen Anforderungen an Datenräume und die zu implementierenden Funktionen die daraus entstehen.

Information Layer Der *Information Layer* definiert das IDS Infomodel, welches ein Domänen-agnostisches Datenmodell ist und die Struktur und das

Format von Informationen und Daten, die zwischen Partnern in IDS Datenräumen ausgetauscht werden können, definiert. Es beschreibt die semantischen Zusammenhänge und Beziehungen zwischen den Daten, um sicherzustellen, dass sie einheitlich und eindeutig verstanden werden. Es ermöglicht auch die Integration von Daten aus verschiedenen Quellen und die gemeinsame Nutzung von Informationen zwischen Partnern. [ebd.][S. 39]

Es umfasst eine Vielzahl von Themenbereichen wie z. B. Identitätsmanagement, Datenaustausch, Authentifizierung, Autorisierung, Vertraulichkeit, Integrität, Datensicherheit, Provenienz und Datenschutz. Dabei basiert es auf Standards und Best Practices aus verschiedenen Branchen und Technologien wie z. B. RDF(S) und OWL für semantische Webtechnologien und Standards für die Modellierung von Vokabularen (DCAT, ODRL, etc.) und ermöglicht eine formale und maschinen-interpretierbare Spezifikation von Datenräumen. [Mader u. a. 2023]

Process Layer Der *Process Layer* beschreibt die Interaktionen, die zwischen den verschiedenen Komponenten des International Data Spaces stattfinden. Es bietet somit eine dynamische Sicht auf das Referenzarchitekturmodell und beschreibt die zentralen Prozesse und deren Subprozesse, wie z. B. das *onboarding* von neuen Teilnehmern in Datenräumen oder die Vereinbarung von Nutzungsbedingungen.

System Layer Auf der Systemebene werden die im *Business Layer* spezifizierten Rollen und die in der *Process Layer* definierten Prozesse auf eine konkrete Daten- und Dienstarchitektur abgebildet, die das technische Herzstück des International Data Spaces darstellt.

Da das Framework die konkrete technische Umsetzung von Datenraumszenarien simulieren soll, ist die Systemebene besonders relevant und soll im Folgenden weiter analysiert werden.

IDS Datenraumkomponenten

Nach [Nagel und Lycklama 2021][S. 44ff] werden die technischen Lösungen für die Systemebene über Bausteine zur Verfügung gestellt und können im Allgemeinen nach Tabelle 2.1 *Technische Bausteine eines Datenraums* kategorisiert werden.

Konkret sieht das IDS RAM die folgenden technischen Datenraumkomponenten vor [B. Otto, Hompel und Wrobel 2022][S. 32], wobei es für die jeweiligen Komponenten unterschiedliche Implementierungen geben kann:

Komponente	Beispiele
Interoperabilität	Datenmodelle und -formate, Datenaustausch APIs, Datenherkunft und -rückverfolgbarkeit
Datensouveränität & Vertrauen	Identity Management, Zugriffs- und Nutzungskontrolle, vertrauenswürdiger Datenaustausch
Wertschöpfung	Metadaten & Auffindbarkeitsprotokoll, Datennutzungsabrechnung, Veröffentlichungs- und Marktplatzservices
Sonstige	Systemadaptierung, Datenverarbeitung, Datenrouting und Vorverarbeitung, Datenanalyse, Datenvisualisierung, Workflowmanagement

Tabelle 2.1: Technische Bausteine eines Datenraums nach [Nagel und Lycklama 2021][S. 44ff]

Connector Der *Connector* ist die zentrale technische Komponente, welche die Souveränität gewährleistet und als Schnittstelle zwischen den internen Systemen der Datenraumteilnehmer und dem IDS Ökosystem dient. Je nach Konfiguration sind zentrale Funktionalitäten in dem Connector verortet, wie die sichere Kommunikation, Durchsetzung der Nutzungsbedingungen der geteilten Inhalte, Systemüberwachung und Protokollierung von Transaktionen. Die Funktionalität kann durch *Data Apps* erweitert werden. Diese können aus einem gemeinsamen *IDS App-Store* bezogen werden, oder als Eigenentwicklung durch den Betreiber des Connectors installiert werden.

Identity Provider Ein *Identity Provider* verwaltet die Identitäten eines Datenraums. Dazu gehört das Erstellen, Verwalten, Überwachen und Validieren von Identitätsinformationen für und von Datenraumteilnehmern.

Metadata Broker Der *Metadaten Broker* ist ein Vermittler zwischen Anbieter und Nutzer von Daten. Er ermöglicht Anbietern die Veröffentlichung von Metainformationen, damit Datenangebote von anderen Datenraumteilnehmern aufgefunden und interpretiert werden können. Nutzern wird es ermöglicht Datenangebote des Datenraums zu durchsuchen und sich Informationen zu den Angeboten anzeigen zu lassen. Darüber hinaus unterstützt der *Metadaten Broker* Anbieter und Nutzer bei Vereinbarungen zur Bereitstellung und Nutzung der Daten.

Clearing House Ein *Clearing House* kann dazu genutzt werden alle Transaktionen zu protokollieren. Hierzu gehört das *Clearing* und die Abwicklung

von Finanz- und Datenaustauschtransaktionen. Hierfür werden alle Aktivitäten im Rahmen des Datenaustauschs protokolliert. Nach dem vollständigen oder teilweisen Abschluss werden von den Teilnehmern der Transfer, durch die Übergabe der Transferdetails an das Clearing House, bestätigt. Auf Basis dieser Informationen kann die Transaktion dann verrechnet, Transaktionen nachvollzogen und Konflikte gelöst werden.

App Store Ein *App Store* bietet die Möglichkeit *Data Apps* in Connectoren zu starten, die Aufgaben wie die Transformation, Aggregation oder Analyse von Daten vornehmen. Eine Zertifizierung von vertrauenswürdigen *Apps* ist möglich.

Vocabulary Provider *Vocabulary Provider* ermöglichen es ein gemeinsames Vokabular innerhalb von Datenräumen. Hierzu gehören Ontologien, Datenreferenzmodelle und Metadaten, welche zur Annotation und Beschreibung von Datensätzen genutzt werden können. Die Grundlage des *Vocabulary Providers* bildet das *IDS Information Model*, welches um domänenspezifische Vokabulare erweitert werden kann.

2.1.5 Aktueller Stand und Entwicklung

Im Themenkomplex der Datenräume gibt es aktuell viel Bewegung und verschiedene Unternehmen, Institute, Vereine, Initiativen, Projekte und Technologien, über die an dieser Stelle ein kurzer Überblick geschaffen werden soll.

Roland Fadraný (Chief Operating Officer von Gaia-X) äußerte auf einer Konferenz im September 2022³, dass alleine die *Lighthouseprojekte*⁴ mindestens drei verschiedene Connectoren einsetzen. Nach einem aktuellen Report aus März 2023 existieren derzeit 23 Connector-Implementierungen, welche untereinander nicht durchweg kompatibel sind [IDSA 2022]. Aus diesem Grund haben sich die Organisationen *Big Data Value Association (BDVA)*, die *FIWARE Foundation*, *Gaia-X* und die IDSA in der *Data Spaces Business Alliance (DSBA)* im September 2021 zusammengeschlossen. Zentrales Ziel dieser Organisation ist die Interoperabilität und Portabilität von Datenräumen durch die Harmonisierung der technischen Komponenten herzustellen.

³<https://www.youtube.com/watch?v=2a12YogCYNg#t=3h38m47s> Konferenz GXFS Connect 2022, Podiumsdiskussion zu *Quo Vadis Gaia-X: Welche Innovationspotenziale birgt Gaia-X für die europäische Industrie?*

⁴<https://gaia-x.eu/who-we-are/lighthouse-projects/>: European Production Giganet (EuProGigant), Catena-X, AGDATAHUB, Smart Connected Supplier Network (SCSN), Mobility Data Space (MDS), EONA-X, Structura-X, ELINOR-X

Als erstes gemeinsames Ergebnis wurde im September 2022 das *Technical Convergence Discussion Document* [DSBA 2022] veröffentlicht, um gemeinsame Lösungen weiter zu forcieren. In dem Dokument wird ein Plan zur Umsetzung und gemeinsame Standards beschrieben. Darüber hinaus arbeitet die IDSA an einer überarbeiteten Version der genutzten Protokolle unter dem Namen *DataSpace Protocol*⁵, mit dem Ziel die Interoperabilität der Datenräume zu erhöhen und das Ergebnis als ‘W3C Standard’ oder als Standard einer anderen Organisation bestätigen zu lassen. Am 09.03.2023 wurde ein erster Entwurf der Öffentlichkeit vorgestellt. [IDSA 2023b]

Wie dem Connector Report [IDSA 2023a] zu entnehmen ist, basieren die meisten Implementierungen entweder auf dem DataSpace Connector (DSC) von *soivity*, oder dem *Eclipse* Projekt Eclipse Dataspace Connector (EDC).

DataSpace Connector (DSC) Der DSC ist Open-Source und steht unter Apache 2.0 Lizenz. Er wurde initial vom *Fraunhofer ISST* als Referenzimplementierung entwickelt und wird von *soivity*, einer Ausgründung des Fraunhofer-Instituts aus Dezember 2021, weiter betreut [Sovity 2023]. Der DSC soll einen einfachen Einstieg in das IDS-Ökosystem bieten und es Projekten und Unternehmen ermöglichen, sich mit Datenräumen zu verbinden und Daten auf souveräne Art und Weise auszutauschen. Dabei werden die definierten Nutzungsrichtlinien nicht nur zwischen den IDS-Teilnehmern übertragen, sondern auch direkt durchgesetzt. [Fraunhofer ISST 2021b]

Eclipse Dataspace Connector (EDC) Der Eclipse Dataspace Connector (EDC) kann als Nachfolger vom DSC verstanden werden [Pampus, Jahnke und Quensel 2022]⁶ und wird als Projekt durch die *Eclipse Foundation* gehostet. Ziel ist die Unterstützung der IDS Standards, aber auch relevante Protokolle und Anforderungen aus dem Gaia-X Kontext umzusetzen. Darüber hinaus soll die Erweiterbarkeit für andere Protokolle gegeben sein. Erweiterungen für viele weit verbreitete Cloud-Umgebungen (AWS, Azure, GCP, OTC usw.) sollen bereits von Haus aus zur Verfügung stehen und moderne Datenübertragung ermöglichen. [Eclipse Foundation 2023]

In diesem Abschnitt wurden ausführlich der fachliche Themenkomplex zu Datenräumen dargelegt und aktuelle Entwicklungen aufgezeigt. Neben der Relevanz des Themas und notwendigen grundlegenden Verständnisses wurde

⁵<https://github.com/International-Data-Spaces-Association/ids-specification>, Dataspace Protocol, aktuell Version 0.8

⁶<https://github.com/eclipse-edc/Connector/discussions/1037>, abgerufen am 15.02.2023

deutlich gemacht, warum die Modularität und Flexibilität des Frameworks hohe Priorität hat. Die Aktualität der Referenzen zeigt, wie jung das Forschungsgebiet ist und wie viel Bewegung in den verwendeten Technologien ist.

2.2 Simulation

Dem Titel nach ist das fachliche Ziel des erstellten Frameworks die ‘Simulation von Datenräumen’. Im Folgenden soll kurz erläutert werden was unter einer Simulation verstanden wird, warum dies notwendig ist und warum der Simulationsbegriff treffender ist als der Begriff *Testsystem* für Datenräume.

Der Begriff *Simulation* ist entlehnt aus dem lateinischen *simulāre* ‘ähnlich machen, nachahmen, sich stellen als ob’ [Pfeifer u. a. 2022] und bezeichnet die Nachbildung von realen Systemen, mit dem Ziel Probleme zu lösen oder Einsichten zu gewinnen. Diese (vereinfachte) Nachbildung eines Originalsystems nennt man Modell. [Günther und Velten 2014][S. 24].

Modelle lassen sich mit verschiedenen Techniken und in unterschiedlicher Form realisieren. Ein Modell muss dem Originalsystem im Hinblick auf den Zweck seiner Realisierung hinreichend ähnlich sein und sich für den entsprechenden Anwendungsfall eignen. Übliche Modelle sind physische Modelle (z. B. Modell eines Autos für Messungen im Windkanal), verbales Modell (Beschreibung eines Verhältnisses zwischen *Ich* und *Über-Ich* in der psychologischen Theorie), mathematisches Modell (Differenzialgleichung zur Beschreibung eines Pendels), Computermodell (Programm zur Simulation einer Warteschlange) [Sauerbier 1999][S. 18].

Auch wenn es der Name nahelegen würde, handelt es sich bei dem Ergebnis dieser Arbeit nicht um ein klassisches Computermodell. Sauerbier [ebd.] schreibt in Bezug auf Computermodelle weiter: *Ein Simulationssystem ist ein Programm, welches für Eingangsdaten und Modell mittels eines geeigneten Algorithmus Berechnungen durchführt und die Ergebnisse abspeichert oder ausgibt.* Im Gegensatz zum klassischen Computermodell wird in dieser Arbeit jedoch kein theoretischer Algorithmus eingesetzt, um die Simulation durchzuführen, sondern reale Datenraumkomponenten (z. B. Datenraum-Connector oder Datenverarbeitungsservices) mit *künstlichen* Anteilen (synthetische Daten, Service Attrappen (*Mocks*), simulierte Infrastruktur) in einem Modell der Realität (Szenariobeschreibung) eingesetzt. Die Ausführung in der simulierten Infrastruktur, erlaubt es Parameter des Modells, wie beispielsweise Netzwerkgeschwindigkeit, Paketverlust, Rechenleistung zu beeinflussen.

Ein Test ist laut Duden [Dudenredaktion 2022] ein *nach einer genau durchdachten Methode vorgenommener Versuch, Prüfung zur Feststellung*

der *Eignung, der Eigenschaften, der Leistung o. Ä. einer Person oder Sache*. Da im Fokus jedoch nicht die Eignung oder die korrekte Funktion der Komponenten steht, sondern die Dynamik unterschiedlicher Szenarien eines Datenraums untersucht werden soll ist der Simulationsbegriff treffender und wurde für diese Arbeit gewählt.

2.3 Softwarearchitektur

Wie im Abschnitt 2.3.4 weiter erläutert wird, ist ein *Framework* eine Möglichkeit Softwarearchitektur und -design wiederverwendbar zu machen. Da ein Framework das zentrale Ergebnis der Arbeit ist, bekommt die Softwarearchitektur und das -design damit eine ganz besondere Relevanz. Um dieser Relevanz Rechnung zu tragen, sollen relevante Architekturprinzipien bei der Umsetzung angewendet und geeignete Architektur- und Entwurfsmuster genutzt werden. In diesem Abschnitt soll ein klares Verständnis der wichtigsten Begrifflichkeiten erreicht und im Framework genutzte Muster erläutert werden, damit auf diese im Kapitel 5 Frameworkarchitektur referenziert werden kann.

2.3.1 Softwarearchitektur & -design

Bei der Gestaltung von Software ist in der Regel von Softwaredesign und Softwarearchitektur die Rede. Während Softwarearchitekturen die wesentlichen Strukturen, übergreifenden technischen Konzepte und Entwurfsentscheidungen eines Softwaresystems festlegen [Gharbi u. a. 2018][S.19][„IEEE Standard Glossary of Software Engineering Terminology“ 1990][S. 10], wird unter Softwaredesign, sowohl der Prozess der Definition der Architektur, der Komponenten, Schnittstellen und anderer Charakteristiken eines Systems oder einer Komponente verstanden, als auch das Ergebnis dieses Prozesses [ebd.][S. 25]. Softwaredesign als Ergebnis ist also umfassender als der Architekturbegriff. Er umfasst beispielsweise auch Steuerungslogik, Datenstrukturen, Ein-/Ausgabeformate, Schnittstellenbeschreibungen oder Algorithmen. Softwarearchitektur hingegen ist abstrakterer Natur und definiert die grundlegende organisatorische Struktur der Software.

2.3.2 Architekturprinzipien

Im Laufe der Jahre wurden diverse Architekturprinzipien herausgearbeitet, von denen einige bereits drei bis vier Jahrzehnte alt sind, jedoch bis heute Relevanz genießen. Architekturprinzipien sind Regeln oder Richtlinien, die

bei zentralen Architekturentscheidungen zu beachten sind und für gute und nachhaltige Strukturen in der Software sorgen sollen. Es sind jedoch keine Gebote, dessen Einhaltung zwingend erforderlich ist. Abweichungen können sinnvoll und notwendig sein. Goll [vgl. Goll 2018, S.55ff] listet in seinem Buch über 20 solcher Prinzipien auf, dessen vollständige Aufführung und Erklärung den Rahmen dieser Arbeit weit überschreiten würde. Bekannte Vertreter sind die *SOLID Prinzipien* (Single-responsibility, Open-closed, Liskov Substitution, Interface Segregation and Dependency Inversion Principle), DRY (*Don't repeat yourself*) oder KISS (*Keep it simple*). Als weiterer wichtiger Vertreter bekannter Prinzipien und mit besonderer Relevanz für das Framework, soll das Prinzip der *losen Kopplung* beispielhaft und repräsentativ für andere Prinzipien erläutert werden.

Das Prinzip der *Lösen Kopplung* ist ein bekanntes Softwarearchitekturprinzip, welches erstmals 1979 von Larry Constantine und Edward Yourdon [Yourdon und Constantine 1979][S. 76ff] beschrieben wurde und die Abhängigkeit zwischen verschiedenen Komponenten eines Systems minimieren soll. Es besagt, dass eine Komponente so gestaltet werden sollte, dass sie von anderen Komponenten unabhängig ist und Änderungen in einer Komponente keine unerwünschten Auswirkungen auf andere Komponenten haben sollten.

Ziel des Prinzips ist es, die Flexibilität, Wartbarkeit und Skalierbarkeit von Systemen zu erhöhen. Wenn Komponenten stark miteinander gekoppelt sind, kann die Änderung einer Komponente unerwartete Auswirkungen auf andere Teile des Systems haben und die Komplexität des Systems erhöhen. Dies kann die Wartbarkeit des Systems erschweren und die Implementierung von neuen Funktionen oder Erweiterungen des Systems beeinträchtigen.

Eine lose Kopplung wird oft durch die Verwendung von Schnittstellen und Abstraktionen erreicht. Eine Komponente definiert eine Schnittstelle, über die sie mit anderen Komponenten kommunizieren kann, ohne Details über ihre Implementierung offenzulegen. Andere Komponenten können dann auf diese Schnittstelle zugreifen, ohne sich um die internen Details der Implementierung kümmern zu müssen. Dadurch wird die Abhängigkeit zwischen den Komponenten verringert, und Änderungen in einer Komponente beeinträchtigen nicht unbedingt andere Komponenten.

Das Prinzip der losen Kopplung ist auch wichtig für die Wiederverwendbarkeit von Komponenten, da sie die Möglichkeit bietet, eine Komponente in verschiedenen Kontexten und Anwendungen zu verwenden, ohne dass Änderungen an der Komponente selbst vorgenommen werden müssen. Durch die Verwendung von Schnittstellen und Abstraktionen kann die Abhängigkeit zwischen den Komponenten verringert werden, was die Flexibilität, Wartbarkeit und Skalierbarkeit von Systemen erhöht.

2.3.3 Architektur- und Entwurfsmuster

Neben der Einhaltung relevanter Prinzipien ist die Anwendung geeigneter Architektur- und Entwurfsmuster (engl. *design patterns*) bei dem Entwurf komplexer Systeme relevant. Diese Muster bieten wiederverwendbare Lösungen für Probleme und auch Muster zur Umsetzung von Prinzipien objektorientierter Programmierung.

Der Übergang zwischen Architektur- und Entwurfsmustern ist fließend und man findet in der Literatur unterschiedliche Einordnungen der Muster in diese Kategorien. Grundsätzlich lässt sich sagen, dass ein Architekturmuster ein allgemeiner Ansatz für die Gestaltung der grundlegenden Strukturen und Systeme einer Software ist, während ein Entwurfsmuster eine spezifische Lösung für ein häufig auftretendes Problem im Bereich der Softwareentwicklung darstellt. Architekturmuster zielen darauf ab, die Integrität, Skalierbarkeit und Wartbarkeit einer Software zu verbessern, während Entwurfsmuster die Implementierung bestimmter Funktionen in einer Software erleichtern sollen.

Als ein Beispiel von vielen, soll im Folgenden anhand des *Beobachtermusters* (engl. *observer pattern*) erklärt werden, wie Entwurfsmuster die Umsetzung von Prinzipien unterstützen.

Im *Beobachtermuster* gibt es zwei Arten von Objekten: das beobachtete Objekt (*Subject*) und die beobachtenden Objekte (*Observer*). Wenn das *Subject* geändert wird, informiert es automatisch alle seine *Observer* darüber, sodass diese darauf entsprechend reagieren können. Dadurch wird vermieden, dass die *Observer* aktiv den Zustand des *Subjects* überwachen müssen, um Änderungen zu erkennen. Stattdessen erhalten sie eine Benachrichtigung, wenn eine Änderung auftritt, und können dann entsprechend reagieren. Dadurch wird die Kopplung zwischen dem *Subject* und den *Observern* verringert, da sie keine direkte Abhängigkeit voneinander haben und Änderungen unabhängig voneinander auftreten können. Dies ermöglicht eine lose Kopplung zwischen den Objekten und eine Flexibilität bei der Handhabung von Änderungen, da neue *Observer* hinzugefügt oder entfernt werden können, ohne das beobachtete Objekt zu ändern. [Gamma u. a. 1994][S.293]

Als anerkannteste Sammlung von Entwurfsmustern, mit dem sich die Wissenschaft ausführlich auseinandergesetzt hat [Almadi, Hooshyar und Ahmad 2021] und in dem das *Beobachtermuster* enthalten ist, gilt das Buch *Design Patterns: Elements of Reusable Object-Oriented Software* [Gamma u. a. 1994]. Weithin ist das Buch als Gang of Four (GoF) bekannt und findet breite Anwendung [Almadi, Hooshyar und Ahmad 2021]. Auch dieses Framework und die konkreten Implementierungen des Frameworks bedienen sich dieser Entwurfsmuster. Die 23 Entwurfsmuster werden wie folgt klassifiziert.

Erzeugungsmuster (Creational Patterns) Dienen der Erzeugung von Objekten. Sie entkoppeln das Wissen darüber, welche konkrete Klasse das System verwendet und sie verstecken wie seine Instanzen erstellt und zusammengesetzt werden [Gamma u. a. 1994][S.81]. Anwendung findet in diesem Framework beispielsweise das *Factory*-Muster, in der Szenariokonfiguration um unabhängig vom Szenario unterschiedliche Datenraum-Komponenten-Instanzen zu erzeugen (Abschnitt 6.2 *Scenario-Execution*). Auch das *Singleton* Muster findet Anwendung bei dem *DSSIM Logger* (Abschnitt 6.1 *Core-Modul*) und dem *KubernetesExecutor* (Abschnitt 6.5 *Kubernetes-Controller*).

Strukturmuster (Structural Patterns) Beschreiben wie Klassen und Objekte zusammengesetzt werden, um größere Strukturen zu formen. Sie benutzen Vererbung um Schnittstellen oder Implementierungen zu bilden. Muster dieser Kategorie sind insbesondere für die Interoperabilität von unabhängig entwickelten Klassenbibliotheken sicherzustellen. Klassische Beispiele sind Fassaden (siehe auch Unterabschnitt 6.4.1 *Fassade*), Adapter oder Dekoratoren, die ebenfalls in diesem Framework Anwendung finden. [ebd.][S.137]

Verhaltensmuster (Behavioral Patterns) Modellieren komplexes Verhalten der Software und erhöhen damit die Flexibilität der Software. Insbesondere befassen sie sich mit Algorithmen, der Zuordnung von Verantwortlichkeiten zwischen Objekten und der Art und Weise wie diese miteinander kommunizieren. [ebd.][S.221] Bekannte Vertreter dieser Kategorie ist das oben beschriebene *Beobachtermuster*, das *Besuchermuster* oder die *Templatemuster*. Letzteres wird von der *Environment-Controller* Implementierung *Kubernetes-Controller* genutzt für das Deployment von Datenraumkomponenten (Abschnitt 6.5 *Kubernetes-Controller*).

2.3.4 Framework

Als zentrales Ergebnis dieser Arbeit ist ein *Framework* definiert, welches das Erstellen und Simulieren von Szenarien ermöglicht. Grundsätzlich ist *Framework* zwar ein häufig genutzter Begriff in der Informatik, wird in der Praxis jedoch unterschiedlich angewendet. Um ein klares Verständnis zu schaffen, soll der Begriff und damit auch die Ergebnisse der Arbeit im Folgenden eindeutig definiert werden.

Die passende wörtliche Übersetzung, des aus dem Englischen stammenden Begriffs *Framework*, lautet *Rahmenwerk*.

Wie Ralph Johnson und Brian Foote bereits 1988 im *Journal of object-oriented programming* [Johnson und Foote 1988] festhalten, ist eine der wich-

tigsten Arten der Wiederverwendbarkeit die Wiederverwendung des Designs. Das Design von Anwendungen wird durch seine Komponenten und deren Interaktion miteinander beschrieben. Mithilfe von Frameworks versucht man dieses Design wiederverwendbar zu machen.

Im Gegensatz zu einer vollständigen *Anwendung* ist ein *Framework* nur semi-vollständig. Ein Framework ist ein abstraktes Design für eine bestimmte Art von Anwendung und besteht aus einer gewissen Anzahl an Klassen. Diese ermöglichen mehr als nur die Wiederverwendbarkeit von Teilen des Quellcodes, sondern geben auch das abstrakte Design von Komponenten vor. Die zentrale Stärke eines Frameworks liegt darin, die Erweiterung seiner Komponenten zu ermöglichen.

Bis zur Etablierung von Frameworks war die Verwendung von *Skeletons*⁷ der konventionelle Weg der Wiederverwendung. *Frameworks* erleichtern es die Konsistenz der Komponenten der Anwendung unter verändernden Bedingungen sicherzustellen.

Angelehnt an die Definition von Massol et al. [Massol und Husted 2004][S. 5], welche ebenfalls auf der vorhergegangenen Ausführung basiert, soll folgende Definition des Begriffs *Framework* im Rahmen dieser Arbeit gelten:

Framework. *Ein Framework ist eine Beschreibung eines Systems, welche aus der Definition von Komponenten und deren Kommunikation miteinander besteht. Ein Framework liefert eine wiederverwendbare, gemeinsame Struktur die von verschiedenen Applikationen geteilt wird. Entwickler können die Struktur in ihren Systemen einsetzen, um ihre spezifischen Ziele zu erreichen. Im Gegensatz zu einem Toolkit liefert eine Framework eine kohärente Struktur, anstatt eine einfache Sammlung an unterstützenden Klassen.*

2.4 Verwendete Technologien

In der Beschreibung der Umsetzung in Kapitel 5 *Frameworkarchitektur* und Kapitel 6 *Umsetzung Framework-Module* wird grundlegendes Wissen zu verschiedenen Technologien vorausgesetzt. Dieser Abschnitt gibt einen kurzen Überblick über diese Technologien und erklärt die wichtigsten Begriffe zur besseren Nachvollziehbarkeit dieser Arbeit. Eine Begründung zur Auswahl der jeweiligen Technologien findet sich in Abschnitt 5.5 *Technologien*.

⁷Programmiergerüst, generierter Quellcode der ausgebaut werden kann

2.4.1 TypeScript

TypeScript ist Programmiersprache, die eine statische Typisierung erlaubt und den Fokus auf Produktivität und berechenbares Verhalten legt. Mithilfe eines Compilers wird TypeScript in JavaScript umgewandelt und kann damit plattformunabhängig durch jede JavaScript Ausführungsumgebung (wie z. B. *Node.js* oder einem Browser) ausgeführt werden. [Freeman 2021][S. 35]

TypeScript gewinnt stetig an Beliebtheit und ist in den Jahren 2017 bis 2020 auf Platz vier der meistgenutzten Sprachen auf Github geworden, auf dem es auch im Jahre 2022 noch liegt – hinter JavaScript, Python und Java. [GitHub 2023]

Grundsätzlich können alle JavaScript Bibliotheken auch durch TypeScript verwendet werden, was wegen der Verfügbarkeit einer Vielzahl an Bibliotheken in zentralen Repositories wie npmjs.com einen Beitrag zu schnellen Ergebnissen leistet. Darüber hinaus erlauben Objektliterale und generische Datentypen, insbesondere bei einem experimentellen Framework wie DSSIM, eine hohe Entwicklungsgeschwindigkeit, während moderne *Linter* für gute Codequalität sorgen.

2.4.2 Kubernetes

Als konkrete Umsetzung für das *Environment-Controller*-Modul des Frameworks wurde ein *Kubernetes-Controller* implementiert. Um den Ausführungen hierzu in den Abschnitten 6.5 Kubernetes-Controller und 6.6 Szenario-Monitoring folgen zu können, sind entsprechend verwendete Begrifflichkeiten einzuordnen.

Kubernetes ist ein *Container-Orchestrator* und beschreibt sich selbst als *portable, erweiterbare Open-Source-Plattform für die Verwaltung von containerisierten Aufgaben und Diensten*. Sie ermöglicht eine deklarative Konfiguration, sowie eine automatisierte und verteilte Ausführung der Container. [Kubernetes Projekt 2023d]

Node

Ein Kubernetes Cluster besteht aus einer Anzahl von *Arbeitsmaschinen*, genannt *Nodes*, welche die containerisierten Anwendungen ausführen. Ein Cluster besteht aus mindestens einem Node. [Kubernetes Projekt 2023b]

Control Plane

Das *Control Plane* ist das Steuerzentrum eines Kubernetes-Clusters und trifft globale Entscheidungen (z. B. Scheduling). Es erkennt und reagiert auf Er-

eignisse und steuert die Reaktion hierauf. Dies kann beispielsweise das Starten eines neuen *Pods* (s. u.) sein, wenn die Anforderungen an die definierte Anzahl Instanzen eines *Deployments* nicht erreicht ist. [Kubernetes Projekt 2023b]

Die *Control Plane* besteht dabei aus verschiedenen Komponenten, welche spezifische Anforderungen erfüllen: [ebd.]

- *kube-apiserver*: Der *kube-apiserver* ist die Schnittstelle für das Management von Kubernetes-Ressourcen. Er bietet ein REST-Interface zum Erstellen, Lesen, Aktualisieren und Löschen von Ressourcen wie *Pods*, *Services*, *Deployments* und anderen Objekten. Diese Schnittstelle wird von dem Framework zur Steuerung der Infrastruktur angesprochen, siehe Abschnitt 6.5 *Kubernetes-Controller*.
- *etcd*: *etcd* ist ein verteiltes *Key-Value*-Speichersystem, das die zentrale Datenbank für Kubernetes bildet. Alle Informationen über den Cluster-Status, wie beispielsweise Konfigurationen, Zustände und Ressourcenzuweisungen, werden in *etcd* gespeichert. Das System wird nicht nur in Kubernetes verwendet, sondern auch in anderen Projekten. [etcd Projekt 2023]
- *kube-scheduler*: Der *kube-scheduler* entscheidet, auf welchem *Node* ein *Pod* ausgeführt werden soll. Hierfür werden verschiedene Faktoren wie z. B. Ressourcenbedarf, definierte (Anti)-Affinitäten und oder Datenlokalität berücksichtigt.
- *kube-controller-manager*: Der *kube-controller-manager* ist für die Ausführung der *Controller*-Prozesse verantwortlich, welche im nächsten Abschnitt weiter erklärt werden.

Zusammen bilden diese Komponenten die Control Plane von Kubernetes. Jede Komponente spielt eine wichtige Rolle im Management und in der Orchestrierung der Container, um sicherzustellen, dass Anwendungen reibungslos und effizient ausgeführt werden können.

Controller

Ein *Controller* ist eine Kontrollschleife, welche den Status eines Clusters überwacht und bei Bedarf Änderungen veranlasst. Er kümmert sich so um das Management von Anwendungen und Diensten, die auf einem Cluster ausgeführt werden. Er automatisiert Aufgaben wie Konfiguration, Überwachung und Wiederherstellung, um sicherzustellen, dass die Anwendungen und Dienste in einem gewünschten Zustand ausgeführt werden.

Operator Pattern

Ein *Operator* ist eine spezielle Form eines Controllers nach einem speziellen Muster. Er erweitert mithilfe von *Custom Resources* die Kubernetes API und beinhaltet applikations- oder domänenspezifisches Wissen um den Betrieb der Applikationen zu automatisieren.

Workloads & Pods

Ein *Workload* ist eine Applikation die auf Kubernetes läuft. Sie kann aus einen oder mehreren Komponenten bestehen und wird in *Pods* ausgeführt. Ein *Pod* besteht aus einem oder mehreren Containern, die gemeinsam auf einem Host ausgeführt werden und einen gemeinsamen Netzwerk- und Speicherbereich teilen. In dem Framework werden zwei Arten von *Workloads* verwendet: [Kubernetes Projekt 2023k]

Ein *Deployment* bietet eine deklarative Beschreibung einer Anwendung, die Bereitstellung und Skalierung erleichtert. Hierbei wird der Zielzustand der Anwendung beschrieben und ein Kubernetes-*Deployment Controller* ist für die Durchsetzung verantwortlich. Zum Zielzustand gehören z. B. Definitionen zum verwendeten Container Image, Umgebungsvariablen, verwendeten Ports oder bereitgestellten Speicher.

Ein *DaemonSet* ist ein Kubernetes Objekt, das sicherstellt, dass mindestens eine Pod-Instanz auf jedem Node im Kubernetes-Cluster ausgeführt wird. Es eignet sich für die Ausführung von Systemdiensten oder anderen Hintergrundprozessen, die auf jedem Node im Cluster benötigt werden, wie z. B. Log-Sammler, Netzwerkservices oder Monitoring-Agents.

Darüber hinaus gibt es standardmäßig noch die Workload-Arten *ReplicaSet*, *StatefulSet*, *Job* und *CronJob*, sowie weitere Typen von Drittanbietern.

Netzwerk

Die Netzwerkimplementierung von Kubernetes basiert auf dem Container Network Interface (CNI). CNI⁸ ist Open-Source-Projekt der *Cloud Native Computing Foundation* und wird auch von anderen Container Runtimes eingesetzt [Yilmaz 2021][S.204]. Es besteht aus einer API Spezifikation und Bibliotheken zur Bereitstellung des Netzwerks für die Container. Bei dem Einsatz von Kubernetes bei einem Cloud-Provider sind CNI Plugins in der Regel vorinstalliert und konfiguriert, um optimal in der jeweiligen Infrastruktur zu agieren. Bei einer on-premise Installation können je nach Anwendungsfall unterschiedliche bestehende CNI Plugins oder eigene Implementierung genutzt

⁸<https://www.cni.dev>

werden [Yilmaz 2021][S.208]. Beispiele für bekannte Kubernetes CNI Plugins sind *Flannel*, *Calico*, *Weavenet*, *Contiv*, *Cilium*, *Kube-router* oder *Romana* [Kumar und Trivedi 2021][S.104].

Services & Ingress

In Kubernetes bekommt jeder *Pod* seine eigene IP-Adresse, welche innerhalb des Cluster erreichbar ist. Container innerhalb eines *Pods* teilen sich dabei diese IP und MAC-Adresse und einen Namensraum. Damit können andere Container im gleichen *Pod* über localhost miteinander kommunizieren. Wenn es mehrere *Replicas* (Kopien eines *Pod* zur Skalierung) gibt, ein Service aus mehreren *Pods* besteht oder von außerhalb des Clusters (z. B. dem Rechner des Forschenden) auf Applikationen zugegriffen werden soll, kann dies mit *Services* und *Ingresses* koordiniert werden.

Über *Services* ist es möglich einen oder mehrere *Pods* über einen konsistenten und stabilen Zugriffspunkt zur Verfügung zu stellen, über den Anwendungen zugreifen können. Ein *Service* besteht dabei aus einer Menge an Endpunkten und Regeln, wie Anfragen an die jeweiligen *Pods* weitergeleitet werden. Die Art des *Services* bestimmt, ob und wie der *Service* verfügbar gemacht wird [Kubernetes Projekt 2023g]. Am Beispiel des Typs *NodePort*: Über einen bestimmten Port auf allen IP-Adressen der *Nodes* des Clusters ist der Service erreichbar. Diese *Nodes* leiten die Anfrage an den entsprechenden Empfänger weiter.

Services ohne externe IP-Adresse (Typ *ClusterIP*), welche über HTTP kommunizieren, können über einen *Ingress* verfügbar gemacht werden. Dieser fungiert als Gateway und ermöglicht es externen Clients, auf Basis definierter Routen (Hostnamen und Pfade) auf die internen *Services* zuzugreifen. Abbildung 2.1 visualisiert einen entsprechenden Aufbau mit vorgeschalteten *Load Balancer*.

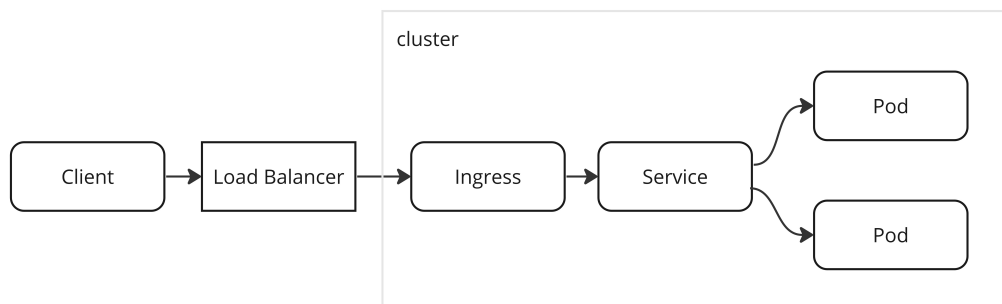


Abbildung 2.1: Service Bereitstellung über Ingress (vgl. [Kubernetes Projekt 2023a])

kubelet

Auf jedem *Node* läuft ein *kubelet*. Das *kubelet* ist eine Applikation (*Agent*), der die Ausführung der Container steuert. Beispielsweise sorgt er dafür, dass alle Container in einem *Pod* ausgeführt werden. Durch die Control Plane initiierte Aktionen werden an die kubelets übergeben, welche dann für die Umsetzung auf dem Node verantwortlich sind.

Sidecar

Der Begriff *Sidecar* bezeichnet ein Designmuster, bei dem ein zusätzlicher Container im gleichen Pod wie die Hauptanwendung ausgeführt wird, um bestimmte Funktionalitäten bereitzustellen. Der Sidecar-Container teilt sich Ressourcen und Netzwerkverbindungen mit dem Hauptcontainer im Pod und arbeitet eng mit ihm zusammen. [Kubernetes Projekt 2023h]

Der Sidecar-Container kann verschiedene Aufgaben erfüllen, wie z. B. Protokollierung, Überwachung, Authentifizierung, Traffic-Shaping, Service-Discovery und vieles mehr. Er ergänzt den Hauptcontainer und stellt sicher, dass dieser nur für seine eigentliche Aufgabe zuständig ist und nicht mit anderen Funktionalitäten belastet wird. Der Sidecar kann auch als eigenständiger Proxy agieren, der den Netzwerkverkehr des Hauptcontainers steuert und optimiert.

Für diese Arbeit ist das Designmuster des *Sidecars* bei der Umsetzung der Netzwerkeinschränkungen im *Kubernetes-Controller* (Abschnitt 6.5.4) und bei den Überlegungen zur Optimierung der Skalierbarkeit des Frameworks im Ausblick (Abschnitt 9) relevant.

Service Mesh

Ein *Service Mesh* ist eine Infrastrukturschicht, welche die Kommunikation zwischen Microservices verwaltet und für eine zuverlässige Übertragung von Anfragen in der komplexen Topologie von modernen cloud-nativen Anwendungen verantwortlich ist. In der Praxis besteht sie aus einer Gruppe von leichtgewichtigen Proxyservern, die zusammen mit den Anwendungen (in Kubernetes beispielsweise als *Sidecar-Container* in jedem *Pod*) ausgeführt werden. Diese Proxyserver sind in der Lage, den Netzwerkverkehr zwischen den einzelnen Microservices innerhalb des Clusters zu kontrollieren, zu überwachen und zu sichern, ohne dass die Applikation hiervon etwas mitbekommt. [Morgan 2017][Khatri u. a. 2020][S. 39].

Multi-Tenancy

Einer der Kerngedanken bei Cloudtechnologien wie Kubernetes ist die Idee, dass die bereitgestellten Ressourcen von mehreren Anwendungen, Teams oder sogar Kunden genutzt werden können. Ziel ist dabei, dass eine Nutzung ohne gegenseitige Beeinflussung möglich ist.

Kubernetes unterstützt dies auf verschiedene Arten, wie z. B. der Verwendung von Namensraum-Isolierung, Begrenzung von Ressourcennutzung und Netzwerkrichtlinien. Durch solche Maßnahmen kann sichergestellt werden, dass jede Anwendung, Team oder Kunde ihre eigenen Ressourcen wie *Pods*, *Services* oder *Netzwerke* erhält und dass diese Ressourcen voneinander isoliert sind. [Kubernetes Projekt 2023e]

Während eine Isolierung über Namensräume schnell hergestellt ist, ist eine sinnvolle Isolierung von Ressourcen, wie CPU Leistung oder physikalischen Netzwerkbandbreite, mit konzeptionellem Aufwand verbunden und kann eine komplexe Aufgabe sein. Eine Simulation in einem Cluster ohne geeignete Maßnahmen und hoher Last durch andere Nutzer, kann zu einem anderen Ergebnis führen als in dem gleichen Cluster ohne Last anderer Nutzer.

Prometheus

Prometheus ist ein eigenständiges Open-Source-System zur Überwachung von Systemen und Anwendungen, welches häufig in Kubernetes-Clustern verwendet wird. *Prometheus* kann Metriken von verschiedenen Quellen (sogenannte *targets*), wie Kubernetes Objekten (z. B. *Pods*, *Deployments*, *Services* und *Nodes*) oder beliebigen anderen Systemen, wie Datenbanken, Applikationen oder HTTP Endpunkten, sammeln. Diese *targets* können entweder direkt, oder über sogenannte *Exporter* integriert werden. Die als Zeitreihe gespeicherten Daten werden in einer, für Zeitreihen optimierten Datenbank, der sogenannten Prometheus Time Series Database (TSDB) gespeichert. Mithilfe einer eigenen Abfragesprache namens *PromQL* können gesammelte Metriken in Echtzeit abgefragt werden. Für die Visualisierung können dann entweder *Console Templates* oder externe Tools, z. B. *Grafana* verwendet werden. [Prometheus Projekt 2023]

Grafana

Grafana ist ebenfalls Open-Source und ein modernes, vielseitiges System zur Analyse und Überwachung von Daten, dass eine Vielzahl an Datenquellen anbinden kann [Chakraborty und Kundan 2021][S. 240].

Über eine benutzerfreundliche Weboberfläche kann *Grafana* verwendet werden. Ein zentrales Element sind Dashboards, welche Grafiken, Diagramme, Tabellen, Texte und andere visuelle Elemente enthalten, die es Benutzern ermöglichen, Daten schnell und einfach zu interpretieren. Die Elemente eines *Dashboards* sind interaktiv und es können Daten gefiltert, Teilbereiche vergrößert oder Datenbereiche verschoben werden.

Kapitel 3

Anforderungen

In diesem Kapitel werden die fachlichen und technischen Anforderungen an die Lösung analysiert und festgeschrieben. Dies erfolgt, um klare Ergebniserwartungen bei allen Beteiligten zu erreichen, zielgerichtete Entwicklung zu ermöglichen und die Ergebnisse gegen das Zielbild evaluieren zu können. Hierfür wurde mit potenziellen Anwendern die bereits in der Einleitung dargestellte Problemstellung diskutiert und Ziele formuliert. Auf Basis der Ziele lassen sich konkrete Anforderungen ableiten.

3.1 Zielformulierung

Es soll ein technisches Rahmenwerk (Framework) geschaffen werden, welches IT-versierten Forschern die Beschreibung von Datenraumszenarien erlaubt. Diese Szenariobeschreibungen sollen in Cloudinfrastrukturen automatisiert aufgebaut und der beschriebene Ablauf durchgeführt werden. Bei der Durchführung der Szenarien sollen Ereignisse und Zustände protokolliert und Funktionen zur Ergebnisanalyse geschaffen werden.

Das Framework ermöglicht

- eine abstrakte und generalisierte Beschreibung von Szenarien in Datenräumen.
- das Experimentieren mit verschiedenen Datenraumtechnologien und -architekturen in verschiedenen Aufbauten und Datenflussvarianten.
- das Ausführen, Steuern und Überwachen der Szenariobeschreibungen.
- die Durchführung eines Szenarios mit verschiedenen Parametern wie z. B. Datensätzen, Datenflüssen, Implementierungen von Datenräumen, Infrastrukturbedingungen.

- eine leichte Erweiterbarkeit für die Unterstützung von alternativen Datenraumimplementierungen und Infrastruktur Umgebungen.

3.2 Anwender

Anwender des Frameworks sind Forschende im Bereich der digitalen Ökosysteme mit grundlegendem IT-Infrastruktur Knowhow und allgemeinen Programmierkenntnissen.

3.3 Fachliche Anforderungen

Aus den aufgeführten Zielen und den in den Grundlagen beschriebenen Rahmenbedingungen lassen sich die in diesem Abschnitt beschriebenen fachlichen Anforderungen ableiten.

Unterstützte Datenraumfunktionen & -technologien Welche konkreten technischen Implementierungen von Datenraumkomponenten unterstützt und welche Funktionen dieser Komponenten durch die Simulation steuerbar sein müssen, ergibt sich aus dem zur Evaluation der Ergebnisse ausgewählten fachlichen Szenario (siehe Kapitel 7 *Evaluation*). In jedem Fall umfasst dies Grundfunktionen wie die Erstellung von Datenangeboten, Vereinbarung von Nutzungsbedingungen und die Übertragung der Daten, um die Funktionalität des Frameworks zu demonstrieren und eine Evaluation vornehmen zu können. Weitere absehbare, zu unterstützende Anwendungsfälle sind Interaktionen mit einem *Broker* und einem *Identity Provider*. Es ist nicht erforderlich alle Funktionen der angebundenen Datenraumkomponenten in Gänze zu unterstützen.

Einfache Nutzbarkeit Anwender des Frameworks sind Forschende mit erweiterten Kenntnissen in der IT und grundlegenden Programmierkenntnissen. Ihnen soll es ermöglicht werden ohne erweitertes Fachwissen zur Infrastruktur und Detailkenntnissen der Datenraumkomponenten einfache Datenräume aufzubauen und zu analysieren. Experten für Datenraumkomponenten und Infrastrukturen soll es ermöglicht werden auch komplexe Szenarien zu beschreiben und durchzuführen.

Spezialisierbarkeit Die Einfachheit der Nutzung soll die Möglichkeiten in der Gestaltung der Datenräume möglichst wenig einschränken. Standardwer-

te sollen überschrieben und technologiespezifische Konfigurationen aufgebaut werden können.

Hohe Erweiterbarkeit und Modularität Da es, wie in den Grundlagen erläutert, viel Bewegung auf dem Markt der Datenräume gibt, soll es mit geringem Aufwand möglich sein, neue Datenraumkomponenten einzusetzen, Funktionalität von Controllern zu erweitern oder auch Simulationen auf anderen Infrastrukturen durchzuführen.

Trennung von Konfiguration und Ablauf Es soll die Möglichkeit geben, die Konfiguration und den Ablauf der Szenarien zu trennen. Unter Konfiguration werden verwendete Technologien, Parameter der Szenarien und Rahmenbedingungen der Infrastruktur verstanden. Ziel ist es, den gleichen Szenarioablauf mit unterschiedlichen Konfigurationen auszuführen und vergleichen zu können. Analog der oben ausgeführten *Spezialisierbarkeit* sollen diese Konfigurationen in den Szenarios überschrieben werden können.

Hardware Simulation Es soll möglich sein, die Ressourcen auf denen die Datenraumkomponenten ausgeführt werden zu beeinflussen, um beispielsweise Datenräume mit schlechten Netzanbindungen oder sehr eingegrenzten Ressourcenverfügbarkeit zu simulieren. Dies ist in der Praxis beispielsweise für die Anbindung von IoT Geräten an die Datenräume wichtig. Es geht im ersten Schnitt nicht darum genaue Prozessoren zu simulieren, sondern nur erste Erkenntnisse auf Geräten mit eingeschränkten Ressourcen zu sammeln.

Parallele Ausführung Es soll möglich sein in einem Cluster mehrere Szenarien gleichzeitig auszuführen. Dabei ist ein gegenseitiger Einfluss bezüglich Performance hinnehmbar.

Auswertung Die gesammelten Informationen während der Ausführung sollen für den Nutzer analysierbar gemacht werden. Entsprechend müssen Werkzeuge angeboten werden, die eine Datenaufbereitung (z. B. Aggregationen, Transformationen oder sonstige Funktionen) und Visualisierung (z. B. Diagramme, Graphen, Tabellen) erlauben.

Simulationsinfrastruktur Als Simulationsinfrastruktur soll mindestens *Kubernetes* unterstützt werden, da es der am weitesten verbreitete Containerorchestrator ist [CNCF 2022][Datadog 2022] und hierfür entsprechende Umgebungen am Institut zur Verfügung stehen.

In diesem Kapitel wurden die wichtigsten fachliche und technische Anforderungen an das Framework analysiert und festgeschrieben, um klare Ergebniserwartungen zu erreichen und zielgerichtete Entwicklung zu ermöglichen. Es folgt eine Analyse zu verwandten Arbeiten, ob diese bereits Lösungen für die gegebene Problemstellung liefern oder wie deren Erkenntnisse einen Mehrwert für die Erstellung des Frameworks haben.

Kapitel 4

Verwandte Arbeiten

Dieses Kapitel beschäftigt sich mit wissenschaftlichen Veröffentlichungen im Themengebiet und soll ein Verständnis für den aktuellen Stand der Forschung vermitteln, um die Ergebnisse dieser Arbeit besser einordnen zu können. Derzeit gibt es keine Arbeiten mit den gleichen Zielen und Inhalten, jedoch zu Teilaspekten inhaltliche Überschneidungen. Grundsätzlich sind Arbeiten zu den Themenbereichen der Simulation von komplexen technischen Systemen und der automatischen Erzeugung von Testumgebungen aufgrund der inhaltlichen Nähe besonders relevant.

Die Arbeiten *Fogify* und *Cloud Security Testbed* haben die größte inhaltliche Nähe, da sie ebenfalls Systeme zur Durchführung von ‘Experimenten’ in der Cloud sind. Im weiteren Sinne ist auch *WorldTravel* relevant, da es um die Bereitstellung komplexer Infrastrukturen geht. Die anderen genannten Arbeiten liefern wertvollen Inhalt, wie im konkreten Datenräume aufgebaut werden.

Fogify Eine große wissenschaftliche Nähe in Bezug auf den Aspekt der automatisierten Infrastrukturbereitstellung und -steuerung hat die Veröffentlichung zum *fogify emulator*, einem Toolset, das die Modellierung, das Deployment und Experimentieren von *fog* und *edge* Testumgebungen erleichtern soll. Es hat zum Ziel Ausführungsumgebungen bereitzustellen, welche realistische Bedingungen und Verhalten von *Fog* Anwendungen darstellen. Da die Ressourcen in diesem Anwendungsgebiet (oftmals *IoT Microservices*) sehr begrenzt sind, ist der Schwerpunkt dieser Arbeit eine möglichst realistische Emulation von definierter Rechenleistung, Netzwerk und Datenverarbeitungsprozessen. Darüber hinaus soll das Toolset eine Skalierung der Testumgebung auf jegliche Größe erlauben und ein Monitoring der emulierten Instanzen, des Netzwerk und auf Anwendungsniveau möglich sein. [Symeonides u. a. 2020]

Für die Beschreibung der Testumgebungen erweitert *Fogify* die docker-compose Spezifikationen. Die Erweiterungen erlauben beispielsweise die Spezifikation von CPU Ressourcen oder Netzwerkbedingungen. Mithilfe eines SDK kann über den *Fogify Controller* die Ausführung gesteuert werden. Eine virtuelle Netzwerkschicht setzt die definierten Rahmenbedingungen mithilfe von linux traffic control (tc) (siehe auch Abschnitt 6.5.4 *Einschränkung des Netzwerks*) um. *Fogify Agents* werden als leichtgewichtige Applikationen auf alle Cluster Nodes deployt und empfangen Befehle des *Fogify Controllers*. Diese setzen die Befehle um und sammeln Informationen über das System zum Zweck des Monitorings. Während der Laufzeit können über den Controller Aktionen auf ihre IoT-Microservices angewendet werden, welcher dann die Ausführung koordiniert und den Cluster Orchestrator, sowie die Fogify Agents steuert. In dem aktuellen Prototyp von Fogify wird *Docker Swarm* als *Container-Orchestrator* (siehe auch Unterabschnitt 2.4.2 *Kubernetes*) eingesetzt. Durch die generische Beschreibung der Umgebung wäre potenziell auch eine Anbindung an Kubernetes oder andere Technologien über Austausch der Komponente *Orchestrator Connector* des Fogify Toolkits möglich. [Symeonides u. a. 2020]

Zwar setzt der Prototyp von *Fogify* einige Funktionen bezüglich der Infrastrukturverwaltung um, welche für die Simulation von Datenraumszenarien in der Cloud notwendig sind, jedoch fehlt die wesentliche Funktion der niedrigschwelligen Szenariobeschreibung (siehe Abschnitt 3.3 *Fachliche Anforderungen*) und auch das Monitoring bietet nur Rohdaten welche an der Programmierschnittstelle bereitgestellt werden und keine Werkzeuge zur Auswertung durch Forschende. Auch da es den Anschein macht, dass die Entwicklung von *Fogify* eingestellt worden ist, da einerseits auf mehrere im Jahr 2021 in github eingestellte Probleme nicht reagiert worden ist und außerdem auch der letzte Commit in dem Repository zum Zeitpunkt der Recherche ein Jahr her ist¹, scheidet *Fogify* auch als Baustein der Lösung aus. Die damit verbundene Forschung zeigt jedoch die aktuelle Relevanz von neuen Lösungen für die Simulation von komplexen Szenarien in der Cloud und Ansätze diese zu lösen.

Cloud Security Testbed Einen weiteren interessanten Lösungsansatz liefert Minna mit *An Open-Source Cloud Testbed for Security Experimentation*, in dem eine Open-Source Cloud Testumgebung beschrieben wird, in der mit einer domänenspezifischen Beschreibungssprache (Domain Specific Language (DSL) als JSON oder YAML Dateiformat) Experiment-Szenarien und Konfigurationen erstellt und ausgeführt werden können. Ziel ist es diese Szenarien

¹<https://github.com/UCY-LINC-LAB/fogify>

auf eine benutzerfreundliche Art und Weise zu erstellen, teilen, anpassen, ausrollen und reproduzieren zu können.

Im Wesentlichen besteht die vorgeschlagene Lösung aus zwei Bestandteilen: Zum einen den DSL Dateien, welche die Konfiguration der Experimente definieren und einem *Testbed Tool*, welches die DSL Dateien verarbeitet und umsetzt. Die Beschreibung in den DSL Files definieren die Konfiguration des Experiments, wie die zu nutzende Plattform, den *Container Orchestrator*, der zu testenden Applikation und eine Beschreibung des Angriffs (*Exploit*). Für die Verarbeitung und Umsetzung werden die eingesetzten Technologien und Werkzeuge auf die Schichten Infrastruktur, Cluster, Container, Anwendung, *Security Tool* und *Exploit* aufgeteilt. Jede dieser Schichten basiert auf unterschiedlichen Werkzeugen zur Realisierung. Für die Infrastrukturschicht sind Infrastructure-as-Code (IaC) Werkzeuge wie *Vagrant* oder *Terraform* vorgesehen, die beispielsweise ein Kubernetes Cluster mit einer *containerd* Laufzeitumgebung bereitstellen. Die Beschreibung der Anwendung und Security Tools der Experimente sollen mithilfe von Plattform-as-Code (PaC) Werkzeugen wie *Helm* umgesetzt werden. *Security Tools* sind dabei Applikationen aus dem Sicherheitskontext wie beispielsweise *Falco*, die zum Aufdecken oder Ausnutzen von Sicherheitslücken verwendet werden. Für die Schicht der *Exploit*-Beschreibungen schlagen die Autoren Attack Specific Language (ASL) oder Meta Attack Language (MAL), Sprachen die zur Beschreibung von Angriffspfaden in anderen wissenschaftlichen Arbeiten entworfen worden sind. Die Aufgabe des *Testbed Tools* würde demnach daraus bestehen, die Definitionen der DSL Dateien in entsprechende Konfigurationen für die eingesetzten Technologien zu übersetzen. Konkret also beispielsweise *Helm-Charts* oder *Terraform*-Definitionen zu erstellen, die dann durch die entsprechenden Tools ausgeführt werden.

Ein offizieller Prototyp für diese Lösung existiert Stand März 2023 noch nicht, ist jedoch als OpenSource Projekt geplant. Damit liefert diese Arbeit zwar grundsätzlich wertvolle theoretische Denkanstöße, dessen praktischer Beweis jedoch noch aussteht. [Minna und Massacci 2022]

WorldTravel Konzeptuell gibt es eine Nähe von SOA (serviceorientierte Architektur) und Datenraumkonzepten, was beispielsweise die lose Kopplung oder die Auffindbarkeit von Services betrifft. Unter diesem Aspekt gewinnt *WorldTravel: A Testbed for Service-Oriented Applications* [Budny, Govindharaj und Schwan 2008], ein Ansatz zur Bereitstellung einer Testumgebung für SOA Anwendungen, an Bedeutung. Diese Testumgebung soll Serviceimplementierungen mitbringen, welche zum Experimentieren und Auswerten von Ideen, Methoden und Umsetzung von Anwendungen in diesem Kontext nötig

sind. Hierzu gehört beispielsweise ein Preis- und Ticketing Service für Flüge oder Lastverfolgung von zugehörigen Anwendungen. Diese Arbeit kann bei der Ausgestaltung von komplexen Testumgebungen helfen und Ideen zur wissenschaftlichen Evaluierung solcher Systeme liefern. Es fehlt jedoch vollständig der automatisierte Aufbau der Umgebung und eine Möglichkeit maschinenlesbare Szenarien zu beschreiben und diese auszuführen. Auch wird Monitoring als sinnvolle und mögliche Erweiterung zur Integration genannt, jedoch nicht implementiert oder weiter in der Ausarbeitung beschrieben.

Weitere Des Weiteren sind Veröffentlichungen im Bereich des Aufbaus von Datenräumen relevant, um Grundlagen zu definieren, die Anforderungen an das Framework zu ermitteln und verschiedene Ausgestaltungsmöglichkeiten zu ermitteln. Hier gibt es Arbeiten zu den Grundlagen von Datenräumen [Franklin, Halevy und Maier 2005][Nagel und Lycklama 2021] und Ansätze Standards zu etablieren [B. Otto, Steinbuß u. a. 2019].

Die Recherche haben gezeigt, dass es in anderen Anwendungsgebieten ebenfalls den Bedarf an automatisch erzeugten und durchgeführten Experimenten von komplexen IT-Infrastrukturen gibt. Keines der Projekte ist jedoch aufgrund der aufgezeigten Gründe für die Nutzung im Problemkontext der Arbeit nutzbar, bieten jedoch Lösungsansätze und Denkanstöße. Auch existieren wissenschaftliche Ausführungen zu dem Aufbau von Datenräumen, welche im Rahmen der Arbeit herangezogen werden können.

Kapitel 5

Frameworkarchitektur

Die Beschreibung der Umsetzung von *DSSIM* teilt sich in zwei Teile. Zunächst wird in diesem Kapitel die grundsätzliche Architektur des Frameworks, dessen Bestandteile und Begrifflichkeiten erläutert. Im Kapitel 6 Umsetzung Framework-Module werden konkrete Implementierungen von Modulen des Frameworks beschrieben.

Das Framework wird im Folgenden in einer Top-Down Methodik beschrieben. Um einen Überblick zu schaffen wird zunächst der Systemkontext dargestellt. Dann folgt eine Zerlegung des Systems in seine Subsysteme (Module genannt) und eine klare Beschreibung der Verantwortlichkeiten dieser. Im darauffolgenden Abschnitt werden die Framework-Module und deren Struktur detaillierter beschrieben. Es folgt eine beispielhafte Erläuterung der Ausführung einer Szenariosimulation.

5.1 Systemkontext

Formal betrachtet ist das Framework nicht selbst ein System. Das Framework stellt einen Rahmen zur Verfügung, der es Forschenden erlaubt Datenraumszenarien zu implementieren. Das System entsteht durch diese konkrete Implementierung und wird im Folgenden *Szenariosimulationssystem* genannt. Die in Abschnitt 2.3.4 erläuterten strukturgebenden Eigenschaften eines Frameworks erlauben es eine valide Systembeschreibung für konkrete Implementierungen zu erstellen.

Die erstellten Szenariosimulationssysteme steuern zum einen die technischen Komponenten eines Datenraums und zum anderen die Infrastruktur auf dem die Datenraumkomponenten ausgeführt werden. Forschende interagieren mit dem System, wenn sie beispielsweise die Szenarioausführung initialisieren, oder die Ausführung, über das Monitoring Modul des Frameworks,

auswerten. Ein Überblick über diesen Systemkontext wird mit der schematischen Darstellung in Abbildung 5.1 gegeben.

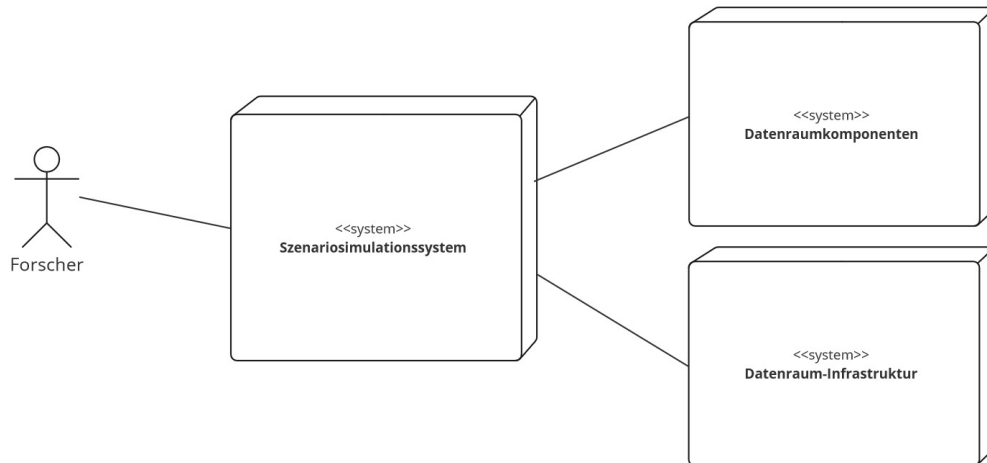


Abbildung 5.1: Systemkontext

5.2 Systemübersicht

Das Szenariosimulationssystem besteht aus den in Abbildung 5.2 *Systemübersicht* dargestellten und im Folgenden genannten Modulen:

- Das **dssim-core** Modul definiert alle Framework Schnittstellen und Datentypen. Es stellt darüber hinaus Hilfsfunktionen für alle Module bereit.
- Das **Scenario-Execution** Modul enthält eine grafische oder textbasierte Benutzerschnittstelle, Szenariobeschreibungen, -konfigurationen und Assets. Es instanziiert den **Scenario-Controller** mit der geladenen Konfiguration und übergibt an diesen die Szenariobeschreibung zur Ausführung.
- Der **Scenario-Controller** ist der zentrale Orchestrator der Szenarioausführung. Er bietet der Szenariobeschreibung grundlegende Methoden und Objekte zur Steuerung der Szenarioausführung an.
- Der **Environment-Controller** ist für die Verwaltung der Simulationsinfrastruktur verantwortlich. Er konfiguriert die Umgebung, erzeugt Instanzen und steuert Ressourcen.

- Der **DataSpace-Component-Controller** bietet Klassen zur Steuerung der Datenraumkomponenten zur Laufzeit an.
- Das **Scenario-Monitoring** sammelt die während der Laufzeit auftretenden Ereignisse und Informationen, führt diese zusammen und bereitet sie auf.

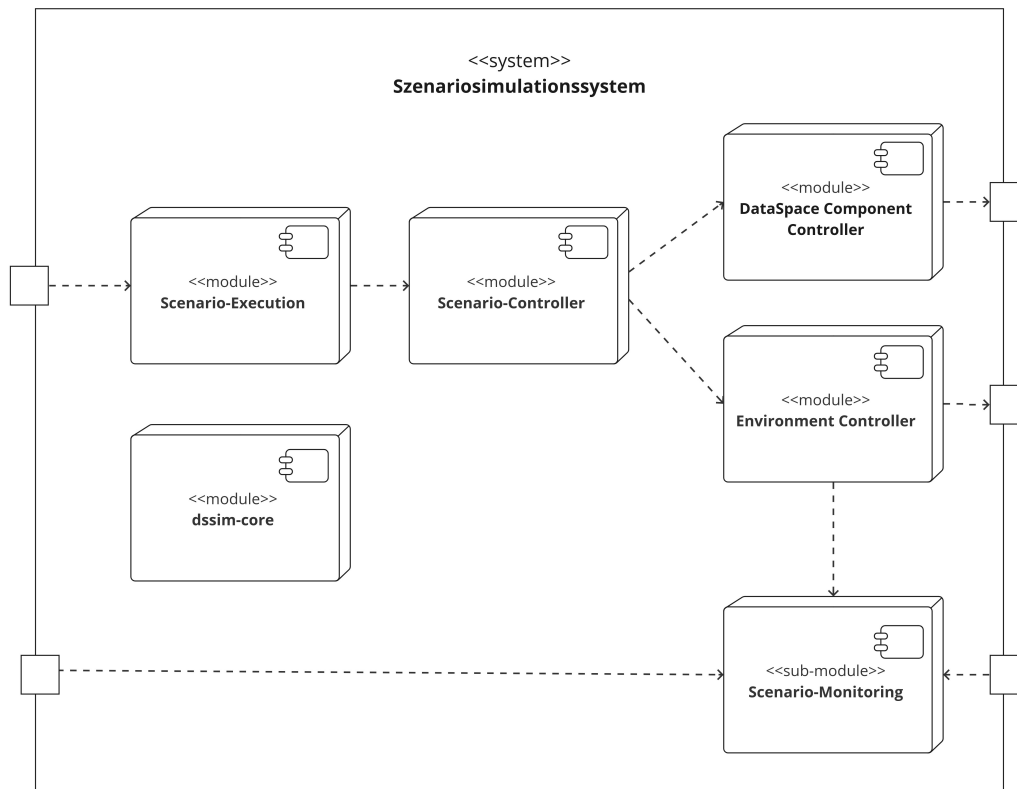


Abbildung 5.2: Systemübersicht

5.3 Module

5.3.1 dssim-core

dssim-core bildet den Kern des Frameworks. Es ist das einzige Modul, von dem es keine verschiedenen Ausprägungen geben soll, sondern nur fortlaufende Versionen. Es definiert für das gesamte Framework die Schnittstellen und das Datenmodell. Darüber hinaus stellt es Hilfsfunktionen bereit, welche von anderen Modulen genutzt werden können.

5.3.2 Scenario-Execution

Die konkrete Ausgestaltung dieses Moduls ist nicht durch das Framework festgelegt und abhängig vom Forschungskontext sind unterschiedliche Umsetzungen möglich. In der Regel besteht das *Scenario-Execution* Modul aus:

- einer grafischen oder textbasierten Benutzerschnittstelle, welche die Auswahl der Szenariobeschreibung und -konfiguration erlaubt und die Ausführung initialisiert.
- den Szenariobeschreibungen, die den Aufbau und den Ablauf eines Datenraumszenarios beschreiben.
- den Szenariokonfigurationen, die Parameter für die Ausführung definieren (z. B. in welcher Umgebung und mit welcher *Environment-Controller*-Implementierung die Simulation durchgeführt wird, welche Datenraum-Connector-Variante gestartet werden sollen und ob die Infrastruktur für das Monitoring des Szenarios hochgefahren werden soll).
- für die Simulation notwendige Assets. Assets sind beispielsweise Daten, mit denen ein Szenario durchgeführt werden soll oder *Dashboard*-Definitionen für das Monitoring Modul.

Als Referenzimplementierung findet sich im Projekt eine Variante, die es erlaubt beschriebene Szenarien über die Kommandozeile auszuwählen, eine Konfiguration zu laden, weiter anzupassen und auszuführen. Einfachere Umsetzungen dieses Moduls könnten auch als Unit-Test erfolgen, oder als Programm welches nur ein Szenario kennt und dieses direkt ausführt. Vorstellbar wären hier auch umfangreichere Implementierungen, die es erlauben analog des *An Open-Source Cloud Testbed for Security Experimentation* (siehe Kapitel 4 *Verwandte Arbeiten*) Szenarios in einer domänenspezifischen Sprache in JSON oder YAML Syntax zu laden und mit dynamisch angepassten Konfigurationen zu starten (siehe hierzu auch Kapitel 9 *Ausblick*).

5.3.3 Scenario-Controller

Der *Scenario-Controller* ist ein Modul, welcher den Zugriff auf die *DataSpace Component Controller* und die *Environment-Controller* kapselt und orchestrierte Funktionen für Szenarien zur Verfügung stellt. Wenn beispielsweise ein Szenario den Start einer Datenraumkomponente wie dem *Connector* anfordert, wird über den *Environment-Controller* das Deployment angetreten und abgewartet. Danach wird der DataSpace-Component-Controller erzeugt und eventuell notwendige Operationen auf diesem durchgeführt (z. B. Anmeldung am *Identity Provider*). Dann gibt die Methode der Szenariobeschreibung ein Objekt zurück, über die die Steuerung der erzeugten Daten-

raumkomponente möglich ist. Damit *orchestriert* er das Zusammenspiel der anderen Framework-Module.

5.3.4 DataSpace-Component-Controller

Der *DataSpace-Component-Controller* ermöglicht die Steuerung der Datenraumkomponenten während der Szenarioausführung. Dies findet in der Regel über HTTP-REST Endpunkte statt, ist jedoch nicht darauf eingeschränkt. Denkbar wäre auch eine Steuerung über Telnet oder andere Protokolle.

Mit den Funktionen, die dieses Modul zur Verfügung stellt, kann man beispielsweise einen Connector dazu veranlassen Datenangebote auf einem Broker zu veröffentlichen, den Datenraum zu durchsuchen oder Nutzungsbedingungen mit anderen Datenraumteilnehmern zu vereinbaren.

Damit ein Szenario mit unterschiedlichen Datenraumimplementierungen (z. B. DSC und EDC) ausführbar ist, muss ein gemeinsames Interface die Funktionen abstrahieren. Entsprechende Implementierungen müssen das Delta zwischen der speziellen Technologie und der abstrahierten Ebene schließen. Dies kann bei Datenräumen mit stark abweichenden Funktionsweisen eine Herausforderung sein. Im Rahmen dieser Ausarbeitung wurde ein pragmatischer und inkrementeller Ansatz gewählt und nur für die tatsächlichen benötigten Funktionen der aktuell eingesetzten Technologien ein gemeinsames Interface definiert. Bei Aufnahme weiterer Technologien oder Nutzung weiterer Funktionen muss das *DataSpace-Component-Controller*-Interface erweitert oder überarbeitet werden. Um zum jetzigen Zeitpunkt eine optimale Ausarbeitung der vollständigen Abstraktionsebene über Datenraumtechnologien hinweg zu erreichen, ist eine umfassende Analyse der verschiedenen Datenmodelle notwendig. Auch ist fragwürdig, ob diese bei der Geschwindigkeit der aktuellen Entwicklungen auf diesem Feld eine lange Halbwertszeit hätte.

5.3.5 Environment-Controller

Das *Environment-Controller*-Modul ist für die Steuerung der Deployments in der vorgesehenen Infrastruktur und der Konfiguration der Infrastruktureigenschaften zuständig. Hierzu gehören u.a.:

- die Installation der Datenraumkomponenten und anderen Szenariobestandteilen (z. B. starten eines Dataspace Connectors oder eines einfachen Datenservices / einer Datenbank)

- die Initialkonfiguration der Komponenten (z. B. festlegen der Anmeldedaten für die API, damit der *DataSpace Component Controller* sich Authentifizieren kann.)
- Sicherstellen der Erreichbarkeit durch Netzwerk- und Routingkonfiguration (z. B. Festlegung von Hostnamen, Einrichten von *Ingress*)
- die Weiterleitung von Informationen an das *Szenario Monitoring* (z. B. Log-Dateien oder Informationen über CPU Auslastung)
- die Steuerung der Infrastrukturressourcen (z. B. Limitierung CPU Geschwindigkeit, Limitierung der Netzwerkgeschwindigkeit, Höhe der Paketverluste)

Das Modul besteht aus einer Implementierung für den *Environment-Controller*, welche allgemeine Umgebungsoperationen zur Verfügung stellt und den Implementierungen für die jeweiligen Instanzen der Datenraumkomponenten (z. B. *ConnectorInstance*, *BrokerInstance*).

5.3.6 Scenario-Monitoring

Das *Scenario-Monitoring* erlaubt die Überwachung der Szenarioausführung und die Aufarbeitung der Ergebnisse. Hierzu gehört zum einen die Sammlung von Informationen über die Infrastruktur (z. B. Ressourcenauslastung einer Node eines Kubernetes-Clusters), Logs der Datenraumkomponenten und Log-Events der DSSIM Framework-Module. Zum anderen gehört die Aufbereitung und die Visualisierung der Informationen dazu.

Das Monitoring Modul ist dabei naturgemäß eng mit dem *Environment-Controller* Modul verbunden. Das Environment-Controller Modul muss dafür sorgen, dass die Logs und Systeminformationen der laufenden Instanzen eines Datenraums zentral gesammelt und für eine Auswertung zur Verfügung gestellt werden. Dies kann bei einem Kubernetes-Cluster diametral anders funktionieren als bei einem Environment-Controller, der beispielsweise virtuelle Maschinen für jede Datenraumkomponente startet. Trotz der Nähe zum Environment-Controller ist das Monitoring Modul nur lose gekoppelt und soll damit austauschbar bleiben.

Wie Abbildung 5.2 *Systemübersicht* zeigt, kommuniziert das *Monitoring*-Modul über zwei Schnittstellen über die Systemgrenze hinaus: Einerseits stellt es eine Benutzeroberfläche für die Forschenden zur Verfügung, auf dem eine Analyse des Szenarios erfolgen kann. Andererseits müssen die Logdateien aus der Ausführungsumgebung in das Modul geladen werden.

In der Referenzimplementierung für das Framework wurde dies mithilfe von Promtail, Prometheus, Loki und Grafana realisiert. Weitere Details hierzu in Abschnitt 6.6.

5.4 Konfiguration

In der Anwendungsarchitektur wird zwischen zwei Arten von Konfigurationen unterschieden:

Benutzer- und Umgebungsspezifische Konfiguration Zu diesen Einstellungen gehören insbesondere Benutzernamen, Kennwörter oder Hostnamen, welche je nach ausführende Benutzer und Umgebung abweichen. Dies sind sensible Informationen, welche nicht geteilt werden dürfen.

Alle benutzer- und umgebungsspezifischen Konfigurationen werden über Umgebungsvariablen gesteuert. Wie die Umgebungsvariablen gesetzt werden, ist dem Anwender überlassen. Für einen einfachen Umgang gibt es die Möglichkeit eine `.env`-Datei in den Ausführungsordner zu legen, welche beim Anwendungsstart automatisch ausgelesen und dessen Inhalt der Anwendung als Umgebungsvariablen zur Verfügung gestellt wird. Diese sollte aus Sicherheitsgründen nicht in eine Sourcecodeverwaltung eingecheckt werden.¹

Szenario Konfiguration Hierzu gehören alle Einstellungen, die ein Szenario definieren, wie beispielsweise die verwendeten Varianten und Versionen von Datenraumkomponenten. Diese können in der Szenariobeschreibung oder separat in der Szenariokonfiguration vorgenommen werden. Diese Einstellungen können über die Sourcecodeverwaltung mit anderen Forschenden geteilt werden.

Die Szenariokonfiguration wird über den Konstruktor in die *Scenario-Controller* Klasse *injected* und bietet neben einfachen *Properties* auch *Factory Methoden* für die Erzeugung von Datenraumkomponenten oder Typ-Informationen über die zu instanziiierenden Controller.

Es kann Mischformen der Konfigurationstypen geben. Als Beispiel sei die Definition eines Containerimages für die in einem Szenario zu verwendenden Connectoren genannt, zusammen mit den zugehörigen Zugangsdaten für die Container Registry. Das folgende Listing zeigt die Realisierung einer solchen

¹Bei einer Sourcecodeverwaltung durch **git** mit Aufnahme von `.env` in die Datei `.gitignore` im Stammverzeichnis.

Mischform, bei dem das verwendete Containerimage im Klartext als Sourcecode geteilt und eingecheckt werden kann, sensible Benutzerdaten jedoch aus der Umgebungskonfiguration gezogen werden.

```
1 {  
2   image:  
3     'registry.gitlab.cc-asp.fraunhofer.de/dssim/  
4     kubernetescontroller/ids-dataspace-connector  
5     -dssim:7.1.0',  
6   pullSecret: {  
7     [process.env.SCECTR_CONNECTOR_IMAGE_HOSTNAME!]: {  
8       username: process.env.  
9         SCECTR_CONNECTOR_IMAGE_PULL_USERNAME!,  
10      password: process.env.  
11        SCECTR_CONNECTOR_IMAGE_PULL_PASSWORD!,  
12    }},  
13 }
```

Quellcode 5.1: Szenarioeinstellungen mit Umgebungsvariablen

5.5 Technologien

Die zentralen Technologien wurden bereits in den Grundlagen erläutert. Die Hauptgründe für die Entscheidung für diese sollen hier zusammenfassend aufgeführt werden:

- TypeScript wurde aufgrund der aktuellen Beliebtheit, Plattformunabhängigkeit, Vielfalt an verfügbaren Bibliotheken und Entwicklungsgeschwindigkeit gewählt.
- Kubernetes wurde aufgrund der großen Verbreitung und Verfügbarkeit am Institut gewählt.
- DSC und EDC wurden gewählt, weil sie zum einen die von IDSA und *Gaia-X* befürworteten Technologien sind und viele andere Konnektoren auf ihnen basieren.

5.6 Versionierung

Aufgrund der starken Modularisierung und der Verteilung auf unterschiedliche Projekte ist eine eindeutige Versionierung der Module notwendig. Diesbezüglich hält sich das Framework strikt an das Konzept *Semantic Versioning 2.0.0* [Preston-Werner 2013]. Dieses Konzept definiert eine dreiteilige Versionsnummer mit der Semantik *MAJOR.MINOR.PATCH*. Bei sogenannten

breaking changes, also Änderungen, bei denen die API nicht abwärtskompatibel ist, muss eine neue *Major* Version eingeführt werden. *Minor* Versionen sind funktionale Änderungen an einem Projekt, die abwärtskompatibel sind und *Patch* Versionen sind fehlerbeseitigende Korrekturen, die ebenfalls abwärtskompatibel sind. Bei Einsatz des Frameworks ist also darauf zu achten, dass die Major Version von allen Modulen übereinstimmt.

In diesem Kapitel wurde die grundlegende Architektur des DSSIM Frameworks und dessen Bestandteile erläutert. Hierbei wurden die Verantwortlichkeiten der einzelnen Module dargestellt und deren Struktur beschrieben. Die Top-Down Methodik ermöglichte es, einen klaren Überblick über den Systemkontext und die einzelnen Module zu schaffen. Im nächsten Kapitel werden konkrete Implementierungen von Modulen des Frameworks beschrieben.

Kapitel 6

Umsetzung Framework-Module

Im vorhergehenden Kapitel wurde ein Überblick über den Gesamtzusammenhang gegeben und alle (abstrakten) Komponenten des Frameworks beschrieben und deren Aufgaben und Kommunikation untereinander erläutert. In diesem Kapitel werden konkrete Implementierungen der Komponenten erläutert. Es wird gezeigt, wie die Architektur innerhalb der Komponenten aussieht, welche Herausforderungen bei der Implementierung mit den gewählten Technologien existieren und wie die zugewiesenen Aufgaben erreicht werden. Die entstandene Implementierung kann als Referenzimplementierung für Erweiterungen durch neue Module des Frameworks verstanden werden.

Jedes dieser Module ist in einem eigenen TypeScript Projekt organisiert, hat ein eigenes Git-Repository und ist jeweils einzeln zur besseren Nutzung als *npm*-Modul über *npmjs.com* verfügbar gemacht worden. Bei der Beschreibung der Projektstrukturen wird nur auf spezifische Dateien und Ordner des Frameworks eingegangen, allgemeine Konfigurationsdateien (z. B. *tsconfig.json*, *package.json*, *.eslintrc.cjs*) und Ordner (z. B. *build*, *node_modules*, *.git*) von TypeScript Projekten finden keine Berücksichtigung.

6.1 Core-Modul

Projektname: dssim-core
DSSIM-Framework-Modul: Core
Repository: <https://github.com/FraunhoferIEE/dssim-core>
npm-Link: <https://www.npmjs.com/package/dssim-core>

Die strukturgebenden Definitionen für das Framework werden zentral in dem *dssim-core* Projekt definiert und durch andere Framework-Module importiert. Außerdem stellt das *dssim-core* Modul Hilfsfunktionen bereit, die

von allen Modulen benötigt werden. Die Ordnerstruktur des Projekts wird in Abbildung 6.1 erläutert.

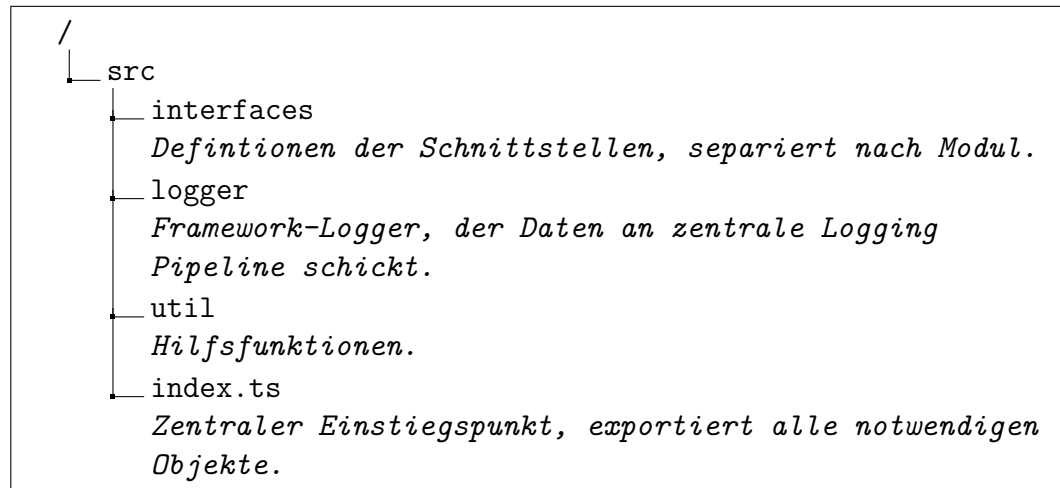


Abbildung 6.1: Struktur dssim-core Projekt

6.1.1 Architektur

Das aus dem Quellcode des *dssim-core* Modul generierte Klassendiagramm findet sich im Anhang unter Abschnitt 1 *Klassendiagramm dssim-core* und lässt die Grundidee aus der Einleitung in Abbildung 1.1 *Lösungsansatz* und die Grundstruktur in Abbildung 5.2 *Systemübersicht* erkennen.

Es ist außerdem ersichtlich, dass zum einen alle Abhängigkeitspfeile in eine Richtung zeigen. Diese gehen vom Abstrakten (der Szenariobeschreibung) zu den Details (konkreten Steuermethoden der Controller). Da der Graph die definierten Schnittstellen darstellt und die Implementierungen diese umsetzen, ist von einer Abhängigkeitsumkehr zu sprechen und das Architekturprinzip *Dependency Inversion* wird realisiert.

6.1.2 Hilfsfunktionen

Da alle Framework-Module das *dssim-core* Modul für die Schnittstellen importieren, eignet sich dieses Modul ebenfalls für Funktionen, die allen Modulen zur Verfügung stehen müssen. Hierzu zählt aktuell eine gemeinsame Logger Klasse, eine Methode um Dateien Base64 zu codieren, eine Methode zum Warten auf Zustände und eine Verzögerungsfunktion. Aufgrund der Relevanz soll kurz auf den Logger eingegangen werden.

Der Logger ist als *Sigleton* implementiert, der Konstruktor damit *private* und die Instanz kann mit der statischen Methode *getInstance()* abgerufen werden. Der Logger basiert auf der populären¹ *winston* Bibliothek mit einer Erweiterung für die Übergabe der Protokolle an einen *Loki* Server. Bei Start der Applikation wird zunächst nur in die Konsole geloggt. Sobald das *Monitoring* Modul des Frameworks hochgefahren ist, wird über die *addLokiOutput* Methode der *Loki* Endpunkt hinzugefügt und alle Logausgaben des Frameworks an die Logging Pipeline geschickt. Details zur Logging Pipeline finden sich in Abschnitt Abschnitt 6.6 *Scenario-Monitoring*.

6.2 Scenario-Execution

Projektname: dssim-scenarios
DSSIM-Framework-Modul: Scenario-Execution
Repository: <https://github.com/FraunhoferIEE/dssim-scenarios>
npm-Link: <https://www.npmjs.com/package/dssim-scenarios>

Als *Scenario-Execution* Modul wurde eine interaktive Konsolenanwendung mit dem Namen *dssim-scenarios* auf Basis von der *Inquirer.js*² Library geschrieben. Die Struktur des Projekts wird in Abbildung 6.2 erläutert.

6.2.1 Szenariokonfiguration

Eine Konfiguration eines Szenarios kommt durch eine konkrete Ausprägung eines Objekts des Typs *ScenarioConfiguration* zustande. Über dieses Objekt werden Eigenschaften gekapselt, die Variationen in der Ausführung eines Szenarios erlauben. Insbesondere werden *factory*-Methoden definiert, die eine Erzeugung von Objekten beschreiben. So beschreibt eine *environmentControllerFactory* welcher Typ eines Controllers erzeugt wird und mit welchen Parametern dies geschieht. Eine *defaultConnectorInstanceFactory* beschreibt die Erzeugung eines Connectors, welcher erzeugt werden soll, wenn eine Szenariobeschreibung nur die Erzeugung eines Connectors anfordert, jedoch keinen konkreten Typ angibt. Ein *ConnectorControllerType* Property der Szenariokonfiguration legt den zu verwendenden *Dataspace-Component-Controller* fest. Damit können Forschende sowohl spezifische Szenariobeschreibungen anfertigen, die speziellen Funktionen eines bestimmten Connectors einsetzen.

¹laut npmjs.com 10 Mio. wöchentliche Downloads in KW 14 / 2023

²<https://github.com/SBoudrias/Inquirer.js>: Die Library vereinfacht die Gestaltung von Kommandozeilenanwendungen und den Ablauf der Dateneingabe.

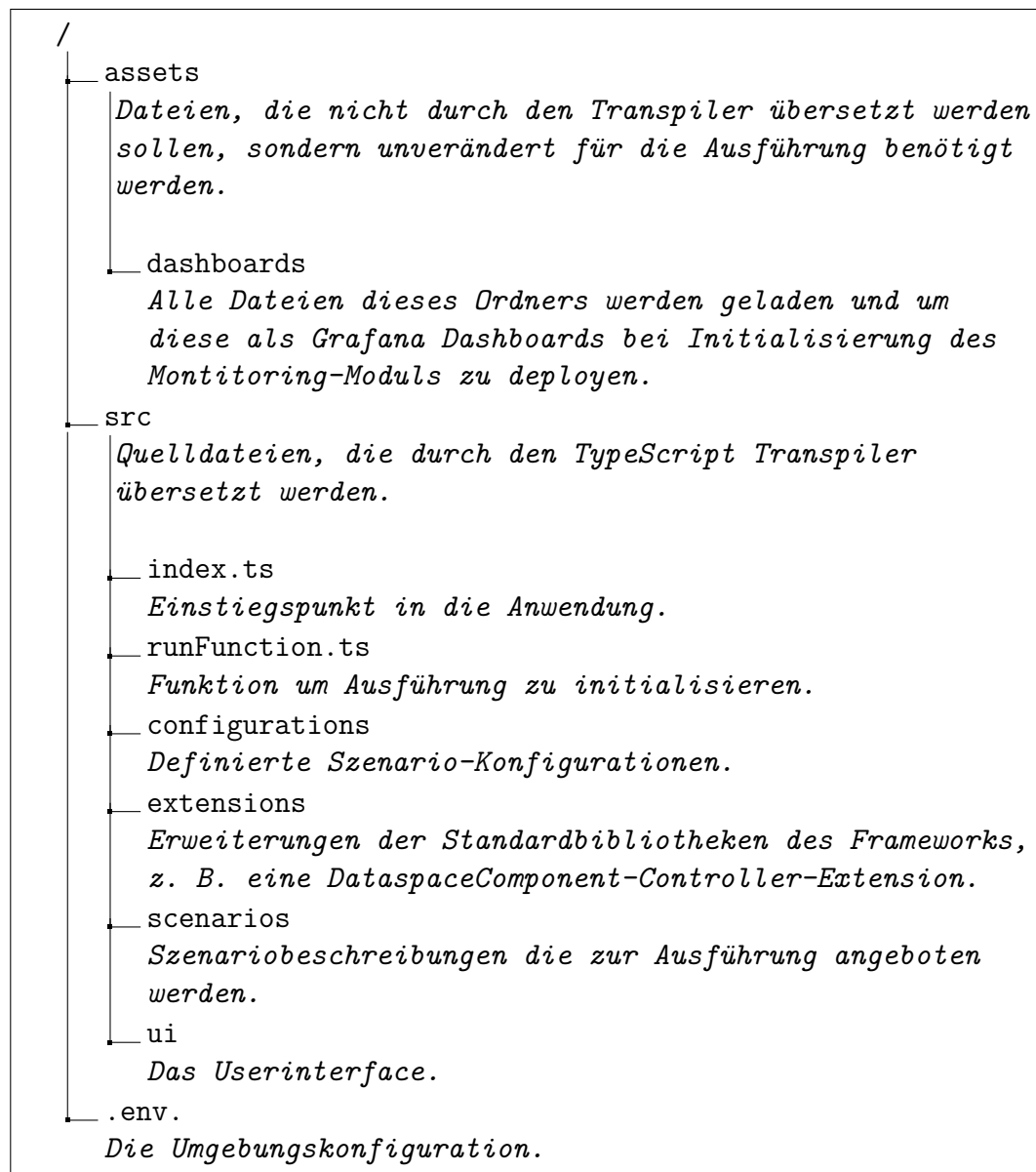


Abbildung 6.2: Struktur Scenario-Execution Projekt

zen, als auch durch die Konfiguration der Ausführungsumgebung bestimmte Instanzen. Quellcode 6.1 demonstriert eine solche Konfiguration.

Warum für die Connector-Instanz eine Factory Methode genutzt und für den Connector-Controller die Nennung der Controller Klasse ausreicht, liegt wie folgt begründet: Die Konstruktor der Connector-Instanzen können stark variieren, da diese sehr unterschiedliche Informationen für ihre Instanziierung benötigen. Der *ScenarioController* müsste damit alle konkreten Typen ken-

nen und wissen wie diese erzeugt werden, was nicht gewünscht ist und gegen diverse Prinzipien verstoßen würde. Die Connector-Controller hingegen benötigen in der aktuellen Ausprägung alle die gleichen Informationen (z. B. Endpunkt, Passwort, Benutzername). Aufgrund der Gleichartigkeit kann der *ScenarioController* die Instanzen selbst erzeugen. Unterabschnitt 6.3.1 *Generische Factory Methoden* geht auf diese Thematik weiter ein.

```

1 {
2   // Name Konfiguration
3   name: 'IDS configuration with daps',
4   // Factory fuer Environment-Controller
5   environmentControllerFactory: async () =>
6     await KubernetesController.createInstance(true, undefined
7     ),
8   // Factory fuer Datenraum-Connector
9   defaultConnectorInstanceFactory: deploymentName =>
10     new DSCInstance(deploymentName, 'username', 'password', [
11     {
12       image:
13         'registry.gitlab.cc-asp.fraunhofer.de/dssim/dssim-
14         kubernetes-controller/ids-dataspace-connector-
15         dssim:7.1.0',
16       pullSecret: pullSecret,
17     },
18   ]),
19   // Connector-Controller Klasse
20   ConnectorControllerType: DSCController,
21   // Zu nutzender Identity Provider
22   identityManagement: {
23     IdentityProviderControllerType: DapsController,
24     InstanceType: DapsInstance,
25   },
26 },

```

Quellcode 6.1: Beispiel für eine Szenariokonfiguration

6.2.2 Szenariobeschreibung

Die Beschreibung des Szenarios ist eine Klasse, die das Interface *Scenario* implementiert. Dieses verpflichtet zur Definition einer *run*-Methode, welche einen *Scenario-Controller* zur Laufzeit *injected*³ bekommt. Der *Scenario-Controller* als zentraler Einstiegspunkt erlaubt es Datenraumkomponenten zu initialisieren und zu steuern. Dies ist ein typisches Beispiel für *Inversion of Control*: Nicht die Applikation (das *Scenario-Execution* Modul oder

³im primitivsten Sinne des Wortes ‘injected’: als Funktionsparameter

die Szenariobeschreibung) steuert wann die Ausführung erfolgt, sondern das darunterliegende Framework.

Der Quellcode 6.2 zeigt, wie in einer Szenariobeschreibung eine neue Instanz eines Connectors erzeugt wird. Von dem *Scenario-Controller* bekommt man ein zusammengesetztes Objekt, über das sowohl die Infrastruktur mithilfe des *Environment-Controllers* gesteuert werden kann, als auch Datenraum-Operationen über den *DataSpace-Component-Controller* veranlasst werden.

```

1 // Starten eines Connectors vom Typ der Konfiguration und
  // abwarten des Starts
2 const connector = await scenarioController.startConnector('
  admin', 'password', 'consumer');
3
4 // Festlegen der Netzwerkgeschwindigkeit (Steuerung
  // Infrastruktur)
5 await connector.instanceController.setNetworkControl({
6   bandwidth: {
7     value: 20,
8     unit: 'kbit',
9   },
10  delay: {value: 200, unit: 'ms'},
11 });
12
13 // Erstellen eines Artifakts (Steuerung Datenraumkomponente)
14 const artifactId = await connector.componentController.
  createDatabaseArtifact(
15   {
16     name: 'database table',
17   },
18   'jdbc:postgresql://applik-d207.iee.fraunhofer.de:5432/
    transferdata',
19   'Postgres',
20   'connector',
21   '12345',
22   'select * from wind'
23 );
24
25 // Veroeffentlichung des Artifakts (Steuerung
  // Datenraumkomponente)
26 await connector.componentController.createOfferForArtifact.
  createOfferForArtifact(
27  artifactId,
28  {
29    name: 'Free PI Value',
30    license: 'http://opendatacommons.org/licenses/odbl/1.0/',
31    sovereign: 'https://www.iee.fraunhofer.de/',
32    start: new Date(2022, 1, 1),
33    end: new Date(2025, 1, 1),

```

```

34   },
35   {
36     name: 'English number format. Dot as Decimal seperator.',
37     mediaType: 'plain/text',
38   },
39   {
40     name: 'Cool Math-Numbers Catalog',
41   },
42   new UnrestrictedPolicy()
43 );

```

Quellcode 6.2: Erzeugen eines Connectors

Um Szenarien mit unterschiedlichen Connector-Implementierungen ausführen zu können oder nicht auf das verallgemeinerte Interface für Connectoren limitiert zu sein und direkt mit dem spezifischen Controller des Connectors zu arbeiten, ist es notwendig die Konfiguration im Szenario zu überschreiben. Hierfür ist *startConnector* eine überladene und generische Funktion. Listing 6.3 zeigt, wie direkt auf die spezifische API des DSC zugegriffen und eine *Camel*⁴ Route angelegt wird, obwohl andere Connector Implementierungen und die gemeinsame Schnittstelle solche Konzepte nicht vorsehen.

```

1  // Expliziter Start des DSC mit DSC-Controller
2  const dscConnector = await scenarioController.startConnector<
    DSCInstance, DSCController>('username', 'password', 'test'
    , new DSCInstance(deploymentName, 'username', 'password',
    {
3      image:
4      'registry.org/project/ids-dsc:7.1.0',
5      pullSecret: {...},
6    }), DSCController);
7
8  // Direkte Nutzung der spezifischen DSC Bibliothek
9  dscConnector.componentController.connectorApi.routesService.
    createRoute(...);

```

Quellcode 6.3: Überschreiben der Standardkonfiguration

Solche direkten Aufrufe der API in der Szenariobeschreibung sollten jedoch vermieden werden, um nicht zu stark an die Version des Connectors gekoppelt zu sein. Hierfür können Forschende *Dataspace-Component-Extensions* erstellen, also Klassen die den entsprechenden *Dataspace-Controller* erweitern. Diese können in unterschiedlichen Szenarien verwendet und Änderungen an der Implementierung zentral vorgenommen werden. Auch können über *Extensions* die Standardimplementierung der Controller überschrieben werden,

⁴Apache Camel ist ein Open-Source-Integrationsframework, das es ermöglicht, verschiedene Systeme und Anwendungen miteinander zu verbinden. Der DSC nutzt Camel für den Datenfluss von Artefakten.

indem die Extension in der Szenariokonfiguration angegeben und bestehende Szenarien mit veränderter Funktionalität ausgeführt werden.

6.2.3 Konnektivität

Für die Steuerung der Szenarien müssen die Endpunkte der Datenraumkomponenten (i. d. R. *HTTP-REST*-Endpunkte, aber nicht darauf limitiert) von dem laufenden Prozess erreicht werden können.

Dafür müssen die in der Szenariobeschreibung definierten Hostnamen aufgelöst werden können. Dies kann entweder durch einen Eintrag in dem verwendeten DNS-Server für diese Hostnamen erfolgen, oder die Namensauflösung muss entsprechend beeinflusst werden. In dem Testaufbau hat sich sowohl die Installation eines lokalen DNS-Servers, als auch eine manuelle Manipulation der *hosts*-Datei als praktikabel erwiesen.

6.2.4 Skalierung

Die Ausführung der Szenariosteuerung findet in dem lokal laufenden *node.js*-Prozess statt. Diese Architekturentscheidung hat einen Einfluss auf die Konnektivität und Skalierbarkeit der Simulation. Aufgrund der aktuell noch einfachen Natur der durchgeführten Szenarien stellt dies aktuell, bei Einhaltung der Prinzipien zur Gestaltung von Szenarien (s. u.), noch kein Problem dar. Wenn Szenarien mit einer sehr hohen Anzahl von Datenraumkomponenten ausgeführt werden sollen, kann eine Umgestaltung oder Weiterentwicklung dieses Moduls notwendig werden. Eine kurze Ausführung zu dieser Gestaltungsvariante findet sich in Abschnitt 9 *Dezentrale Szenariosteuerung*.

Um zu vermeiden, dass die Szenario steuernde Einheit auf dem lokalen Rechner des Forschenden zum Engpass wird und die Messergebnisse verfälscht, sollten die beiden folgenden Prinzipien bei der Gestaltung der Szenarien eingehalten werden:

- Keine netzwerkintensive Kommunikation zwischen der *dssim* Instanz und den Datenraumkomponenten. Die Bereitstellung der Daten durch die Provider sollte aus dem Cluster heraus stattfinden. Zwar ist es möglich, dass die zu veröffentlichenden Daten durch den Controller direkt an den Provider geschickt werden, jedoch führt dies bei einer hohen Anzahl von Providern und großen Datenmengen schnell zu einem Engpass. Eine Provisionierung der Provider durch Datenbanken oder Datenservices innerhalb des Clusters und (skalierende) Instanzen hält die Kapazitäten der steuernden Einheit frei.
- Keine rechenintensiven und Input/Output Operationen innerhalb der Szenariobeschreibung. *Node.js* ist grundsätzlich *single threaded* [Open-

JS Foundation 2022a]. Rechenintensive Operationen können zu einer Blockade des Szenarioablaufs führen. Zur Auslagerung von rechenintensiven Operationen ist es möglich, diese in *worker threads* [OpenJS Foundation 2022b] auszulagern oder ebenfalls als Service im Cluster zu starten. Der *Environment-Controller* bringt hierfür bereits die Möglichkeit mit, containerisierte Anwendungen im Cluster mit der Methode *deployContainerizedService* zu starten. Anzumerken ist, dass *worker threads* nicht bei Input/Output intensiven Operationen helfen.

6.3 Scenario-Controller

Projektname: dssim-scenario-controller
DSSIM-Framework-Modul: Scenario-Controller
Repository:
<https://github.com/FraunhoferIEE/dssim-scenario-controller>
npm-Link:
<https://www.npmjs.com/package/dssim-scenario-controller>

Obwohl der *Szenario-Controller* Einstiegspunkt bei der Szenariobeschreibung ist und eine zentrale Rolle im Framework spielt, ist es in der aktuellen Umsetzung das leichtgewichtige Modul des Frameworks. Seine Aufgabe besteht darin die wichtigsten Objekte und Methoden für die Steuerung des Ablaufs bereitzustellen. Für Datenraumkomponenten stellt der *Szenario-Controller* beispielsweise Objekte zur Verfügung, die über ein Attribut die Steuerung der Infrastruktur erlaubt und über ein anderes Attribut Zugriff auf fachliche Datenraum-Operationen zur Verfügung stellt. Falls für die Ausführung von Operationen ein Zusammenspiel von verschiedenen Controllern notwendig ist, wird diese durch den *Szenario-Controller* orchestriert. Ein Beispiel hierfür wäre, dass zunächst eine Datei über den *Environment-Controller* zur Verfügung gestellt wird, welche danach durch einen *Component-Controller* verwendet wird.

6.3.1 Generische Factory Methoden

Die Methode *startConnector* (Methodensignatur siehe Listing 6.4) soll aufgrund seiner zentralen Relevanz und stellvertretend für andere Methoden dieses Moduls näher betrachtet werden. Über diese Methode kann eine fertig konfigurierte Instanz einer Datenraumkomponente in der Simulationsumgebung erzeugt werden. Wie in den Anforderungen in Kapitel 3 beschrieben, soll es in der Szenariobeschreibung einerseits möglich sein einen in der Konfiguration definierten Connector zu nutzen, oder einen konkreten Typen

festzulegen. Ziel ist es, die Möglichkeit zu schaffen heterogene Szenarien zu beschreiben (ein Datenraum mit unterschiedlichen Datenraumtechnologien), als auch ein Szenario mit unterschiedlichen Konfigurationen auszuführen.

Die Methode erzeugt und gibt ein Objekt vom Typ *Connector* zurück, das aus einer Kombination von *ConnectorInstance* und *ConnectorController* besteht. Sie akzeptiert eine optionale Instanz von *I* und die optionale Angabe einer Klasse (in TypeScript beschrieben als Konstruktor-Signatur) von *C*. Wenn diese Parameter nicht angegeben werden, werden sie durch den Aufruf von *defaultConnectorInstanceFactory* und *ConnectorControllerType* aus dem Konfigurationsobjekt initialisiert.

Die Methode erstellt dann eine neue Instanz von *ConnectorController*, indem sie entweder den bereitgestellten Konstruktor verwendet oder den Standardkonstruktor aufruft. Schließlich wird eine Instanz von *Connector* zurückgegeben, die sowohl die Instanz von *ConnectorInstance* als auch die Instanz von *ConnectorController* enthält.

Allgemein lässt sich die *startConnector* Methode dem Entwurfsmuster der Gruppe Erzeugungsmuster *Factory* zuordnen. *Factory Methoden* zeichnen sich dadurch aus, dass ein Interface für die Erstellung eines neuen Objekts zur Verfügung gestellt wird. Die konkrete Instanziierung erfolgt dann über entsprechende Unterklassen. Genutzt wird dieses Entwurfsmuster insbesondere in Frameworks, um die konkrete Erzeugung von Objekten zu kapseln. Erzeugende Applikationen (hier die Szenariobeschreibung) wissen dann nur wann die Objekte erzeugt werden, aber nicht wie. [Gamma u. a. 1994][S. 107]

Eine Besonderheit hier ist, dass die Standarderzeugung mit Methodenparametern überschrieben werden kann. Eine Methode mit unterschiedlichen Parametern wird in der objektorientierten Softwareentwicklung klassischerweise mit *Überladung* realisiert. Auch *TypeScript* unterstützt das Überladen von Methoden, jedoch soll nach den offiziellen Best Practices [Microsoft 2022] hierauf zugunsten der besseren Lesbarkeit verzichtet werden, wenn eine Realisierung ebenfalls über optionale Parameter realisierbar ist.

Eine weitere Spezialität in dieser Methode ist die Verwendung von *Generics*, welche die Typen der Attribute *controller* und *instanz* des Rückgabeobjekts definieren. Zweck ist es, bei einer Szenariobeschreibung mit festgelegten Controller-Typ die Verwendung nicht auf das allgemeine Interface zu limitieren, sondern typsicher und ohne *cast* den speziellen Controller verwenden zu können. Über den speziellen Controller stehen den Forschenden sämtliche Funktionen zur Verfügung und es können sehr technologiespezifische Szenarien definiert werden.

Um ein einheitlicheres Interface zu haben, wäre die Angabe einer Klasse für den Parameter *connectorInstance* und das Überlassen der Erzeugung des Objekts durch den *Environment-Controller* ebenfalls vorzuziehen gewe-

sen. Jedoch würde dadurch die Szenariobeschreibung die Kontrolle darüber verlieren, mit welcher konkreten Version (bei dem *Kubernetes-Controller*: mit welchem Container-Image) die Instanz erzeugt wird. Eine Abgabe des Container-Images als Parameter ist ebenfalls nicht vorteilhaft, weil nicht gesagt ist, dass alle Instanzen immer aus Container-Images erzeugt werden. In einer Simulationsumgebung ohne Container (z. B. mit vollständig virtualisierten Maschinen oder ohne Virtualisierung) könnte dies auch über *rpm-Pakete* o. ä. geschehen. Aus diesem Grund wurde die Entscheidung getroffen, die Instanziierung der Szenariobeschreibung oder der Factory-Methode des Konfigurationsobjekts zu überlassen.

```

1  async startConnector<
2    I extends ConnectorInstance,
3    C extends ConnectorController
4  > (
5    username: string,
6    password: string,
7    hostname: string,
8    connectorInstance?: I,
9    ConnectorControllerType?: new (
10     hostname: string,
11     username: string,
12     password: string
13   ) => C
14   ): Promise<Connector<I, C>> {
15     ...
16   }

```

Quellcode 6.4: Methodensignatur zum Starten eines Connectors

6.4 IDS Controller

Projektname: dssim-ids-controller
DSSIM-Framework-Modul: DataSpace-Component-Controller
Repository:
<https://github.com/FraunhoferIEE/dssim-ids-controller>
npm-Link:
<https://www.npmjs.com/package/dssim-ids-controller>

Sub-Repositories:

Projektname: dsc-lib
Repository: <https://github.com/FraunhoferIEE/dsc-lib>
npm-Link <https://www.npmjs.com/package/dsc-lib>

Projektname: ids-broker-lib
Repository: <https://github.com/FraunhoferIEE/ids-broker-lib>
npm-Link <https://www.npmjs.com/package/ids-broker-lib>

Das Projekt *dssim-ids-controller* soll *Dataspace-Component-Controller* für die Datenraumkomponenten des IDS Ökosystem beinhalten. Hierzu gehören unter anderem Controller für den DataSpace Connector (DSC), Controller für den Identity Provider *omejdn* oder einen Controller für *IDS Metadata Broker*. In der aktuellen Implementierung werden der Connector und der Broker unterstützt.

Es ist ebenfalls im Rahmen der Arbeit ein Projekt mit dem Namen *dssim-edc-controller* angelegt worden, welches zur Simulation von Datenräumen genutzt werden soll, welche den Eclipse Dataspace Connector (EDC) als zentrale Datenraumkomponente einsetzt. Da das *dssim-ids-controller* Projekt jedoch weiter fortgeschritten und komplexer ist, nimmt die folgende Beschreibung ausschließlich Bezug hierauf.

6.4.1 Fassade

Die jeweiligen Controller implementieren das, durch das Framework definierte, Interface der entsprechenden Datenraumkomponente und nutzen separate Bibliotheken für die Kommunikation mit den Instanzen (siehe Abbildung 6.3 *Vereinfachte Architektur DSC-Controller*).

Damit diese Bibliotheken vielseitig nutzbar bleiben und auch nicht nur für das Framework wertstiftend sind, bilden sie exakt die API der Datenraumkomponente ab und werden separat veröffentlicht. Die Orchestrierung der Aufrufe zu abstrakteren Funktionen findet in dem Controller statt. Damit wird die Nutzung der spezifischen Datenraumkomponente abstrahiert und über verschiedene Implementierungen verallgemeinert. Man könnte argumentieren, dass die Klasse daher vielmehr nach dem GoF Muster *Adapter* [Gamma u. a. 1994][S. 139] (auch: *Wrapper*) bezeichnet werden sollte, da es die Bibliotheken kompatibel zu dem Framework Interface macht. Jedoch delegiert sie nicht nur Methoden der Zielklasse, sondern orchestriert komplexe Abläufe. So werden bei der Methode *createDatabaseOffer* je nach Parametrisierung bis zu 16 API Aufrufe vorgenommen. Die Methode ermöglicht es, Ergebnisse einer Datenbankabfrage als Datenangebot in einem Datenraum anzubieten. Um dies zu erreichen, müssen diverse Objekte angelegt, konfiguriert und verlinkt werden. Das Datenmodell des DSC orientiert sich dabei an dem IDS Infomodel (siehe hierzu Abschnitt 2.1.4 *Information Layer*). Zur Veranschaulichung werden im Folgenden die Methodenaufrufe, die zur vollständigen Ausführung von *createDatabaseOffer* führen, gelistet. Jeder Me-

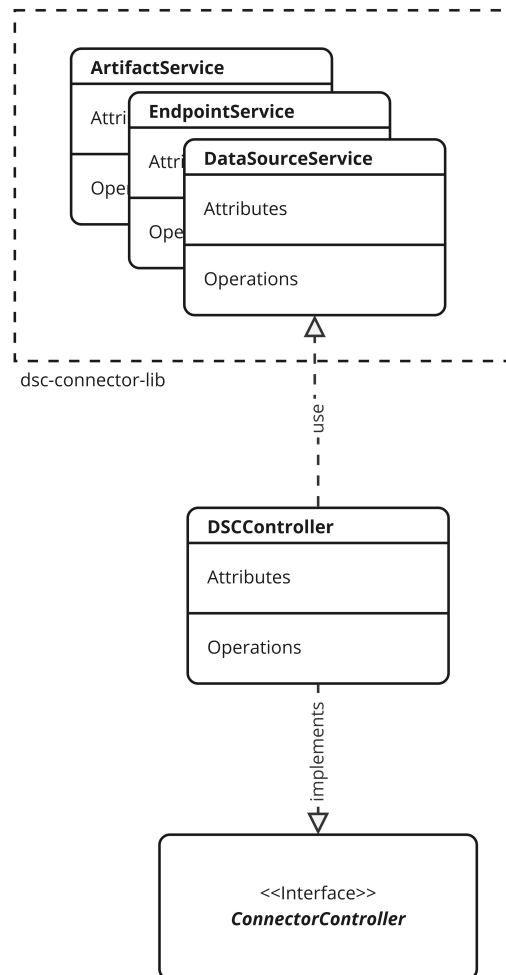


Abbildung 6.3: Vereinfachte Architektur DSC-Controller

thodenaufruf entspricht einer REST Anfrage an die Instanz der Datenraumkomponente.

1. `dataSourcesService.createDataSource`: Anlegen der Datenquelle (Verbindungsinformationen zum Datenbankserver).
2. `endpointsService.createEndpoint`: Anlegen des Camel⁵ Endpunkts und Definition SQL Query.
3. `endpointsService.linkDataSource`: Den Endpunkt von Schritt (2) mit der Datenquelle aus Schritt (1) verlinken.

⁵Der DSC setzt Apache Camel zum Routing ein (<https://camel.apache.org/>)

4. `routesService.createRoute` Anlegen der Camel Route
5. `routesService.createStartEndpoint` Festlegen des in (2) angelegten Endpunkts als Start der in (4) angelegten Route
6. `artifactsService.createArtifact`: Anlegen eines Artefakts, welches über den Connector angeboten werden soll
7. `catalogsService.createCatalog` Erstelle ein Katalog, in dem das Artefakt aus (6) gelistet sein soll.
8. `offeredResourcesService.createOffer` Erstelle ein Angebot
9. `representationsService.createRepresentation` Erstelle eine Repräsentation (Dateiformat, ..)
10. `contractsService.createContract` Erstelle ein Vertrag
11. `rulesService.createRule` Erstelle eine Regel
12. `catalogsService.addOfferedResourceToCatalog`: Füge Angebot aus (8) dem Katalog aus (7) hinzu.
13. `offeredResourcesService.addRepresentationToOfferedResource`: Füge Repräsentation aus (9) dem Angebot aus (8) hinzu.
14. `representationsService.addRepresentationArtifact`: Füge Repräsentation aus (9) dem Artefakt (6) hinzu.
15. `offeredResourcesService.addContractToResource` Füge den Vertrag aus (10) zum Angebot (8) hinzu.
16. `contractsService.addRuleToContract`: Füge Regel aus (11) dem Vertrag aus (10) hinzu.

Ein solcher Ablauf ist stark von der gewählten Connector-Implementierung abhängig und würde bei einem anderen Connector, wie beispielsweise dem EDC, anders aussehen. Sowohl diese Varianz, als auch die Komplexität soll für Forschende in der Szenariobeschreibung transparent sein, um die Niedrigschwelligkeit der Nutzung zu erhalten. Darüber hinaus ermöglicht diese Architektur die Ausführung der Szenarien unter Verwendung unterschiedlicher Connector-Implementierungen.

6.4.2 Usage Rule Mapper

Um einen Datenaustausch in Datenräumen durchzuführen, müssen die Parteien Nutzungsbedingungen vereinbaren, die dann durch die Connectoren durchgesetzt werden. Da verschiedenen Datenraumtechnologien diese auf unterschiedliche Art und Weise beschreiben können, braucht es einen Weg um in allgemeinen Szenarien diese dennoch definieren zu können. Das Framework abstrahiert hierfür die Nutzungsbedingungen auf die Klasse *UsagePolicy*. Unterklassen (wie z. B. *TimerangeRestricted* oder *NumberUsagesRestricted*) definieren dann die konkreten Bedingungen. Die *DataSpace-Component*

Controller stellen *Mapper* bereit, die diese Klassen dann in konkrete Bedingungen übersetzen.

Da TypeScript zur Laufzeit keine Objekte mit *typeof* vergleichen kann [Freeman 2021][S. 235, S. 238], muss ein *Type Guard* implementiert werden. Im Gegensatz zur Identifizierung kompatibler Klassen bei der generischen *deploy* Methode des *KubernetesControllers* (siehe Abschnitt 6.5.1) ist hier eine Erkennung über die Eigenschaften der Klasse nicht möglich. Es könnte mehrere *Usage Rules* ohne Attribute oder mit den gleichen Attributen geben. Daher wurde ein Diskriminator mit dem Namen *type* in der Basisklasse eingeführt. Diesen müssen die Unterklassen überschreiben und sich identifizieren. Auf Basis dieses Diskriminators kann der *Mapper* die richtige *Usage Rule* instanziiieren.

6.4.3 Steuerungsbibliotheken

Die konkrete Ansteuerung der Controller ist in einem eigenen Projekt realisiert, das eigenständig sinnvoll nutzbar und separat veröffentlicht worden ist. Die Bibliothek bildet den REST-Endpunkt direkt als JavaScript (node) API ab, ohne erweiterte Logik und kann somit für die Steuerung der Controller auch außerhalb des Simulationskontexts genutzt werden. Hierfür wurde unter anderem das gesamte Objektmodell des Endpunkts typsicher abgebildet, die Struktur der Routen in Services strukturiert und der Algorithmus zum Absenden der HTTP Requests mithilfe einer geeigneten Bibliothek implementiert.

Da es die Möglichkeit gibt eine OpenAPI Spezifikation für den DSC aus dem Java Quelltext zu generieren [Fraunhofer ISST 2021a], war es möglich einen Teil der Arbeit an der *dsc-lib* einzusparen. Mit dem Tool *openapi-typescript-codegen*⁶ konnte eine Grundstruktur der Bibliothek automatisiert generiert werden. Danach wurden kleinere Anpassungen vorgenommen, um beispielsweise das Projekt als *esm* Modul veröffentlichen zu können, die Typsicherheit noch weiter zu erhöhen oder *linting* Regeln bzw. dem eigenen Programmierstil zu entsprechen.

⁶<https://github.com/ferdikoomen/openapi-typescript-codegen>

6.5 Kubernetes-Controller

Projektname: dssim-kubernetes-controller
DSSIM-Framework-Modul: Environment-Controller
Repository:
<https://github.com/FraunhoferIEE/dssim-kubernetes-controller>
npm-Link:
<https://www.npmjs.com/package/dssim-kubernetes-controller>

Sub-Repositories:

Projektname: docker-tc-kubernetes
Repository: <https://github.com/otmi100/docker-tc-kubernetes>

Der *Kubernetes-Controller* ist eine Implementierung des *Environment-Controller* Moduls und erlaubt es, Szenariobeschreibungen (Klassen, die das *Szenario-Interface* implementieren) in einem Kubernetes-Cluster zu simulieren. Hierfür werden alle notwendigen Konfigurationen vorgenommen, Deployments durchgeführt sowie Services und das Routing eingerichtet. Eine manuelle Konfiguration des Clusters soll vermieden werden.

Kubernetes stellt hierfür eine REST API zur Verfügung [Kubernetes Projekt 2023i], über die der Status von Objekten abgefragt und manipuliert werden kann. Das vielfach verwendete Kommandozeilenwerkzeug *kubectl* baut ebenfalls auf diese API auf [Kubernetes Projekt 2023j]. Um die REST API Aufrufe nicht selbst implementieren zu müssen, wird die offiziell unterstützte *@kubernetes/client-node* Bibliothek verwendet. Diese abstrahiert die Schnittstelle und erlaubt eine typsichere Verwendung der API über Methodenaufrufe.

6.5.1 Architektur

Ein vereinfachtes Klassendiagramm wird in Abbildung 6.4 dargestellt und soll im Folgenden erläutert werden. Das *Kubernetes-Controller* Modul besteht aus der Klasse *KubernetesController*, *Instance* Implementierungen zu Datenraumkomponenten (z. B. DSC/EDC ConnectorInstance, IdentityProviderInstance, BrokerInstance) und Klassen für andere Infrastrukturkomponenten wie einem System für die Netzwerkmanipulation. Außerdem befindet sich in diesem Modul das Monitoring Submodul (siehe Abschnitt 6.6 *Scenario-Monitoring*) und unterstützende Klassen wie z. B. den *KubernetesExecutor*.

Die Klasse *KubernetesController* implementiert das *EnvironmentControllerInterface* und stellt damit zentrale Funktionen zur Verfügung. Da die Initialisierung der Klasse damit beendet sein soll, dass alle notwendigen In-

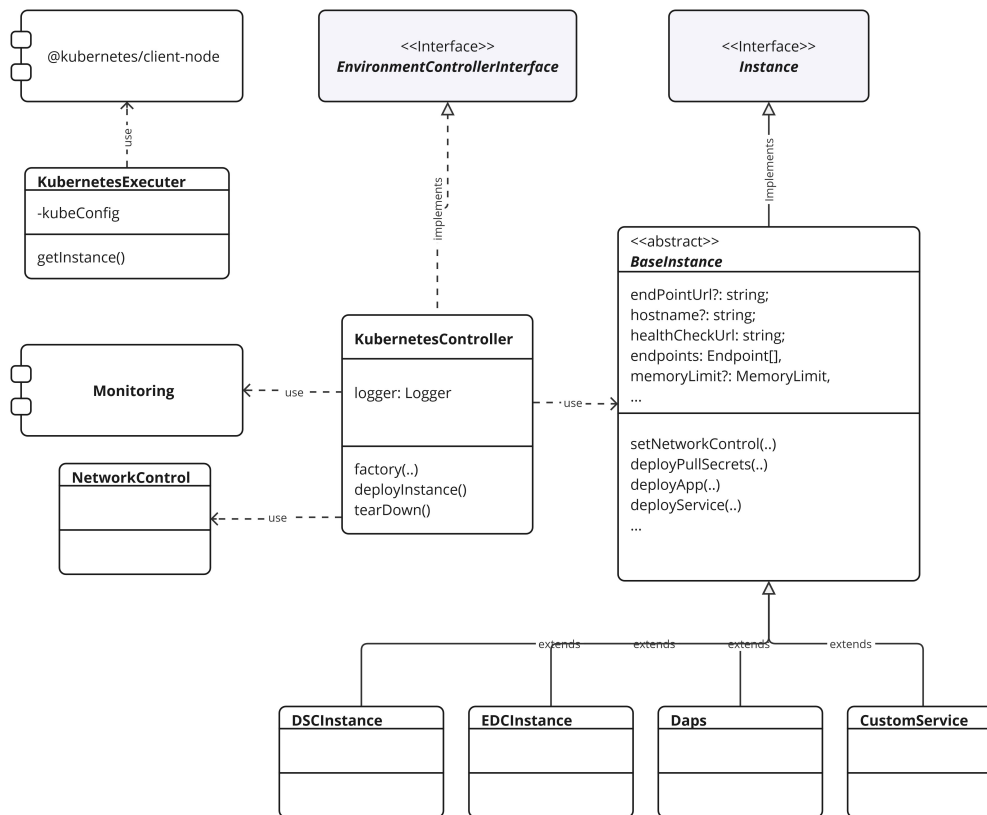


Abbildung 6.4: Klassendiagramm Kubernetes-Controller

infrastrukturkomponenten einsatzbereit sind, gibt es eine asynchrone *Factory* Methode und der Konstruktor ist *private*. Die generische *deployInstance* Methode führt das Template zum Deployment von *Instance* Klassen bereit. Der folgende Abschnitt 6.5.1 *Deployment von Komponenten* erklärt das Deployment ausführlich.

Die Klasse *KubernetesExecutor* bildet eine Fassade (siehe Abschnitt 2.3.3) für die *@kubernetes/client-node* Bibliothek und vereinfacht und zentralisiert die Interaktion mit dem Cluster. Sie erzeugt notwendige Client-Instanzen, definiert standardmäßige Einstellungen und Labels und legt die Kubernetes Objekte über die Client Library an. Um einen globalen Zugriff aus allen *Kubernetes-Controller* Klassen zu ermöglichen und sicherzustellen, dass die Steuerung der Kubernetes Instanz nur an einer Stelle erfolgt, ist der *KubernetesExecutor* nach dem *Singleton* Entwurfsmuster realisiert. Diese eindeutige Instanz ist zweckmäßig, wenn die Reihenfolge der Erzeugung von Objekten eingehalten werden muss, Queues genutzt oder Informationen über

den aktuellen Zustand der Kubernetes Instanz vorgehalten werden. Um das Klassendiagramm übersichtlich zu halten, sind die Referenzen auf den *KubernetesExecuter* nicht eingezeichnet.

Deployment von Komponenten

In der aktuellen Ausbaustufe des Frameworks erfolgt die Beschreibung von deploybaren Datenraumkomponenten noch nicht technologieagnostisch. Das heißt, dass der *KubernetesController* spezifische Beschreibungen benötigt und das Implementierungen des *Instance* Interface vom *dssim-core* Modul nicht zwingend hinreichend sind. Der *KubernetesController* kann nur Erweiterungen seiner abstrakten Klasse *BaseInstance* deployen. Inkompatible Instanzen werden zur Laufzeit mit einem Laufzeitfehler abgewiesen werden.

Auch hier besteht wieder das bereits in Unterabschnitt 6.4.2 *Usage Rule Mapper* aufgeführte Problem, dass *TypeScript* nicht ohne weiteres den Laufzeittyp einer Klasse bestimmen kann und ein eigener *Type Guard* implementiert werden muss. Eine relativ elegante Variante bietet eine sogenannte *Type Predicate Function*. Das Listing 6.5 zeigt einen Ausschnitt des Quellcodes, bei dem vor dem Deployment der Instanz überprüft wird, ob es sich tatsächlich um eine Klasse handelt die ein Deployment in Kubernetes beschreibt. Falls dem nicht so ist, wird ein nachvollziehbarer Laufzeitfehler geworfen.

```

1  async deployInstance<T extends Instance>(instance: T):
    Promise<T> {
2      if (this.isKubernetesControllerInstance(instance)) {
3          await this.deployWithTemplate<T>(instance);
4          return instance;
5      } else {
6          throw new Error(
7              'Only Instances that implement BaseInstance of the
                KubernetesController Package can be deployed by this
                controller'
8          );
9      }
10 }
11
12 private isKubernetesControllerInstance(
13     instance: Instance
14 ): instance is BaseInstance {
15     return (
16         (instance as BaseInstance).deployPullSecrets !==
            undefined &&
17         (instance as BaseInstance).deploySecrets !== undefined &&
18         (instance as BaseInstance).deployConfigMaps !== undefined
            &&
19         (instance as BaseInstance).deployApp !== undefined &&

```

```

20     (instance as BaseInstance).deployServices !== undefined
      &&
21     (instance as BaseInstance).deployIngress !== undefined &&
22     (instance as BaseInstance).containerImages !== undefined
23   );
24 }

```

Quellcode 6.5: Type Predication in TypeScript

Instance Definitionen

Für die Bereitstellung einer nutzbaren Datenraumkomponente muss der *KubernetesController*, in Abhängigkeit der Szenariokonfiguration und Szenariobeschreibung, diverse Objekte im Cluster bereitstellen und konfigurieren. Hierzu gehören *Workload* Ressourcen (z.B. *Deployments*, *DaemonSets*), *Services*, Speicher, Konfigurationsdateien, *(Pull-)Secrets* oder auch *CustomResources* (siehe 6.6). Diese werden in einer deklarativen Beschreibung des Zielzustandes an die Kubernetes REST API übergeben und das Cluster sorgt selbstständig für eine entsprechende Umsetzung. Das Framework muss also diese deklarativen Beschreibungen generieren, an das Cluster übergeben und die Durchführung abwarten.

Folgende konzeptionelle Überlegungen wurden vor der Implementierung festgelegt:

- Die Generierung der Ressourcenbeschreibung soll in separaten Klassen stattfinden, um eine saubere Kapselung sicherzustellen.
- Für generelle Methoden und eine einheitliche Struktur dieser Klassen soll es eine abstrakte Klasse geben, von der abgeleitet werden muss.
- Es soll einen zentralen Service geben, der die Kommunikation mit der REST API über die Node-Kubernetes-Bibliothek verantwortet. So wird eine zentrale Konfiguration sichergestellt; Ressourcenspezifikationen können manipuliert werden, um beispielsweise alle Spezifikationen mit einem gemeinsamen Label zu versehen und eventuelle Anpassungen an den Schnittstellen können zentral vorgenommen werden.

Zur Umsetzung wurde auf das Entwurfsmuster *Template Method* der GoF [Gamma u. a. 1994][S. 325] zurückgegriffen. Hierfür wurde das *Gerüst* des Deploymentalgorithmus in der generischen Methode *deployWithTemplate* (siehe Quellcode 6.6 *Template Methode*) definiert. Die Basisklasse stellt bereits Standardimplementierungen für Teilschritte des Deployments zur Verfügung, um beispielsweise die Pullsecrets der Container zu deployen, oder Services und Ingresses für definierte Endpunkte einzurichten. Die Unterklassen haben die Möglichkeit einzelne Schritte zu überschreiben, falls der Standard nicht den spezifischen Anforderungen genügt. Somit bleibt eine saubere Grund-

struktur, eine hohe Wiederverwendung von Quellcode und dennoch die Möglichkeit spezifische Lösungen umzusetzen.

```
1 private async deployWithTemplate<T extends Instance>(
2     instance: T & BaseInstance
3 ) {
4     const pullSecrets = await instance.deployPullSecrets();
5     await instance.deploySecrets();
6     await instance.deployConfigMaps();
7     await instance.deployApp(pullSecrets);
8     await instance.deployServices();
9     await instance.deployIngress();
10 }
```

Quellcode 6.6: Template Methode

6.5.2 Bereitstellung von Dateien

Einige Datenraumkomponenten benötigen für die Ausführung bereitgestellte Dateien, beispielsweise Konfigurationsdateien oder binäre *Vault* Dateien für Zertifikate. Diese Dateien werden bei Instanziierung an die *Instance* Klasse übergeben und der *KubernetesController* sorgt dafür, dass diese als *ConfigMap* zur Verfügung gestellt werden. Beim Start des *Pods* werden die Dateien von einem *initContainer* aus der *ConfigMap* in ein *Storage Volume* kopiert. Binärdateien werden für die Übertragung an das Cluster *base64* codiert und als Zeichenkette übertragen. Dies stellt sicher, dass jede Datenraumkomponente die Dateien an der richtigen Stelle abgelegt und überschreibbar sind.

6.5.3 Routing

Wie in den Grundlagen beschrieben findet die Kommunikation innerhalb des Clusters über *Kubernetes Services* statt, die standardmäßig über das zuvor beschriebene deployment *Template* bereitgestellt werden. Für den Zugriff von außerhalb des Clusters werden *Ingresses* eingerichtet, welche auf Basis des Hostnamen und des Pfades die Anfragen an den entsprechenden Service weiterleiten. Hieraus resultiert das Problem, dass die Hostnamen auf dem Framework-ausführenden System bekannt sein müssen und auf die IP Adresse eines Nodes des Clusters auflösen müssen. Dies kann mitunter auf folgenden Wegen erfolgen:

- Es werden Hostnamen verwendet, die als Sub-Domain eines bereits bestehenden DNS auf eine IP Adresse des Clusters auflösen.

- Ein Eintrag in der *hosts* Datei⁷ sorgt für eine lokale Auflösung.
- Ein lokaler *DNS* Server wird auf dem ausführenden System gestartet und zur Namensauflösung genutzt.

Die für die zweite und dritte Variante ist eine Automatisierung durch das Framework denkbar. Zum aktuellen Zeitpunkt muss dies noch manuell erfolgen.

6.5.4 Umsetzung Ressourcensteuerung

Das Framework soll es ermöglichen Systemressourcen wie Netzwerk, CPU oder RAM Kapazitäten zu beeinflussen, um die Ausführung in Umgebungen mit eingeschränkten Ressourcen simulieren zu können. Durch Implementierung des Interfaces einer Datenraumkomponente des EnvironmentController-Moduls (z. B. *ConnectorInstance* oder *IdentityProviderInstance*) wird auch das Interface *Instance* implementiert, welche Methoden (z. B. *setNetworkControl*) oder Properties (*memoryLimit?: value: number; unit: MemoryUnit;*) zur Einschränkung von Ressourcen einfordert.

Realisiert werden diese Einschränkungen der Infrastruktur in der abstrakten Basisklasse *BaseInstance*. Wie Abbildung 6.4 zeigt, leiten alle Klassen der Datenraumkomponenten von dieser Klasse ab. Während die Limitierung von CPU und Arbeitsspeicher-Ressourcen in Kubernetes standardmäßig eingeschränkt werden können, muss für einen Eingriff in die Netzwerkressourcen eine andere Lösung gefunden werden.

Einschränkung von CPU und Arbeitsspeicher Ressourcen

Bei der Definition der Kubernetes Objekte über die API können je Container Ressource Limitierungen angegeben werden. Das *kubelet* setzt diese Limitierungen um und sorgt dafür, dass die Container nicht mehr Ressourcen beanspruchen als angegeben. [Kubernetes Projekt 2023f]

Die Einschränkung von Arbeitsspeicherkapazitäten erfolgt in bytes und kann in den Einheiten Kilo (k), Mega (M), Giga (G), Tera (T), Peta (P) und Exa (E) angegeben werden. Auch die Angabe in der entsprechenden Binär-Größe Kibi (Ki), Mebi (Mi), Gibi (Gi), Tebi (Ti), Pebi (Pi) und Exbi (Ei) ist möglich.

CPU Ressourcen werden in *cpu units* gemessen. In Kubernetes entspricht eine *cpu unit* einem physischen CPU Kern (bzw. in virtuellen Umgebungen einem virtuellen CPU Kern). Dabei können Anteile als Dezimalzahlen

⁷Unter Linux i. d. R. */etc/hosts*, unter Windows 10 *C:/Windows/System32/drivers/etc/hosts*

angegeben werden. So entspricht 0,5 *cpu units* der Rechenkapazität von einem halben CPU Kern. Alternativ ist die Angabe in *millicpu* (auch *milli-cores* genannt) möglich, also einem tausendstel CPU. Die Einschränkung 0,5 *cpu units* könnte dementsprechend mit 500 *millicpu* angegeben werden. Die kleinste Einheit ist 1 *millicpu* (0.001 *cpu units*). Da die CPU Geschwindigkeit nicht nur aus dem Rechentakt in Megahertz besteht, sondern auch CPU-Architektur oder die Größe der Caches eine Rolle spielt, ist ein vertieftes fachlich technisches Wissen des Anwenders notwendig und umfassende Kenntnisse über die zu simulierende Umgebung, als auch die Simulationsumgebung, um eine Zielumgebung realitätsnah simulieren zu können.

Einschränkung des Netzwerks

Über die oben beschriebenen Limitierungen in der Deploymentspezifikation können nur CPU und Arbeitsspeicher Ressourcen beeinflusst werden, eine Einschränkung der Netzwerkkapazitäten ist auf diesem Weg aktuell nicht möglich.

Im Rahmen der Lösungsfindung haben sich verschiedene Ansätze als potenziell gangbar herausgestellt: Umsetzung über eines bestehenden oder neu zu entwickeltes CNI Plugin, einem *Service Mesh*, eine direkte Manipulation der Netzwerkschnittstelle über *linux traffic control (tc)* oder die Verwendung des Projekts *Docker Traffic Control*. Die Varianten werden im folgenden kurz beschrieben, die Anwendbarkeit im Problemkontext bewertet und der gewählte Lösungsweg begründet.

CNI Plugin Es existiert ein CNI Plugin, dass *traffic shaping* als experimentelles Feature unterstützt, es erlaubt jedoch nur eine Beschränkung der eingehenden und ausgehenden Bandbreite [Kubernetes Projekt 2023c][CNI Projekt 2023]. Außerdem bedeutet die Installation eines CNI Plugins einen Eingriff in die Kubernetes-Systemkonfiguration und ist nur mit entsprechenden Berechtigungen möglich. Zur Durchsetzung der konfigurierten Bandbreite nutzt das *bandwidth* Plugin *tc*, worauf im Verlauf des Abschnittes weiter eingegangen wird. Auch wäre eine Eigenentwicklung eines CNI Plugins möglich, was jedoch einen erheblichen Aufwand mit sich bringen würde.

Service Mesh Potenziell wäre eine Einflussnahme auf die Netzwerkressourcen über eine *Service Mesh* Implementierung möglich. *Service Meshes* zentralisieren das Routing zwischen Services und sind als *Side Car Proxy* realisiert [Goniwada 2021][S.281f]. *Istio*, als eine der bekannteren *Service Mesh* Implementierungen, erlaubt beispielsweise die Einrichtung von *Rate*

limits [Istio Projekt 2023] zur Limitierung des Datentransfers zwischen Services. Diese Rate Limits können jedoch nur die Anzahl der Requests in einem Zeitfenster begrenzen und nicht Datenmengen oder Übertragungsqualität beeinflussen. Diese Einschränkungen werden dann nicht auf einen Netzwerkdapter angewendet, sondern auf eine Service zu Service Brücke.

linux traffic control (tc) Als weitere Alternative bietet sich der Einsatz linux traffic control (tc) an. tc ist der Name für einen Satz an Werkzeugen um die Warteschlangen und Wartemechanismen einer Netzwerkschnittstelle eines Linux Systems zu steuern. Typische Funktionen von tc ist das *Shaping*, mit dem sich sowohl Verzögerungen in der Paketzustellung, als auch eine Limitierung der Bandbreite implementieren lassen, oder das *Dropping*, mit dem der Paketverlust beeinflusst und schlechte Netzwerkverbindungen simuliert werden können. Auch viele andere Lösungen setzen im Hintergrund tc für die Durchsetzung ein. Ein Nachteil von tc ist die Komplexität in der Anwendung [Brown 2006] und es müsste eine eigene Lösung entwickelt werden, wie tc in das Cluster integriert und die Regeln angewendet werden.

Docker Traffic Control Das vollständig in Bash programmierte Projekt *Docker Traffic Control*⁸ ermöglicht es Netzwerkbedingungen wie Verzögerungen, Paketverlust, -doppelung oder -beschädigungen für virtualisierte Container zu simulieren. Hierfür werden mithilfe von tc die Warteschlangen der virtuellen Netzwerkschnittstelle manipuliert. *Docker Traffic Control* ist jedoch für den Einsatz auf einer lokalen Docker-Instanz geschrieben worden und nicht ohne weiteres in einer Kubernetes Umgebung einsetzbar, was eigene Anpassungen notwendig macht. Weitere Details hierzu werden im separaten Abschnitt Docker Traffic Control beschrieben.

Zusammenfassung und Auswahl Variante Die Tabelle 6.1 zeigt eine Übersicht über die verschiedenen Varianten und ob die Mindestanforderungen für die Anwendung im Problemkontext erfüllt sind. Die Kriterien und Mindestanforderungen sind wie folgt definiert:.

1. *Möglichkeiten*: Die *Möglichkeiten* der Einflussnahme auf die Netzwerkkommunikation ist ausreichend. Mindestens muss die Bandbreite und die Verzögerung (Latenz) der Paketzustellung konfigurierbar sein.
2. *Aufwand*: Der *Aufwand* bewertet ob die Lösung im angemessenen Zeitaufwand für den Autor umsetzbar ist.

⁸<https://github.com/lukaszlach/docker-tc>

3. *Systemunabhängigkeit*: Die *Systemunabhängigkeit* bewertet, ob die Lösung weitestgehend unabhängig von der Konfiguration der Infrastruktur einsetzbar ist.

Variante/Kriterien	1	2	3
CNI Plugin <i>Bandwidth</i>	×	✓	×
CNI Plugin Eigenentwicklung	✓	×	×
Service Mesh <i>istio</i>	×	×	✓
Linux Traffic Control	✓	×	✓
Docker Traffic Control	✓	✓	✓

Tabelle 6.1: Gegenüberstellung Simulation von Netzwerkbedingungen.

Sowohl eine Eigenentwicklung als CNI Plugin, als auch eine Umsetzung mit *linux traffic control* bieten die höchste Flexibilität und die meisten Möglichkeiten zur Einflussnahme, benötigen jedoch so viel Aufwand, dass diese nicht im Rahmen dieser Arbeit umsetzbar sind. Der Einsatz des CNI Plugins *bandwidth* kann nur die Bandbreite einschränken und erfordert mit der Installation einen Eingriff in die Systemkonfiguration, was nicht in jedem System möglich ist. Die Service Mesh Lösung *istio* ist mit der Zentralisierung der Serviceorchestrierung und Nutzung von *Side Car Proxies* sehr schwergewichtig, aufwändig und bietet nur geringe Möglichkeiten der Einflussnahme. Als praktikabelste Lösung bietet sich *Docker Traffic Control* an. Die Anpassungsaufwände sind gering, die Nutzung einfach und die Möglichkeiten der Einflussnahme sehr hoch. Da das Projekt open source ist und tc zur Steuerung genutzt wird, wäre auch eine Erweiterung des Funktionsumfangs möglich, der einer individuellen Lösung mit tc in wenig nachsteht.

Docker Traffic Control *docker-tc* wird in einem Container mit ausgeteigten Berechtigungen auf dem gleichen Host wie der Container, dessen Netzwerk beeinflusst werden soll, ausgeführt. Der Docker Socket des Hosts wird in dem Container eingebunden, damit von dem *docker-tc* Container aus der *docker-daemon* gesteuert werden kann. Die Kontrolle von *docker-tc* erfolgt über HTTP Aufrufe, die an einem gestarteten Endpunkt eingehen, oder über *label* an den zu beeinflussenden Containern, welche kontinuierlich überwacht werden. Wenn über einen dieser Wege erkannt wird, dass eine Beeinflussung erfolgen soll, versucht *docker-tc* die betreffende Netzwerkschnittstelle herauszufinden, übersetzt die gewünschte Beeinflussung in tc Befehle und führt diese aus.

Wie im vorhergehenden Abschnitt beschrieben, ist *Docker Traffic Control* nicht ohne weiteres in einer Kubernetes Umgebung nutzbar, da es für

den lokalen Einsatz mit *docker* programmiert worden ist. Im Unterschied zu einer lokalen Ausführung der Container mit *docker*, wird in Kubernetes das Netzwerk mithilfe von CNI realisiert⁹. Konkret wurde in der zur Entwicklung und Erprobung eingesetzten Kubernetes Umgebung *Calico*¹⁰ als CNI-Plugin für die Erstellung des *Overlay Netzwerks* verwendet. Das bedeutet, dass das *docker network* deaktiviert und mit dem Befehl ‘*docker inspect*’ der virtuelle Netzwerkadapter nicht ausgelesen werden kann. Daher musste der Algorithmus zur Identifikation der richtigen Netzwerkschnittstelle angepasst werden. Auch die Prefixe der Netzwerkschnittstelle unterscheiden sich zum *docker network* und wurden über eine Umgebungsvariable anpassbar gemacht.¹¹

6.6 Scenario-Monitoring

Projektname: Teil von *dssim-kubernetes-controller*
DSSIM-Framework-Modul: Monitoring

Das *Scenario-Monitoring*-Modul benötigt für eine gesamtheitliche und umfängliche Betrachtung der Szenarioausführung Informationen von den Datenraumkomponenten, den Kubernetes-Nodes und dem DSSIM-Framework selbst. Ziel ist es die Daten zusammenzuführen und so aufzubereiten, dass versierte Forschende Erkenntnisse über die Ausführung der gestalteten Szenarios erlangen können. Auch für das Debugging beim Aufbau von Szenarien oder beim Experimentieren ist eine zentrale Anlaufstelle wertvoll, um bei der Fehlersuche nicht auf alle Informationsquellen einzeln zugreifen zu müssen.

Aufgrund der inhaltlichen Nähe zum *Environment-Controller*-Modul ist das *Monitoring*-Modul eher als ‘Submodul’ zu betrachten und in der aktuellen Ausprägung auch Teil des *Kubernetes-Controller*-TypeScript-Projekts. Dennoch soll das Modul nur lose gekoppelt und austauschbar sein.

Das Modul besteht aus mehreren Komponenten:

- Die **Logging-Pipeline** transportiert Log Dateien von allen Datenraumkomponenten und DSSIM an eine zentrale Stelle und stellt diese für eine Abfrage durch das Auswertungsmodul bereit.
- Die **Scraping-Pipeline** sammelt Informationen über den Systemzustand der Ausführungsumgebung (Auslastung der Nodes, Konfigurati-

⁹weitere Ausführungen hierzu im Abschnitt 2.4.2 Kubernetes in den Grundlagen

¹⁰<https://github.com/projectcalico/calico/tree/master/cni-plugin>

¹¹Alle Änderungen können hier im öffentlichen Reposirty des *Forks* eingesehen werden: <https://github.com/lukaszlach/docker-tc/compare/master...otmi100:docker-tc-kubernetes:master>

on, ..), transportiert diese an eine zentrale Stelle und stellt diese für eine Abfrage durch das Auswertungsmodul bereit.

- Das **Auswertungsmodul** greift auf diese so bereitgestellten Daten zu, bereitet die Daten sinnvoll auf und stellt diese auf einem *Dashboard* den Forschenden für eine Analyse zur Verfügung.

Der Aufbau und die Umsetzung dieser Komponenten wird in den nächsten Abschnitten erläutert.

6.6.1 Logging-Pipeline

In der zur Entwicklung und Test des Frameworks genutzten Umgebung wird als Kubernetes Verwaltungsplattform *Rancher* eingesetzt, welches bereits vorinstalliert Monitoringmechanismen mitbringt und Logging Infrastruktur über eine Erweiterung bereitstellt. Um den Overhead für das Cluster gering zu halten, werden diese Mechanismen genutzt und im Folgenden erklärt.

Das Logging System von Rancher, in der eingesetzten Version 2.6, basiert auf dem *Banzai Cloud Logging*-Operator. Es übernimmt das Deployment und die Konfiguration einer Logging Pipeline bestehend aus folgenden Komponenten: *Fluent Bit* wird als *DaemonSet*, also auf jedem Node des Clusters, deployt und sammelt die Container- und Anwendungslogs vom Dateisystem der Nodes. Außerdem werden Metainformationen über die Pods durch *Fluent Bit* über die Kubernetes API gesammelt. Diese Metainformationen und die Logs der Container werden an eine *Fluentd*-Instanz weitergeleitet, der diese auf Basis von *Flow*-Definitionen filtert und die Ergebnisse basierend auf *Outputs*-Definitionen an entsprechende Empfänger weiterleitet. Mithilfe von *CustomResourceDefinitions* erweitert der *Banzai Cloud Logging*-Operator die Kubernetes API und wird über diese gesteuert. Dadurch lassen sich dann die *Flow*- und *Output*-Definitionen direkt über die Kubernetes API oder über die Weboberfläche von Rancher steuern. [SUSE 2023a]

Die *Flow* und *Output* Definitionen werden zusammen mit einer *Loki*-Instanz und einer *Grafana*-Instanz bei der Initialisierung des *Szenario-Monitoring*-Moduls automatisiert über die Kubernetes API deployt. *Loki*¹² ist ein durch *Prometheus* inspiriertes Log-Aggregationssystem. *Loki* indexiert die Logs nicht, sondern speichert diese unstrukturiert und komprimiert. Die Logs werden nur gelabelt, die Metadaten indexiert und die Daten für eine weitere Verwendung bereitgestellt.

Da das DSSIM-Framework in der Regel lokal auf dem Computer des Forschenden ausgeführt wird, können die Logs nicht auf demselben Weg zusammengeführt werden, wie von den Datenraumkomponenten. Es gäbe theo-

¹²<https://github.com/grafana/loki>

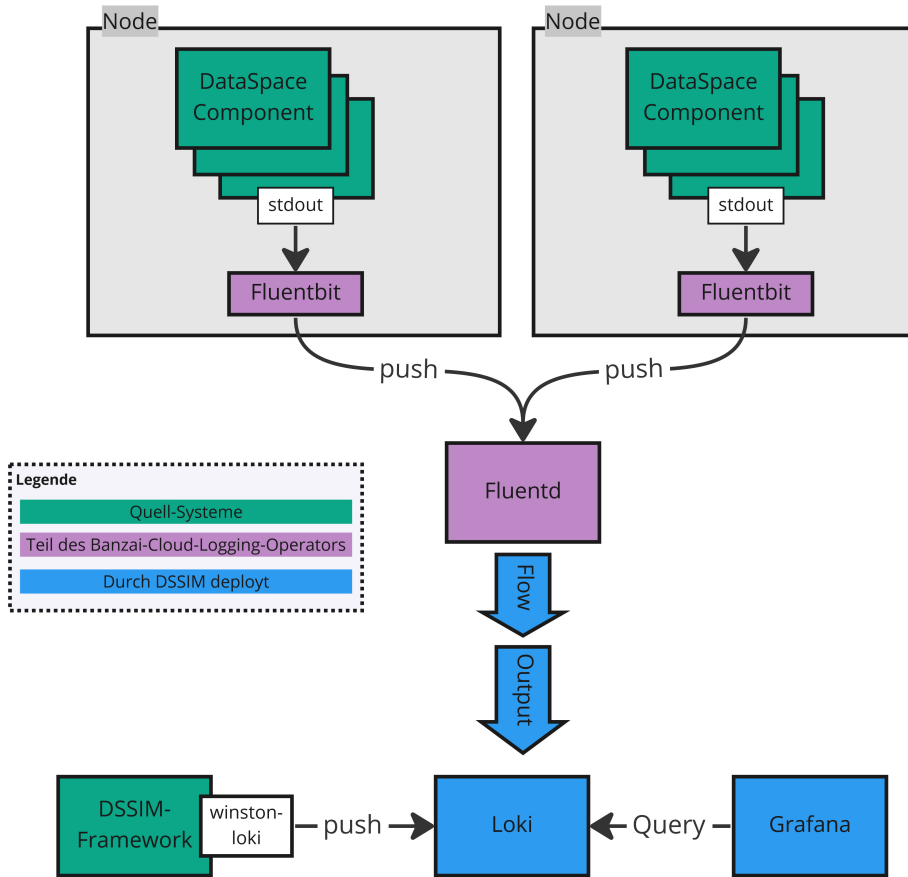


Abbildung 6.5: Logging-Pipeline des Scenario-Monitoring Moduls (eigene Darstellung)

retisch die Möglichkeit auch auf dem ausführenden Computer eine *Fluentbit* Instanz laufen zu lassen, um die Konsolenausgabe abzugreifen und in die Logging-Pipeline zu schicken. Damit jedoch keine weitere Software installiert werden muss, wurde die JavaScript Bibliothek *winston*¹³ mit der Erweiterung *winston-loki*¹⁴ genutzt. Diese schickt die Logs direkt an die API der *Loki*-Instanz.

Die Abbildung 6.5 visualisiert die Logging-Pipeline in Gänze. Die Grün dargestellten Komponenten sind Informationsquellen von denen die Logs erfasst werden sollen. Dazu gehören Datenraumkomponenten, sonstige Simu-

¹³<https://github.com/winstonjs/winston>

¹⁴<https://github.com/JaniAnttonen/winston-loki>

lations- oder Infrastruktur Instanzen wie z. B. *docker-tc* oder *Datenservices* und die (in der Regel lokal auf dem Rechner des Forschenden) Ausführung des Frameworks zur Steuerung der Simulation. Die in Lila dargestellten *Fluentbit* und *Fluentd* Instanzen werden durch den *Banzai Cloud Logging*-Operator bereitgestellt und verwaltet, die blau dargestellten Komponenten werden durch das Framework bei Initialisierung des *Monitoring-(Sub-)Moduls* deployt. Die *Grafana*-Instanz ist Teil des 6.6.3 Auswertungsmodul und wird dort näher erläutert.

6.6.2 Scraping-Pipeline

Auch für das Monitoring von Systemen bringt *Rancher* ein Werkzeug mit der Bezeichnung *rancher-monitoring* mit. Es basiert im Wesentlichen auf Prometheus¹⁵ und Grafana¹⁶, sowie Erweiterungen hierzu. [SUSE 2023c]

Auch *Grafana* folgt modularisierten Ansätzen und unterstützt vielfältige Datenquellen. Damit könnte die *Logging- & Scraping-Pipelines* durch andere Technologien ersetzt werden, ohne eine neue Plattform für die Aufarbeitung und Visualisierung zu benötigen.

Abbildung 6.6 zeigt die Funktionsweise der *Scraping-Pipeline*. Analog zu den Logging Mechanismen, werden auf allen Nodes des Clusters Applikationen (*PushProx Client*) via DaemonSets deployt, welche Monitoring Informationen von den zu beobachtenden Komponenten sammeln (auch *scraping* genannt). Diese nehmen zu einem Intermediär (*PushProx Proxy*) Kontakt auf, erfragen die zu sammelnden Informationen und Zeitpunkte und übermitteln die Ergebnisse. Diese leitet der *PushProx Proxy* dann an Prometheus weiter, welcher die Daten in einer Prometheus Time Series Database (TSDB) ablegt. Dort können sie dann, mit der Prometheus-Abfragesprache *PromQL*, abgefragt werden. Darüber hinaus sammelt Prometheus auch direkt Informationen von den kubelets, Ingress, coreDNS/kubeDNS und dem kube-api-server, die ebenfalls in der TSDB abgelegt werden. [SUSE 2023b][PushProx Projekt 2023]

6.6.3 Auswertungsmodul

Das Auswertungsmodul besteht im Wesentlichen aus einer, durch das DSSIM Framework konfigurierten und deployten, *Grafana* Instanz.

Im Rahmen einer Szenariosimulation durch das Framework werden die Datenquellen automatisch eingestellt, vorkonfigurierte Dashboards installiert

¹⁵<https://prometheus.io/>

¹⁶<https://grafana.com/>

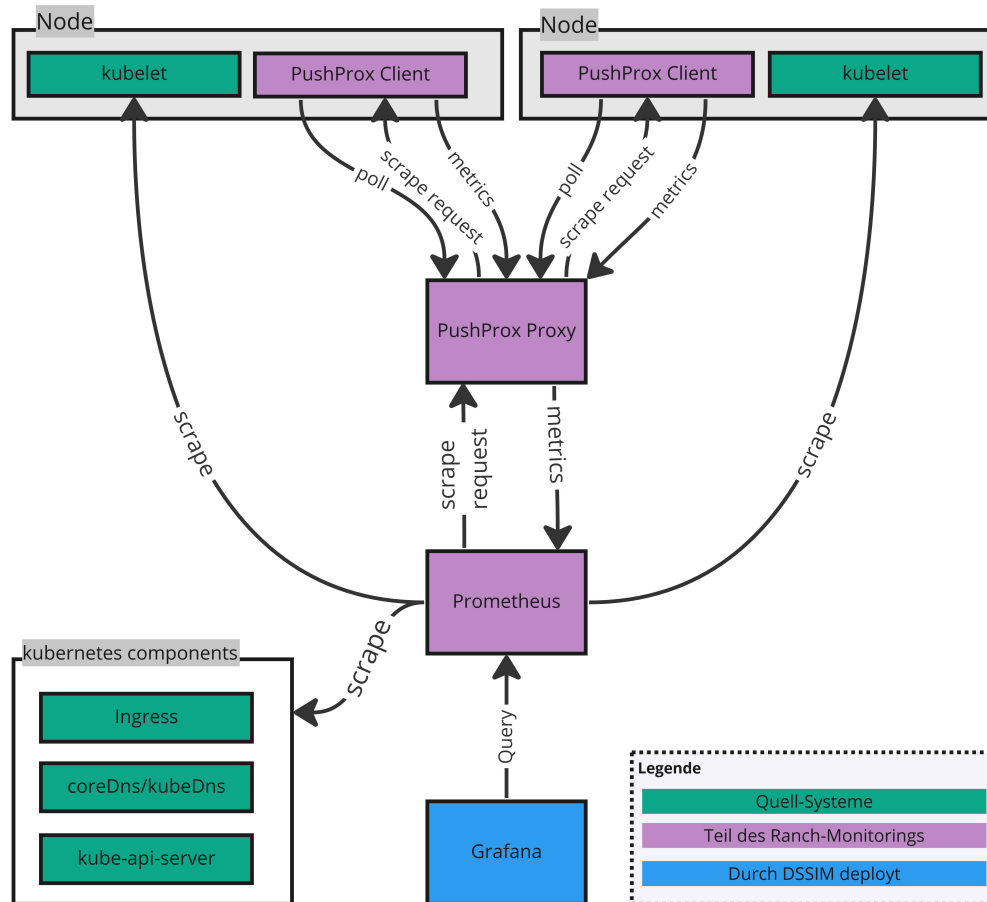


Abbildung 6.6: Scraping-Pipeline des Scenario-Monitoring Moduls (eigene Darstellung)

und Forschenden zur Verfügung gestellt. Die vorkonfigurierten Dashboards werden, ebenso wie die Szenariobeschreibungen und die Szenariokonfiguration, im *Scenario-Execution*-Modul geladen und bei der Initialisierung des *Monitoring*-Moduls übergeben.

Die Forschenden können diese Dashboards in der Weboberfläche interaktiv verwenden, die Konfiguration anpassen und auch neue Elemente (sog. *Panel*) hinzufügen. In vielen Fällen ist es sinnvoll, sich eigene Dashboards für die konkrete Problemstellung anzulegen, wie es auch für die Evaluation des Frameworks in Abschnitt 7.1 erfolgt ist. Abbildung 6.7 zeigt das Standard-Dashboard für eine allgemeine Szenarioüberwachung. Im oberen Bereich werden die Nachrichten des DSSIM Frameworks angezeigt, welche von der Sze-

nariobeschreibung selbst, dem *Scenario-Controller*, *Environment-Controller* oder anderen Framework-Modulen geloggt werden. Darunter befindet sich ein *Panel*, in dem alle Log-Nachrichten der Datenraumkomponenten zusammenlaufen. Darunter befinden sich Grafiken über den zeitlichen Verlauf von CPU Last, des Arbeitsspeicher- und Festplattenverbrauchs der einzelnen Nodes, sowie der Netzwerkdurchsatz je Netzwerkschnittstelle. Da jeder *Pod* eine dedizierte Netzwerkschnittstelle erhält, dann mit dem zuletzt genannten *Panel* auf die Netzwerklast der Datenraumkomponenten geschlossen werden. Auf der rechten Seite befinden sich verschiedene (teilweise visualisierte) Kennzahlen zum aktuellen Zustand des Clusters.

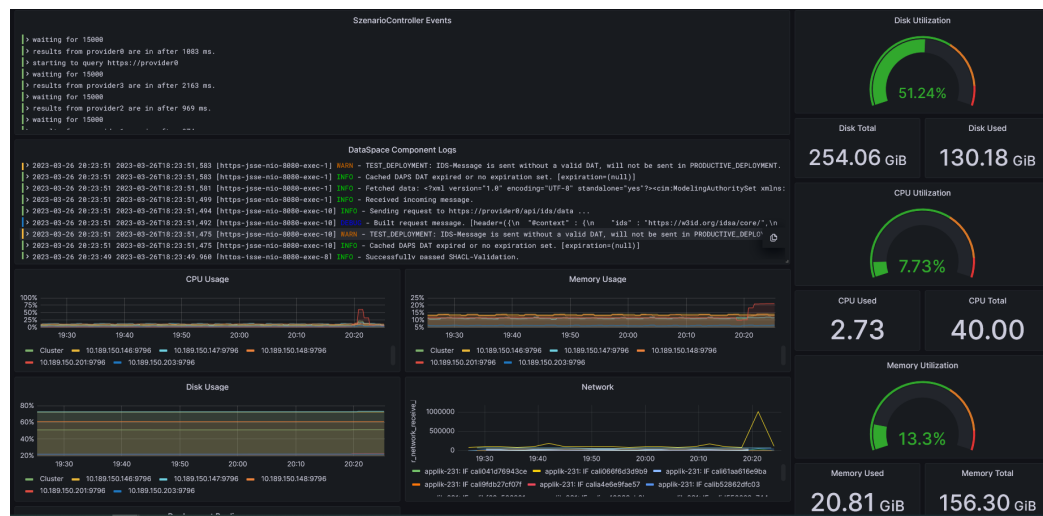


Abbildung 6.7: Beispiel für Dashboard zur Szenarioüberwachung

Kapitel 7

Evaluation

In den vorausgehenden Kapiteln wurde gezeigt, wie eine Architektur und Implementierung eines Frameworks für den speziellen Anwendungsbereich der Simulation von Datenräumen in Cloudinfrastrukturen aussehen kann. Die in diesem Kapitel folgende Evaluation analysiert und bewertet systematisch die Ergebnisse der Arbeit hinsichtlich ihrer Zielerreichung und Ergebnisse. Wie eingangs beschrieben erfolgt dies an Hand von (a) der Gegenüberstellung der ermittelten funktionalen und nicht-funktionalen Anforderungen mit den tatsächlich durch das Framework bereitgestellten Funktionen und (b) durch eine Betrachtung, wie die Ausführung des ausgewählten Szenarios bei der Gestaltung der Datenräume Erkenntnisse liefern konnten. Zur Gewährleistung der Praxisrelevanz wird zur Validierung der Ergebnisse ein vereinfachtes Szenario aus Projekten des Fraunhofer-Instituts herangezogen.

Um einer argumentativen Linie zu folgen, ist dieses Kapitel in zwei Teile aufgeteilt und wie folgt gegliedert: Der erste Teil 7.1 beschreibt das Szenario fachlich, seine technische Ausführung im Framework und seine Auswertung. Der zweite Teil 7.2 befasst sich mit der Betrachtung der initial formulierten Anforderungen und der tatsächlich erfolgten Umsetzung.

7.1 Szenario

In dem Abschnitt 7.1.1 erfolgt zunächst eine fachliche Beschreibung des Hintergrunds und welche Gestaltungsfragen sich in dem konkreten Anwendungsfall stellen. Daraus abgeleitet werden entsprechende Abläufe, die als Szenario simuliert werden. Eine Beschreibung der Umsetzung und Ausführung dieser Abläufe als Szenariobeschreibung im Framework wird in Abschnitt 7.1.2 erläutert. Der Abschnitt 7.1.3 beschäftigt sich dann mit der Auswertung der

Ausführung und mit möglichen gewonnenen Erkenntnissen aus der Simulation.

7.1.1 Fachliche Beschreibung

Das gewählte Szenario unterstützt eine Vorstudie zu dem Projekt *energy data-X*. Das Ziel des Gesamtprojekts ist *‘der Aufbau eines Datenraums für die Energiewirtschaft, der eine sichere und souveräne Datennutzung ermöglicht sowie die Grundlage für innovative Geschäftsmodelle legt’* [Fraunhofer IEE 2021]. Das Projekt wurde bereits im Juli 2021 im Rahmen eines GAIA-X Förderwettbewerbs ausgewählt und sollte vorbehaltlich der Verfügbarkeit ausreichender Haushaltsmittel einen Förderantrag in 2022 stellen können [BMWK 2021]. Die neu gewählte Bundesregierung kürzte jedoch entsprechende Mittel, woraufhin fünf Projekte aus dem Gaia-X Themenkontext keine Fördermittel erhielten. Hierzu gehörte neben dem *energy data-X* Datenraum auch ein weiterer Datenraum zur Unterstützung der Digitalisierung in der Forstwirtschaft [Tagesspiegel 2022]. Im Jahr 2023 laufen Bemühungen, zumindest Teilaspekte des ursprünglichen Projekts, mit anderen Mitteln zu finanzieren.

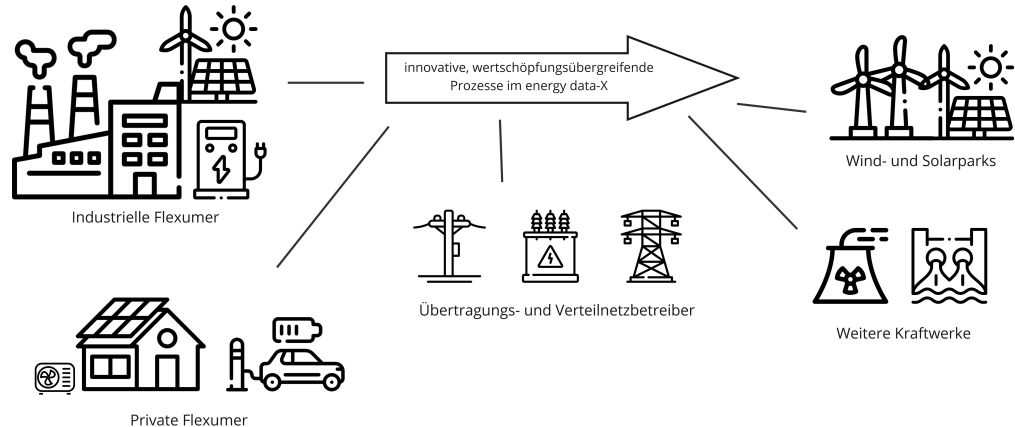


Abbildung 7.1: Schematische Darstellung Energy Data-X [BMWK 2023, S. 42]²

Der *energy data-X* Datenraum soll Energie-Flexibilitätsoptionen durch intelligenten und automatisierten Datenaustausch erschließen und als digitales Ökosystem innovative Geschäftsmodelle (z. B. netzdienliche Preise & Automatisierung, Redispatch, Flexibilitätshandel, co2-optimierter Produktfußabdruck durch intelligenten Energiebezug, Nachhaltigkeitsdatenaustausch

uvm.) ermöglichen. Abbildung 7.1 zeigt eine schematische Skizze eines solchen Datenraums. Dabei ist *Prosumer* eine Person oder ein Unternehmen, das sowohl Energie produziert als auch verbraucht. Dies kann beispielsweise durch die Installation von Solarzellen auf dem Dach eines Gebäudes erfolgen, um selbst produzierten Strom zu nutzen, während der überschüssige Strom ins Netz eingespeist wird. Ein *Flexumer* hingegen bezieht sich auf eine Person oder ein Unternehmen, das aufgrund seiner Flexibilität im Energieverbrauch in der Lage ist, Energie effektiver zu nutzen und das Stromnetz stabil zu halten. Zum Beispiel kann ein *Flexumer* seine Energieverbrauchsmuster anpassen, um auf Spitzen in der Stromnachfrage oder -versorgung zu reagieren. Durch die Bereitstellung von Flexibilität im Stromnetz kann ein *Flexumer* dazu beitragen die Integration erneuerbarer Energien zu erleichtern. [Forschungsstelle für Energiewirtschaft e. V. 2023][BMWK 2023]

Der Wechsel zu einem dezentralisierten und dekarbonisierten Energiemarkt erfordert große Veränderungen, um den gestiegenen Anforderungen an die Datenqualität (z. B. hochgranulare oder Echtzeit-Smart-Meter-Daten) und die Datenübertragung zur Sicherstellung der Versorgungssicherheit gerecht zu werden. Die derzeit verwendeten Technologien sowie das gegebene Design der Markt-Kommunikation, bei dem (Smart-) Meter-Werte alle 15 Minuten am folgenden Tag übertragen werden, werden in naher Zukunft an ihre Grenzen stoßen. Das *energy data-X* Projekt soll ein Datenraum aufbauen, der mögliche Lösungen definiert und testet, wie *Smart Meter* und *Sensordaten* identifizierten und autorisierten Teilnehmern des Datenraums verfügbar gemacht werden können. Unter Sensordaten werden Daten verstanden, die beispielsweise von Wechselrichtern von Solaranlagen oder unkalibrierten Zählern von Ladestationen für Elektroautos stammen. Dieser Datenraum soll die Grundlage bilden, für ein neues digitales Ökosystem. [Gaia-X European Association for Data and Cloud AISBL 2023c][Gaia-X European Association for Data and Cloud AISBL 2023a, S. 27]

Simulationsziel Um bereits zur Antragstellung und vor Projektstart erste Ansätze für die Datenraumgestaltung liefern zu können, soll mit dem Framework vorab ein Teilaspekt betrachtet werden. In dem Szenario sollen Leistungsdaten von mehreren Photovoltaikanlagen (PV-Anlagen) durch Betreiber in einem Datenraum angeboten werden. Diese Leistungsdaten sollen dann kontinuierlich an ein Empfänger übertragen werden, welcher auf Basis dieser und weiterer Daten Vorhersagen über die voraussichtliche Netzlast ermittelt.

Ablauf des Szenarios Von Bereitstellung bis Übertragung and Analyse-service müssen grob folgende Schritte durchlaufen werden:

- Selbstbeschreibung der Datenanbieter anfragen
- Durchsuchen des Datenangebots der Betreiber durch den Empfänger
- Vereinbaren von Nutzungsbedingungen
- Zyklische oder laufende Übertragung der Daten
- Übertragung der Daten an einen Prognoseservice

Fachliche und technische Gestaltungsvarianten Da das Projekt aktuell noch in Vorbereitung ist und die konkrete Ausgestaltung des Datenraumaufbaus Projektziel ist, gibt es bezüglich des Aufbaus noch keine konkreten Lösungen. Da Anbieter der Daten jedoch explizit auch kleine PV-Anlagen und Privatpersonen sind ist klar, dass mit möglichst geringen Ressourcenaufwand bezüglich der IT-Infrastruktur gearbeitet werden muss. Auch da die Bundesnetzagentur in einer Änderung des Energiewirtschaftsrechts im Zusammenhang mit dem Klimaschutz-Sofortprogramm klargestellt hat, dass Smart-Meter-Gateways (SMGWs) für die Übermittlung energiewirtschaftlich relevanter Mess- und Steuerungsvorgänge erforderlich ist [Bundesnetzagentur 2023], werden diese voraussichtlich eine zentrale Rolle in einem potenziellen *energy data-x* Datenraum spielen und gegebenenfalls als Plattform genutzt werden. Wie eine Architektur konkret aussehen kann, soll in dem Projekt durch Wissenschaft und Industrie gemeinsam erarbeitet werden.

Die Gestaltungsmöglichkeiten eines Datenraums für diesen Anwendungsfall sind vielfältig und bieten diverse Fragestellungen. Um den Zweck der Validierung des Frameworks bestmöglich dienlich zu sein, sollen Fragestellungen betrachtet werden, die auf spezielle Funktionen des DSSIM Frameworks angewiesen sind. Ausgesucht wurden daher zur näheren Analyse folgende besonders geeignete Aspekte:

- Unterschiede in der Komplexität des Aufbaus eines Szenarios mit DSC und EDC Technologie:
Diese Fragestellung ermöglicht eine Betrachtung der Frameworklösung zur Bereitstellung von gemeinsamen Schnittstellen über Datenraumtechnologien hinweg und der damit verbundenen Modularität des Frameworks. Darüber hinaus ist die Auswahl der Technologie eine zentrale Fragestellung, dessen Beantwortung sich kein Projekt entziehen kann.
- Einfluss schlechter Netzwerkverbindungen auf die Übertragungsdauer:
Eine Betrachtung dieses Aspekts erlaubt eine Demonstration, wie mit dem Framework auf einfachem Wege Sachverhalte analysiert werden können, dessen manueller Aufbau und Nachstellung erheblichen Aufwand und Kenntnisse erfordert. Auch wird dies eine Herausforderung beim Aufbau des *energy data-X* Datenraums, aufgrund der begrenzten Ressourcen bei beispielsweise einem SMGW, eine Rolle spielen.

7.1.2 Umsetzung im Framework

Als Datenquelle wurde ein Service genutzt, der im Rahmen des *SysAnDUk*-Projekts entstanden ist. Das Projekt wurde vom Fraunhofer-Institut in Zusammenarbeit mit verschiedenen Industriepartnern umgesetzt. Auch bei diesem Projekt stand der Wandel am Energiemarkt im Fokus: Hintergrund des Projekts war, dass dezentrale Stromerzeugung durch Solaranlagen, Windkraftanlagen oder Biogasanlagen für eine nachhaltigere Energieversorgung immer wichtiger werden. In kritischen Netzsituationen können diese Anlagen eine wichtige Rolle spielen, um Engpässe im Netz auszugleichen oder die Versorgungssicherheit zu gewährleisten. Systemdienliche Anforderungen beziehen sich dabei auf die Fähigkeit der Anlagen, flexibel auf die Anforderungen des Stromnetzes zu reagieren und bei Bedarf schnell reagieren zu können, um beispielsweise Frequenzschwankungen auszugleichen. Ein koordiniertes Management dieser dezentralen Anlagen kann dazu beitragen, das Stromnetz stabil zu halten und effektiv zu betreiben. [Fraunhofer IEE 2019]

Ein Teilergebnis des SysAnDUk-Projekts ist ein Datenservice, der reale Betriebsdaten von mehreren PV-Anlagen über eine API verfügbar macht. Dieser Service kennt jedoch keinerlei Nutzungsbedingungen und -vereinbarungen, Clearing Mechanismen, automatisierte Verträge und ähnliches. Idee ist es diese Daten über einen Connector in dem Datenraum zu teilen. Da ein Datenraum mit diversen Anbietern (Erzeugern) simuliert werden soll, werden die einzelnen Anlagen, deren Daten der *SysAnDUk*-Service zur Verfügung stellt jeweils als eigener Akteur am Datenraum auftreten. Für jede Anlage wird entsprechend ein eigener Connector gestartet und die Daten über die Schnittstelle geladen. Ein weiterer Connector wird als Empfänger gestartet. Dieser schließt mit den Anbietern entsprechende Nutzungsvereinbarungen ab, überträgt die Daten und leitet diese an ein Prognosesystem weiter. Die Implementierung und Verarbeitung im Prognosesystem wird in dem Szenario vernachlässigt und eine einfache Datensenke verwendet. Die Abbildung 7.2 stellt den Datenfluss vereinfacht dar.

Szenariobeschreibung

Für den ersten Teil der Evaluation, bei dem der Aufbau eines solchen Datenraums mit DSC und EDC verglichen werden soll, wurde zweistufig vorgegangen: Zunächst erfolgte eine Implementierung des Szenarios technologienah direkt mit den Services und Methoden der Connector-Bibliotheken, was die Unterschiede im grundsätzlichen Ablauf deutlich macht. Danach erfolgte eine Implementierung auf Basis des *DataSpace Component Interfaces* mithilfe der Fassade, welche dann mit unterschiedlichen Datenraumkonfigurationen

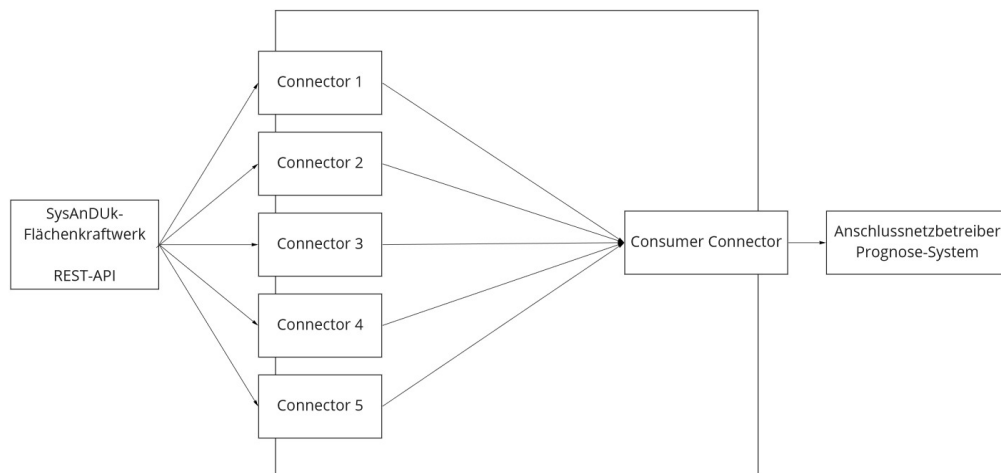


Abbildung 7.2: Datenfluss im Evaluationsszenario

ausgeführt werden kann. Diese prüft die Praktikabilität diverser Anforderungen, wie eine einfache Anwendung des Frameworks oder mit einer Szenariobeschreibung unterschiedliche Datenraumtechnologien und Konfigurationen simulieren und vergleichen zu können.

Die Implementierungen der Szenarien befinden sich im *dssim-scenarios* Repository und im Anhang Nummer 2.1, 2.2 und 2.3. Der Ablauf und die Unterschiede sollen hier kurz erläutert werden. Grundsätzlich wurde in der Szenariobeschreibung konsequent sequenziell vorgegangen und eine Strukturierung in Unterfunktionen vermieden, um den Ablauf aufzuzeigen und die Varianten besser vergleichen zu können.

Wie in Unterabschnitt 6.2.2 *Szenariobeschreibung* beschrieben, ist der Einstiegspunkt für Szenarien die *run* Methode von Klassen, die das *Szenario*-Interface implementieren. Zum Ausführungszeitpunkt der Methode der Datenraum bereits initialisiert. Das heißt, dass alle Infrastrukturkomponenten (wie z. B. das Monitoring oder die Netzwerk-Steuersysteme) gestartet und einsatzbereit sind. Dies beinhaltet nicht Datenraumkomponenten, welche im Verlauf eines Szenarios explizit angefordert werden.

Der Einstieg ist bei allen drei Varianten gleich: In der *run*-Methode wird zunächst ein Array definiert, welches alle Identifier der PV-Anlagen des *SysAnDUK* Service enthält. Dann wird asynchron ein Start von einem Connector je Element des Arrays angefordert und der Abschluss des Starts von allen Connectoren abgewartet. Bei den speziellen DSC und EDC Szenariobeschreibungen wird eine konkrete Instanz einer Datenraumkomponente über eine Factory erzeugt und ein ControllerTyp übergeben. Bei der generischen Beschreibung werden diese Parameter weggelassen. Der *Scenario-Controller*

wird dann die entsprechende Komponente der Szenariokonfiguration verwendet.

Folgend wird ein Datenangebot mit allen dazugehörigen Objekten auf jedem der gestarteten Connectoren erzeugt. Dieser Teil unterscheidet sich bei den drei unterschiedlichen Implementierungen wesentlich. Während die Bestückung bei dem EDC mit drei REST-Aufrufen je Connector erledigt ist, erfolgen bei der DSC Variante 16 Aufrufe, um die Konfiguration abzuschließen. Die Fassade bietet diese Funktion mit einem Methodenaufruf *createHttpEndpointOffer* an.

Dann wird ein Connector gestartet, welcher als *Consumer* agiert und die Daten von den zuvor gestarteten *Providern* abrufen. Als Datensenke wird folgend ein einfacher Service gestartet, welchen einen Endpunkt bereitstellt und die empfangenen Daten in der Konsole ausgibt. Für den DSC erfolgt an dieser Stelle dann die Konfiguration einer *Apache Camel* Route, an den die Daten bei Übertragung weitergeleitet werden. Der EDC registriert seine *Data Plane* (eine Erweiterung für den Datentransfer über HTTP). Die generische Szenariobeschreibung erlaubt das Setzen eines HTTP Datenempfängers über *setHttpDataReceiver*.

Die Phase des Aufbaus des Datenraums ist nun abgeschlossen. Es folgt der Ablauf der zu untersuchenden Schritte. Der *Consumer* untersucht die Datenangebote der *Provider* und handelt für das erste gefundene Datenangebot eine Nutzungsvereinbarung aus. Der Identifier der ausgehandelten Nutzungsvereinbarungen wird in einem Array gespeichert, um in weiteren Verlauf beim Abruf der Daten verwendet zu werden. Während der DSC eine synchrone Kommunikation verwendet und direkt eine Vereinbarung zurückgibt, wird bei dem EDC der Prozess nur initialisiert. Danach wird in regelmäßigen Abständen der aktuelle Stand abgefragt und der Ablauf des Szenarios fortgesetzt, wenn eine gültige Vereinbarung vorliegt. Bei der generischen Beschreibung werden diese Vorgänge mit der *negotiateContract* Methode gekapselt.

Als letzte Phase des Szenarios erfolgt die regelmäßige Datenübertragung. Nach einer zufälligen Verzögerung von 0 bis 5 Sekunden initiiert der *Consumer* einhundert Mal alle 15 Sekunden eine Datenübertragung der aktuellen PV-Leistungsdaten von jedem Provider. Auch hier findet bei dem DSC die Übertragung synchron statt und nach Abschluss des REST-Calls ist die Übertragung abgeschlossen. Bei dem EDC wird wiederum eine Übertragung nur initiiert und der Erfolg wird nachträglich in regelmäßigen Abständen erfragt. Außerdem ist zu beachten, dass der DSC die Daten nicht nur über die *Apache Camel* Route an die Datensenke schickt, sondern darüber hinaus an DSSIM zurückgibt. Dies kann bei skalierten Szenarien und bei Performancemessungen eine Herausforderung darstellen.

Für die Untersuchung des Einflusses der Netzwerkqualität wurde die generische Szenariobeschreibung mit der Fassade (2.3) verwendet. Mit den Methoden des *instanceController*, wie Quellcode 7.1 zeigt, wurde nach dem Start der Komponenten die Einschränkung in den verschiedenen Varianten vorgenommen. Der *Environment-Controller* des Frameworks setzt die definierten Bedingungen entsprechend um.

```
1 // Setzen der Paketverzögerung auf 500 ms
2 await connector.instanceController
3   .setNetworkDelay(500, 'ms');
4 // Setzen der Bandbreitenbeschränkung auf 56 kbps
5 await connector.instanceController
6   .setNetworkBandwidthLimit(56, 'kbps');
```

Quellcode 7.1: Beeinflussung Netzwerkqualität Connectoren

7.1.3 Erkenntnisse aus Szenarioaufbau und Simulation

Aufbau mit DSC und EDC

Bei einem Vergleich des Quellcodes 2.1 DSC spezifisch und 2.2 EDC spezifisch (siehe Anhang), sowie aus den Ausführungen zum Szenarioaufbau zeigt sich, dass die Bereitstellung eines Datenangebots und der Abschluss einer Nutzungsvereinbarung mit dem EDC mit deutlich weniger API Aufrufen erfolgt, als mit dem DSC. Dies macht die Verwendung der Komponenten einfacher und übersichtlicher.

Die Vereinbarung der Nutzungsbedingungen und der Datentransfer erfolgen bei dem DSC grundsätzlich synchron. Bei Anfrage der Daten beim Consumer Connector leitet dieser die Anfrage direkt an den Provider weiter, wartet auf Antwort und liefert das Ergebnis synchron zurück. Beim EDC wird der Datentransfer initiiert, ohne direkt ein Ergebnis zurückzuliefern. Der Transfer der Daten wird delegiert, asynchron durchgeführt und der Status nach Erfolg aktualisiert. In der Szenarioimplementierung wird daher in zyklischen Abständen überprüft, ob der Datentransfer erfolgreich war.

Da die Abläufe der beiden Technologien sehr unterschiedlich sind und Algorithmen der Fassade, wie das Polling auf den Status des Transfers oder die Datenübertragung der Ergebnisse an das Framework, die Ergebnisse verfälschen, wurde auf eine Performancebetrachtung verzichtet. Wegen der zuvor aufgeführten Gründe können in dem konkreten Fall wenig Rückschlüsse über Leistungsunterschiede der beiden Technologien getroffen werden. Dennoch kann es sinnvoll sein einen solchen Technologievergleich mit dem Framework durchzuführen, um die Funktionsweisen der Technologien deutlich zu machen. Die Nutzung der Fassade ist dabei nur dann sinnvoll, wenn man auf

einfachen und schnellen Weg die Funktion demonstrieren möchte. Diese kapselt jedoch die Komplexität so stark, dass Vergleiche nicht immer möglich sind. Außerdem werden durch die Nutzung der Fassade die verschiedenen Gestaltungsvarianten ausgeblendet.

Einfluss von schlechten Netzwerkverbindungen

Die Analyse des Einflusses von schlechten Netzwerkverbindungen wurde auf Basis der Szenariobeschreibung gegen das Interface (2.3) und mit dem DSC Connector durchgeführt. Sie sollen zeigen, welchen Einfluss schlechte Netzwerkverbindungen (hohe Latenz, geringe Bandbreite, Paketverlust) auf das Szenario haben. Gemessen wurde die Dauer der Ausführung der *transferArtifactsForAgreement* Methode der DSC-Fassade. Diese besteht aus zwei REST Calls, welche die Artefakt-URL vom Consumer holt und damit den Transfer von der PV-Anlage bis zur Datensenke durchführt. Die Szenariobeschreibung sendet eine Log-Nachricht vor und nach dem Transfer über die *Logging Pipeline* an das Monitoring Modul Grafana, mit einer eindeutigen und zuvor erzeugten *Request ID*. Grafana selektiert und gruppiert alle Lognachrichten nach der *Request ID* und berechnet den Zeitraum zwischen den Zeitstempeln des Erzeugungsmoments. Dieser Wert wird als *Übertragungsdauer* angenommen.

Für die Analyse der Ergebnisse wurde ein Szenario-spezifisches Dashboard angelegt, welches die Übertragungsdauer in unterschiedlichen Graphen darstellt. Abbildung 7.3 *Ausführung Szenario ohne Netzwerkeinschränkungen* zeigt die Durchführung ohne gesetzte Netzwerklimitierungen. Bei der Analyse der Ergebnisse in dem Monitoring Tool kommt man zu folgenden Erkenntnissen:

- In der Regel liegt die Übertragungsdauer zwischen 800 und 1200 Millisekunden (ms).
- Bei zwei Übertragungen lag die Dauer knapp über 7000 ms.
- Während zu Beginn des Szenarios die Übertragungsdauer regelmäßig unter 800ms lag, steigerte diese sich im Verlauf der Ausführung nach ca. 12 Minuten auf regelmäßig über 900ms und erreichte keinen Wert unter 880ms.

Um eine Analyse der Übertragungszeit unter unterschiedlichen Netzwerkbedingungen zu erreichen wurden in einem nächsten Schritt die Connectoren der Datenanbieter mit den in Tabelle 7.1 *Netzwerkeinstellungen in zweiter Ausführung* aufgelisteten Konfigurationen gestartet. Da keine realistischen und absehbaren Werte für das künftige Szenario vorliegen, wurden synthetische Werte verwendet, um die Nutzbarkeit zu demonstrieren und Einsatz-



Abbildung 7.3: Ausführung Szenario ohne Netzwerkeinschränkungen

gebiete aufzuzeigen. Als Referenz wurde ebenfalls ein Connector (*provider₄*) ohne Einschränkungen gestartet.

Connector	Verzögerung	Bandbreite
provider0		20 kbps
provider1	200 ms	
provider2	50 ms	
provider3	200 ms	50 kbps
provider4		

Tabelle 7.1: Netzwerkeinstellungen in zweiter Ausführung

Eine eingeführte Differenzierung zwischen den Connectoren auf dem Dashboard ermöglicht einen besseren Vergleich zwischen den Varianten. Die Durchführung mit diesen Einstellungen kam zu den in Abbildung 7.4 *Ausführung Szenario mit Netzwerkeinschränkungen* dargestellten Ergebnissen. Eine Analyse der Grafik erlaubt unter anderem folgende Beobachtungen:

- Es gab einen extremen Ausreißer mit ca. 9 Sekunden Übertragungszeit.

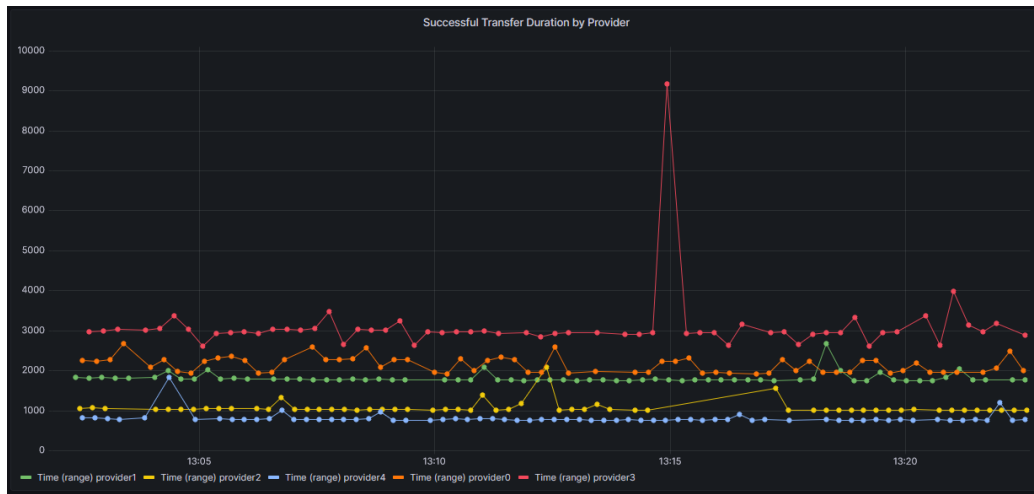


Abbildung 7.4: Ausführung Szenario mit Netzwerkeinschränkungen

- *provider4* und *provider2* hatten die geringsten Ausschläge in der Übertragungszeit, womit die Übertragungszeit am regelmäßigsten war.
- Der Referenz-Connector *provider4* ohne Netzwerkeinschränkungen hatte die geringsten Übertragungszeiten, die in der Regel bei ca. 750 ms lagen.
- Die 200 ms Verzögerung der Datenpakete von *provider1* führten regelmäßig zur Steigerung der Übertragungszeit auf ca. 1700 ms.
- Die 50 ms Verzögerung von *provider2* führte regelmäßig zu Steigerung der Übertragungszeit auf ca. 1000 ms.
- Mit einer Verzögerung von 200 ms und einer Bandbreiteneinschränkung von 50 kbps benötigte der *provider3* nicht nur deutlich länger (2689 ms bis 3903 ms), sondern zeigt auch deutliche Schwankungen.
- Die Gesamtausführungszeit des Szenarios war bei *provider3* vier Minuten länger als bei *provider4*.

Der erste Teil der Evaluation beschreibt, wie Datenräume und deren Abläufe mithilfe des Frameworks beschrieben werden können. Der zweite Teil zeigt das grundsätzliche Potenzial der interaktiven Analyse der Szenarioausführung mit dem Monitoring Modul des Frameworks.

7.2 Gegenüberstellung der Anforderungen & Funktionen

Im Folgenden werden die im Kapitel 3 *Anforderungen* gelisteten Anforderungen aufgeführt und bewertet, ob und wie diese Anforderungen mit dem Framework erfüllt werden konnten.

Unterstützte Datenraumfunktionen & -technologien Wie der vorhergehende Abschnitt zeigt, wurden alle notwendigen Funktionen hinreichend in den Modulen des Frameworks unterstützt, um eine Ausführung der ausgewählten Szenarien zu ermöglichen. Hierzu zählt beispielsweise die Veröffentlichung von Datenangeboten, die Vereinbarung von Nutzungsbedingungen und der Transfer von Daten. Für diese, aber auch für weitere Funktionen wurden generalisierte und vereinfachte Implementierungen über Fassaden bereitgestellt. Dank der *node*-Bibliotheken für die Datenraumkomponenten, die automatisiert auf Basis der jeweiligen OpenAPI-Beschreibungen generiert und direkt in der Szenariobeschreibung genutzt werden können, steht darüber hinaus der vollständige Funktionsumfang diverser Datenraumkomponenten zur Verfügung, wenn auf die Verwendung einer Fassade verzichtet wird. Dies gilt für den DSC, EDC, sowie dem *IDS Broker*. Auch *Instance* Erweiterungen für das Kubernetes *Environment-Controller*-Modul wurden für diese Komponenten und den *Identity Provider DAPS* erstellt, was ein Deployment im Rahmen von Szenarien ermöglicht.

Einfache Nutzbarkeit Das Framework ermöglicht es, einen Datenraum mit einfachen Szenarien ohne Infrastrukturkenntnisse und ohne spezifische Kenntnisse der eingesetzten Datenraumkomponenten zu erzeugen. Die selbstklärende Namensgebung der Methoden wie *startConnector* und die typsichere und IDE gestützte Erstellung von Szenariobeschreibungen tragen wesentlich zu einem niedrigschwelligen Einstieg in die Thematik bei. Für komplexere Szenarien und für das Debugging von Fehlern sind erweiterte Kenntnisse notwendig.

Spezialisierbarkeit Das Framework ermöglicht, die Standardimplementierung auf verschiedene Weisen anzupassen und eigene Konfigurationen, Module sowie Algorithmen zu nutzen, die auf konkrete Anwendungsfälle spezialisiert sind. Bezüglich der Infrastruktur können mithilfe des *Template*-Entwurfsmusters Deployment Schritte überschrieben und spezifische Konfigurationen deployt werden. Auch die DataSpace-Controller Implementie-

rungen erlauben die Nutzung des vollständigen Funktionsumfangs der eingesetzten Komponenten, ohne auf das Interface zu beschränken.

Hohe Erweiterbarkeit und Modularität Das Framework ist hochmodular entwickelt worden und nicht nur Framework-Module, sondern auch einzelne Klassen können ausgetauscht oder erweitert werden. Als wichtigste Werkzeuge zur Realisierung sind *Dependency Injection* und *Factories* zu nennen, welche die Abhängigkeit von konkreten Klassen vermeiden.

Trennung von Konfiguration und Ablauf Der in Unterabschnitt 6.2.1 *Szenariokonfiguration* beschriebene Aufbau erlaubt es, die Szenariokonfiguration außerhalb der Szenariobeschreibung zu definieren. So können sowohl verschiedene Szenarien mit der gleichen Konfiguration, als auch verschiedene Konfigurationen mit dem gleichen Szenario ausgeführt werden. Szenariospezifische Parameter können über Variablen in den Szenarios eingeführt und zentral definiert werden. Mit wenig Aufwand könnte das *Scenario-Execution* Modul dahingehend erweitert werden, dass vor Szenarioausführung die Parameter über das Userinterface eingegeben werden müssen.

Hardware Simulation Das Framework sieht entsprechende Einschränkungen in den Schnittstellen und Klassendefinitionen vor und für die Umsetzung ist die konkrete Implementierung des *Environment Controller* Modul verantwortlich. Der *Kubernetes-Controller*, als konkrete Ausprägung eines solchen, setzt die Einschränkung der Rechen- und Speicherkapazitäten über Standardfunktionalität von Kubernetes um. Für die Einschränkung des Netzwerkverkehrs werden, auf linux traffic control (tc) basierende, Softwarekomponenten deployt, die umfassende Einflussmöglichkeiten auf den Netzwerkverhalten ermöglichen.

Parallele Ausführung Durch die Aufteilung in Namensräume können mehrere Szenariosimulationen gleichzeitig ausgeführt werden. Hierbei gibt es jedoch zwei Einschränkungen: (a) Die Szenarien teilen sich die physikalischen Ressourcen der Cloudumgebung. Entsprechend können sich die Szenarien gegenseitig bezüglich der Leistung beeinflussen und punktuelle Performancemessungen sind nur eingeschränkt aussagekräftig. (b) Hostnamen für Datenraum- und andere Infrastrukturkomponenten, für die ein *Ingress* konfiguriert wird, können nicht doppelt vergeben werden.

Auswertung Als Teil des Monitoring Moduls wird zur Auswertung ein *Prometheus/Grafana* System durch das Framework konfiguriert und gest-

artet, an welches alle gesammelten Daten geleitet werden. Damit steht den Benutzern eine interaktive und umfassende Auswertungsmöglichkeit zur Verfügung. Als Einstiegspunkt wird standardmäßig ein Dashboard mit den Szenariologs und üblichen Statistiken zur CPU- und Arbeitsspeicherlast, sowie Netzwerkaktivität bereitgestellt. Benutzerspezifische Dashboards können zu den Szenarien über die Oberfläche konfiguriert und dessen Definitionen mit der Szenariobeschreibung abgelegt werden. Diese werden dann bei der Ausführung automatisch installiert.

Simulationsinfrastruktur Kubernetes wird durch eine Implementierung des *Environment-Controller* Moduls unterstützt. Alle Tests und Szenariodurchführungen haben mit Kubernetes stattgefunden.

Die Validierung anhand des Szenarios ein die vorhergehende Analyse der Anforderungen zeigt, dass alle eingangs genannten und für eine Evaluation des Frameworks erforderlichen Anforderungen hinreichend umgesetzt worden sind. Das Framework unterstützt die notwendigen Funktionen für die Evaluierung und ermöglicht es, auch komplexe Szenarien ohne spezifische Kenntnisse der eingesetzten Datenraumkomponenten zu erzeugen und ist zugleich hochmodular und erweiterbar. Auch die Trennung von Konfiguration und Ablauf sowie die Hardware Simulation tragen wesentlich zur Flexibilität des Frameworks bei. Insgesamt ist das Framework eine vielversprechende Lösung für die Simulation von Datenraumszenarien in der Cloud.

Kapitel 8

Zusammenfassung und Fazit

Diese Arbeit zeigt ausführlich, wie ein flexibles und modulares technisches Rahmenwerk für Simulation von Datenraumszenarien aufgebaut werden kann. Hierfür ist die aktuelle Relevanz der Forschung dargelegt und die Problemstellung beschrieben. Es ist die eingehende Idee dargelegt, ein Rahmenwerk zu entwerfen und beispielhaft zu implementieren, welches eine abstrakte Beschreibung von Datenraumszenarien erlaubt, aufwändigen und manuellen Aufbau automatisiert, die beschriebenen Szenarien ausführt und Möglichkeiten bietet die Ausführung zu analysieren.

Fachliche Ausführungen zu Grundlagen von digitalen Datenräumen, Softwarearchitekturen und genutzten Technologien bilden die Basis für weitere Ausführungen. Eine Analyse der Anforderungen und verwandten wissenschaftlichen Arbeiten konkretisiert die Ziele und stellt fest, dass bisher keine Lösungen für die Problemstellung beschrieben oder entwickelt worden sind. Jedoch finden sich in den Arbeiten Ansatzpunkte für Teilaspekte der Arbeit, wie Architekturen ähnlicher Systeme oder Simulation von Ressourcenbeschränkungen.

Der Hauptteil befasst sich mit der Darstellung des Frameworks und konkreten Ausprägungen der Module. Hier ist die modulare Gesamtarchitektur und verschiedene Designaspekte, beispielsweise mit welchen Entwurfsmustern die Kapselung durchgesetzt ist, beschrieben. In der Beschreibung der konkreten Modul-Implementierungen wird auf die speziellen Herausforderungen und Problemstellungen eingegangen und Lösungswege begründet.

In der Evaluation der Ergebnisse ist beschrieben, wie ein praxisnahes Szenario zum Austausch von Photovoltaik-Leistungsdaten mit zwei verschiedenen Datenraumtechnologien aufgebaut, ausgeführt und verglichen wurde. Darauf folgt eine Vorgehen- und Ergebnisbeschreibung der Durchführung des Szenarios unter verschiedenen Netzwerkbedingungen und eine Betrachtung

wie diese sich auf die Ausführungsdauer auswirkt. Eine qualitative Bewertung der initial formulierten Anforderungen schließt die Evaluation ab.

Folgende Schlussfolgerungen lassen sich aus dieser Forschungsarbeit ziehen:

Zielerreichung Die Forschungsarbeit zeigt, dass die vorgeschlagene Architektur und die Referenzimplementierungen für den Einsatzzweck geeignet sind. Forschende ohne fachkundiges Wissen in dem Themengebiet können mit dem Framework erste Experimente mit Datenräumen durchführen und Erfahrungen sammeln. Ebenso eignet es sich für Experten, die komplexe Szenarien aufbauen, erproben und so Erkenntnisse für laufende und künftige Forschungs- und Wirtschaftsprojekte ermitteln. Bereits zur Entwicklungszeit wurde das Framework für verschiedene Experimente von Kolleginnen und Kollegen eingesetzt.

Szenarien werden typischer und mit IDE Unterstützung gegen vereinfachte Schnittstellen der Datenraumkomponenten oder direkt unter Nutzung der technologiespezifischen Bibliotheken beschrieben. Der automatisierte Aufbau der Datenräume und die Ausführung der Szenarien ermöglichen erhebliche Zeiteinsparungen gegenüber dem manuellen Experimentieren. Eine Implementierung des *Environment-Controller* Moduls unterstützt mit Kubernetes die Nutzung des am weitesten verbreiteten *Container-Orchestrators* als Infrastruktur für die Ausführung von Simulationen. Zentrale Konfigurationen ermöglichen die Durchführung der Szenarien mit unterschiedlichen Technologien und unter unterschiedlichen Rahmenbedingungen. Die modulare Gestaltung des Frameworks eignet sich für ein dynamisches Anwendungsgebiet wie den Datenraumtechnologien und erlaubt eine Anpassung oder den Austausch der Module. Die Ausführung der Simulationen wird überwacht und eine Implementierung für das Monitoring Modul erlaubt eine interaktive und eingehende Analyse der erfassten Daten.

Abstrahierbarkeit der DataSpace-Component Schnittstelle Bei der Gestaltung des Frameworks stellte sich die Frage, ob es möglich und sinnvoll ist, die verschiedenen Implementierungen von Datenraumkomponenten auf eine gemeinsame Schnittstelle zu verallgemeinern. Die Evaluierung mit den Szenarien zeigt, dass es zwar für einige Funktionen grundsätzlich möglich ist, aber zwei Probleme damit einhergehen: Die Konstrukte, die eine Fassade aufbauen muss, um die gleiche Funktion mit verschiedenen technischen Komponenten zu erreichen, fügen dem Szenario Parameter hinzu, welche eine Vergleichbarkeit der Implementierungen nicht mehr gewährleisten können. Konkret ist in dem durchgeführten Szenario der Fall aufgetreten, dass der

DSC grundsätzlich eine synchrone Datenübertragung verfolgt, während der EDC asynchrone Datenübertragungen durchführt. Die Abstrahierung über die Fassade führte zu einer Unvergleichbarkeit der Ergebnisse.

Ebenfalls damit einher geht, dass DSSIM gerade dafür geschaffen worden ist, verschiedene Gestaltungsvarianten aufzubauen, zu testen und zu vergleichen. Werden die Varianten über eine solche Abstraktion versteckt, ist für den Forschenden nicht offensichtlich, wie der konkrete Aufbau erfolgt. Zielführender kann es sein, die Forschenden die spezifischen Möglichkeiten der Datenraumkomponenten direkt nutzen zu lassen und so den Aufbau der Varianten mit bewussten Entscheidungen herbeizuführen. Nur die Nutzung der vollen Funktionspalette der Komponenten ermöglicht es ihnen ihre Stärken voll auszuspielen. Das Framework erlaubt dies bereits über die direkte Verwendung von den speziellen APIs der Datenraumkomponenten über die exponierten *node*-Bibliotheken in den Szenarien.

Grenzen in der Skalierung Da das Framework eine zentrale Szenariosteuerung implementiert, sind Grenzen in der Skalierbarkeit gesetzt. Wenn beispielsweise eine zu hohe Zahl an Datenraumkomponenten gesteuert oder zu große Datenmengen ausgetauscht werden, wird die steuernde Einheit zum Engpass. Durch eine konsequente Verfolgung von Prinzipien, wie der Vermeidung von rechen- oder netzwerkintensiven Operationen, wird dieser Punkt verzögert. Eine Lösung kann eine dezentrale Szenariosteuerung bringen, wie in Kapitel 9 *Ausblick* beschrieben.

Multi-Tenancy in Kubernetes Während der Ausführung kann es unterschiedliche, durch das Framework nicht zu beeinflussende Faktoren geben, die eine Betrachtung erschweren. Hierzu zählt die Verteilung der *Pods* auf den *Nodes* oder die allgemeine Lastsituation im Cluster. Damit solche Analysen tatsächlich eine hohe Aussagekraft haben, müssen weitere Maßnahmen ergriffen werden. Hierzu zählt beispielsweise die Wiederholung der Szenarien zu unterschiedlichen Zeitpunkten oder eine Beobachtung der sonstigen Aktivitäten im Cluster.

Zusammenfassend kann festgestellt werden, dass das vorgestellte technische Rahmenwerk eine geeignete Lösung für die Simulation von Datenraumszenarien bietet. Durch die modulare Architektur, den automatisierten Aufbau der Szenarien und die Möglichkeit zur Analyse der Ausführungsergebnisse können Zeit- und Ressourceneinsparungen erzielt werden. Das Framework eignet sich sowohl für Forscherinnen und Forscher ohne fachkundiges Wissen als auch für Expertinnen und Experten, die komplexe Szenarien auf-

bauen und erproben möchten. Die Evaluierungsergebnisse zeigen, dass das Framework erfolgreich eingesetzt werden kann und die gesteckten Ziele erreicht wurden. Zu verschiedenen Teilaspekten, wie einer Skalierung der ausgeführten Szenarien oder *Multi-Tenancy* Aspekten existiert noch weiteres Forschungspotential, auf das im folgenden Kapitel weiter eingegangen wird.

Kapitel 9

Ausblick

In diesem Kapitel sollen Anknüpfungspunkte für weitere Forschungen, mögliche Weiterentwicklungen und Themen, dessen Ausarbeitung den Umfang dieser Arbeit übertreffen, aufgeführt werden.

Erweiterung der Funktionalität Aktuell werden durch die *DataSpace-Component-Controller* nur eine Auswahl an Funktionen bestimmter Datenraumkomponenten zur Verfügung gestellt. Bei einer breiten Anwendung des Frameworks werden voraussichtlich weitere Komponenten, mehr Funktionen und erweiterte Parameter für komplexere Szenarien notwendig werden. Hierzu zählen z. B. eine Erweiterung um unterstützte Komponenten wie dem *Clearing House* oder die Anbindung andersartiger Datenquellen.

Dezentrale Szenariosteuerung Um die Herausforderungen der Konnektivität und Skalierbarkeit zu lösen, könnte eine dezentrale Szenariosteuerung in das Framework eingeführt werden. Hierfür bieten sich verschiedene Lösungen an. Beispielsweise könnte die zentrale Szenariosteuerung durch dezentrale *Agenten* ergänzt oder ersetzt werden. Diese *Agenten* könnten auf Basis ihres eigenen Algorithmus als autarke Einheit agieren und mithilfe von Events mit anderen Agenten kommunizieren. In der *Kubernetes-Controller* Implementierung könnten die Datenraumkomponenten, die jeweils für sie zuständigen Agenten, beispielsweise mit dem *Sidecar* Pattern möglich und eine Kommunikation der Agenten untereinander könnte durch Event-Streams realisiert werden.

Technologie-agnostische Deployment Definitionen Eine technologie-agnostische Beschreibung der Deployments von Datenraumkomponenten würde es erleichtern die Szenarien in unterschiedlichen Umgebungen und mit ver-

schiedenen *Environment-Controllern* zu deployen ohne, dass für jede Zielfrastruktur eigene Definitionen für die Datenraumkomponenten geschrieben werden müssen. Hier wäre ein Konzept denkbar, bei dem die Deployments so abstrakt beschrieben sind, dass unterschiedliche *Environment-Controller* diese interpretieren und nutzen könnten.

DSL für Szenariobeschreibungen Analog des, in verwandten Arbeiten beschriebenen, *Cloud Security Testbeds* könnte es eine Domain Specific Language (DSL) für Datenraumszenarien geben. Damit könnten der Aufbau und die Abläufe der Simulation ohne Entwicklungserfahrung und -umgebung formuliert werden. Diese Beschreibungen könnten durch eine entsprechendes *Szenario-Execution* Modulimplementierung erfolgen.

Es zeigt sich, dass trotz des Umfangs dieser Arbeit, Themen noch Potenzial für weitere Untersuchungen bieten. Auch wenn noch nicht alle Aspekte möglicher Simulationen und Konstellationen adressiert sind, schafft das Framework DSSIM eine Basis auf der Weiterentwicklungen und Lösungen für spezifische Herausforderungen aufbauen können. Es legt den Grundstein für eine vereinfachte Forschung an Datenräumen und unterstützt somit die Weiterentwicklung jener.

Literatur

- Almadi, Sara H. S., Danial Hooshyar und Rodina Binti Ahmad (2021). „Bad Smells of Gang of Four Design Patterns: A Decade Systematic Literature Review“. In: *Sustainability* 13.18. ISSN: 2071-1050. DOI: 10.3390/su131810256. URL: <https://www.mdpi.com/2071-1050/13/18/10256>.
- Berlin Institute of Health in der Charité (BIH) (2022). *13 Millionen Euro für den Gesundheitsdatenraum HEALTH-X dataLOFT in Gaia-X*. <https://www.bihealth.org/de/aktuell/13-millionen-euro-fuer-den-gesundheitsdatenraum-health-x-dataloft-in-gaia-x>. (Besucht am 04.11.2022).
- BMDV (2022). *Verknüpfung kommunaler, regionaler und nationaler Datenplattformen durch Data-Space-Konzepte sowie Veredelung und Verwertung als Mobilitätsdaten-Ökosystem - Mobility Data Space*. <https://www.bmdv.bund.de/SharedDocs/DE/Artikel/DG/mfund-projekte/mobility-data-space.html>. (Besucht am 24.10.2022).
- BMWK (2021). *Der deutsche Gaia-X Hub*. <https://www.bmwk.de/Redaktion/DE/Dossier/gaia-x.html>. (Besucht am 01.03.2023).
- (2023). *Sammelband Pitchday Datenraum Industrie 4.0*. https://www.plattform-i40.de/IP/Redaktion/DE/Downloads/Publikation/Sammelband_Pitchday_Datenraum_Industrie40.html. (Besucht am 02.03.2023).
- Brown, Martin A (2006). „Traffic control howto“. In: *Guide to IP Layer Network* 49, S. 36.
- Budny, Peter, Srihari Govindharaj und Karsten Schwan (2008). „WorldTravel: A Testbed for Service-Oriented Applications“. In: *Service-Oriented Computing – ICSOC 2008*. Hrsg. von Athman Bouguettaya, Ingolf Krueger und Tiziana Margaria. Berlin, Heidelberg: Springer Berlin Heidelberg, S. 438–452. ISBN: 978-3-540-89652-4.

- Bundesnetzagentur (2023). *Veröffentlichung eines Positionspapiers zur Konkretisierung der Reichweite energiewirtschaftlich relevanter Mess- und Steuerungsvorgänge nach § 19 Abs. 2 MsbG*. https://www.bundesnetzagentur.de/DE/Beschlusskammern/1_GZ/BK6-GZ/2022/BK6-22-253/BK6-22-253_Veroeffentl_PoPa_ERD.html. (Besucht am 02.03.2023).
- Catena-X Automotive Network e.V. (2022). *Über uns*. <https://catena-x.net/de/ueber-uns>. (Besucht am 04.11.2022).
- Chakraborty, Mainak und Ajit Pratap Kundan (2021). „Grafana“. In: *Monitoring Cloud-Native Applications: Lead Agile Operations Confidently Using Open Source Software*. Berkeley, CA: Apress, S. 187–240. ISBN: 978-1-4842-6888-9. DOI: 10.1007/978-1-4842-6888-9_6. URL: https://doi.org/10.1007/978-1-4842-6888-9_6.
- CNCF (2022). *CNCF Annual Survey*. <https://www.cncf.io/reports/cncf-annual-survey-2022/>. (Besucht am 15.04.2023).
- CNI Projekt (2023). *bandwidth plugin*. <https://www.cni.dev/plugins/current/meta/bandwidth/>. (Besucht am 09.01.2023).
- Curry, Edward (Jan. 2015). „The Big Data Value Chain: Definitions, Concepts, and Theoretical Approaches“. In: ISBN: 978-3-319-21568-6. DOI: 10.1007/978-3-319-21569-3_3.
- Dabrock, Peter (2017). *Stellungnahme des Deutschen Ethikrates „Big Data und Gesundheit. Datensouveränität als informationelle Freiheitsgestaltung“*.
- Datadog (2022). *9 insights on real-world container use*. <https://www.datadoghq.com/container-report/>. (Besucht am 15.04.2023).
- Dr. Robert Habeck (2022). *Videobotschaft von Dr. Robert Habeck, Bundesminister für Wirtschaft und Klimaschutz*. <https://catena-x.net/de/vision-ziele/robert-habeck>. (Besucht am 04.11.2022).
- DSBA, Data Space Business Alliance (2022). *Technical Convergence*. <https://data-spaces-business-alliance.eu/dsba-releases-technical-convergence-discussion-document/>. (Besucht am 22.11.2022).
- Dudenredaktion (2022). „Test“ auf Duden online. <https://www.duden.de/node/181341/revision/1418664>. (Besucht am 14.11.2022).

- Eclipse Foundation (2023). *Eclipse Dataspace Components*.
<https://projects.eclipse.org/projects/technology.edc>.
(Besucht am 19.02.2023).
- etcd Projekt (2023). *Frequently asked questions*.
<https://etcd.io/docs/v3.5/faq/>. (Besucht am 05.04.2023).
- Forschungsstelle für Energiewirtschaft e. V. (2023). *Flexumer als Gestalter einer digitalen Energiezukunft – eine Begriffseinordnung*. <https://www.ffe.de/veroeffentlichungen/flexumer-als-gestalter-einer-digitalen-energiezukunft-eine-begriffseinordnung/>.
(Besucht am 02.03.2023).
- Franklin, Michael, Alon Halevy und David Maier (Dez. 2005).
„From databases to dataspace: a new abstraction for information management“. In: *SIGMOD Rec.* 34.4, S. 27–33. ISSN: 0163-5808.
DOI: 10.1145/1107499.1107502.
URL: <http://doi.acm.org/10.1145/1107499.1107502>.
- Fraunhofer IEE (2019). *Forschungsprojekt - SysAnDUk*. <https://www.ief.fraunhofer.de/de/projekte/suche/laufende/SysAnDUk.html>.
(Besucht am 02.03.2023).
- (2021). *Konsortium ‘energy data-X’ im GAIA-X Förderwettbewerb des Bundeswirtschaftsministeriums vorausgewählt*.
<https://www.ief.fraunhofer.de/de/presse-infothek/Presse-Medien/Pressemitteilungen/2021/konsortium--energy-data-x--in-foerderwettbewerb-vorausgewaehlt.html>.
(Besucht am 01.03.2023).
- Fraunhofer ISST (2021a). *DataspaceConnector Documentation - Build*.
<https://international-data-spaces-association.github.io/DataspaceConnector/Deployment/Build>.
(Besucht am 19.02.2023).
- (2021b). *Introduction*. <https://international-data-spaces-association.github.io/DataspaceConnector/Introduction>.
(Besucht am 19.02.2023).
- Fraunhofer-Gesellschaft (2022). *Industrial Data Space e.V. gegründet*.
<https://www.fraunhofer.de/de/presse/presseinformationen/2016/januar/industrial-data-space-ev-gegruendet.html>.
(Besucht am 16.12.2022).
- Freeman, Adam (2021). *Essential TypeScript 4*. Apress Berkeley, CA.
ISBN: 978-1-4842-7011-0. DOI: 10.1007/978-1-4842-7011-0.
- Fu, Huaiguo (2006). „Formal Concept Analysis for Digital Ecosystem“. In: *2006 5th International Conference on Machine Learning and Applications (ICMLA’06)*, S. 143–148. DOI: 10.1109/ICMLA.2006.24.

- Gaia-X European Association for Data and Cloud AISBL (2023a).
The energy dataspace.
https://www.gaia-x.eu/sites/default/files/2021-06/Gaia-X_Data-Space-Energy_Position-Paper.pdf.
 (Besucht am 02.03.2023).
- (2023b). *Use cases*.
<https://gaia-x.eu/what-is-gaia-x/about-gaia-x/>.
 (Besucht am 02.03.2023).
- (2023c). *Use cases*. <https://gaia-x.eu/use-cases/>.
 (Besucht am 02.03.2023).
- Gamma, Erich u. a. (1994).
Design Patterns: Elements of Reusable Object-Oriented Software.
 1. Aufl. Addison-Wesley Professional. ISBN: 0201633612.
 URL: http://www.amazon.com/Design-Patterns-Elements-Reusable-Object-Oriented/dp/0201633612/ref=ntt_at_ep_dpi_1.
- Gelhaar, Joshua und Boris Otto (Juni 2020).
 „Challenges in the Emergence of Data Ecosystems“.
 In: *PACIS 2020 Proceedings*. 175.
- Gharbi, Mahbouba u. a. (2018). *Basiswissen für Softwarearchitekten*.
 dpunkt.verlag.
- GitHub (2023). *The top programming languages*.
<https://octoverse.github.com/2022/top-programming-languages>.
 (Besucht am 22.02.2023).
- Goll, Joachim (2018). *Cloud Native Architecture and Design*.
 Springer Vieweg Wiesbaden. ISBN: 978-3-658-20055-8.
 DOI: 10.1007/978-3-658-20055-8.
- Goniwada, Shivakumar R (2021). *Cloud Native Architecture and Design*.
 Apress Berkeley, CA. ISBN: 978-1-4842-7225-1.
 DOI: 10.1007/978-1-4842-7226-8.
- Günther, Marco und Kai Velten (2014).
Mathematische Modellbildung und Simulation.
 WILEY-VCH Verlag GmbH & Co.KGAA.
- IDSA (2022). *Data Connector Report, Version 1.0*.
https://internationaldataspaces.org/wp-content/uploads/dlm_uploads/IDSA-Data-Connector-Report-November-2022.pdf. (Besucht am 22.11.2022).
- (2023a). *Data Connector Report No.5*.
<https://internationaldataspaces.org/data-connector-report/>.
 (Besucht am 20.03.2023).

- (2023b). *Dataspace Protocol Preview*.
<https://internationaldataspaces.org/events/dataspace-protocol-preview-2/>. (Besucht am 12.03.2023).
- IEEE Standard Glossary of Software Engineering Terminology* (1990).
In: *IEEE Std 610.12-1990*, S. 1–84.
DOI: 10.1109/IEEESTD.1990.101064.
- International Data Spaces e. V. (2023).
The name now clearly shows the strategic orientation.
<https://internationaldataspaces.org/the-name-now-clearly-shows-the-strategic-orientation/>. (Besucht am 05.01.2023).
- Istio Projekt (2023). *Enabling Rate Limits*.
<https://istio.io/v1.4/docs/tasks/policy-enforcement/rate-limiting/>. (Besucht am 09.01.2023).
- Johnson, Ralph E und Brian Foote (1988). „Designing reusable classes“.
In: *Journal of object-oriented programming* 1.2, S. 22–35.
- Kaufmann, Tobias, Johannes Mayer und Philipp Niemietz (2021).
„Datenökonomie - Wie eine geteilte Datenbasis den Nutzen für alle Stakeholder maximieren kann“. In: *Sicherer Datenaustausch*, S. 33.
- Khatri, A. u. a. (2020). *Mastering Service Mesh: Enhance, secure, and observe cloud-native applications with Istio, Linkerd, and Consul*.
Packt Publishing. ISBN: 9781789611946.
- Koutroumpis, Pantelis, Aija Leiponen und Llewellyn DW Thomas (2017).
The (unfulfilled) potential of data marketplaces. Techn. Ber.
ETLA Working Papers.
- Kubernetes Projekt (2023a). *Ingress*. <https://kubernetes.io/docs/concepts/services-networking/ingress/>.
(Besucht am 09.01.2023).
- (2023b). *Kubernetes Components*.
<https://kubernetes.io/docs/concepts/overview/components/>.
(Besucht am 09.01.2023).
- (2023c). *Network Plugins*. <https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/#support-traffic-shaping>.
(Besucht am 09.01.2023).
- (2023d). *Overview*.
<https://kubernetes.io/de/docs/concepts/overview/what-is-kubernetes>. (Besucht am 09.01.2023).
- (2023e). *Overview*.
<https://kubernetes.io/docs/concepts/security/multi-tenancy/>.
(Besucht am 09.01.2023).

- Kubernetes Projekt (2023f).
Resource Management for Pods and Containers.
<https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>. (Besucht am 09.01.2023).
- (2023g). *Services, Load Balancing, and Networking.*
<https://kubernetes.io/docs/concepts/services-networking/>.
 (Besucht am 09.01.2023).
- (2023h).
The Distributed System ToolKit: Patterns for Composite Containers.
<https://kubernetes.io/blog/2015/06/the-distributed-system-toolkit-patterns/>. (Besucht am 09.01.2023).
- (2023i). *The Kubernetes API.* <https://kubernetes.io/docs/concepts/overview/kubernetes-api/>.
 (Besucht am 09.01.2023).
- (2023j). *The Kubernetes API.* <https://kubernetes.io/docs/concepts/overview/kubernetes-api/>.
 (Besucht am 09.01.2023).
- (2023k). *Workloads.*
<https://kubernetes.io/docs/concepts/workloads/>.
 (Besucht am 09.01.2023).
- Kumar, Ritik und Munesh Chandra Trivedi (2021). „Networking Analysis and Performance Comparison of Kubernetes CNI Plugins“. In: *Advances in Computer, Communication and Computational Sciences*. Hrsg. von Sanjiv K. Bhatia u. a. Singapore: Springer Singapore, S. 99–109.
- Lasson, A. (1907). *Aristoteles Metaphysik.*
 URL: <http://www.zeno.org/nid/20009149392>.
- Li, Wenbin, Youakim Badr und Frédérique Biennier (Okt. 2012). „Digital ecosystems: Challenges and prospects“. In: S. 117–122.
 DOI: 10.1145/2457276.2457297.
- Mader, Christian u. a. (2023).
International Data Spaces Information Model.
 (Besucht am 14.03.2023).
- Massol, Vincent und Ted Husted (2004). *JUnit in Action.*
 Birmingham: Manning. ISBN: 978-1-930-11099-1.
- Mathis, Christian (Nov. 2017). „Data Lakes“. In: *Datenbank-Spektrum* 17.3, S. 289–293. ISSN: 1610-1995.
 DOI: 10.1007/s13222-017-0272-7.
 URL: <https://doi.org/10.1007/s13222-017-0272-7>.
- Microsoft (2022). *Do's and Don'ts.*
<https://www.typescriptlang.org/docs/handbook/declaration->

- [files/do-s-and-don-ts.html#use-optional-parameters](#).
(Besucht am 17. 02. 2023).
- Minna, Francesco und Fabio Massacci (2022).
„An Open-Source Cloud Testbed for Security Experimentation“.
In: *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, S. 756–759.
DOI: 10.1109/CCGrid54584.2022.00086.
- Moore, James (1993).
„Predators and Prey: A New Ecology of Competition“.
In: *Harvard business review* 71, S. 75–86.
- Morgan, William (2017). *What’s a service mesh? And why do I need one?*
<https://web.archive.org/web/20180104233448/https://buoyant.io/2017/04/25/whats-a-service-mesh-and-why-do-i-need-one/>. (Besucht am 05. 04. 2023).
- Nagel, Lars und Douwe Lycklama (Apr. 2021).
Design Principles for Data Spaces. Version 1.0.
DOI: 10.5281/zenodo.5244997.
URL: <https://doi.org/10.5281/zenodo.5244997>.
- OpenJS Foundation (2022a). *Introduction to Node.js*.
<https://nodejs.dev/en/learn/>. (Besucht am 16. 02. 2023).
- (2022b). *Worker threads*.
https://nodejs.dev/en/api/v19/worker_threads/.
(Besucht am 16. 02. 2023).
- Otto, Boris, Michael ten Hompel und Stefan Wrobel (2022). *Designing Data Spaces: The Ecosystem Approach to Competitive Advantage*.
Springer International Publishing. ISBN: 9783030939779.
- Otto, Boris, Sebastian Steinbuß u. a. (2019).
International Data Space-Reference Architecture Model 3.0.
<https://internationaldataspaces.org/publications/ids-ram/>.
(Besucht am 21. 10. 2022).
- Otto, Michel (2021). *Anforderungen an Datenmarktplätze bezüglich souveräner Datenbereitstellung*. unveröffentlicht.
- Pampus, Julia, Brian-Frederik Jahnke und Ronja Quensel (2022).
„Evolving Data Space Technologies: Lessons Learned from an IDS Connector Reference Implementation“.
In: *Leveraging Applications of Formal Methods*.
- Parmar, Rashik u. a. (2014). „The new patterns of innovation“.
In: *Harvard Business Review* 92.1, S. 2.
- Pfeifer, Wolfgang u. a. (2022). „Simulation“, in: *Etymologisches Wörterbuch des Deutschen (1993), digitalisierte und von Wolfgang Pfeifer*

- überarbeitete Version im Digitalen Wörterbuch der deutschen Sprache.
<https://www.dwds.de/wb/Simulation>. (Besucht am 14.11.2022).
- Preston-Werner, Tom (2013). *Semantic Versioning 2.0.0. (2013)*.
<https://semver.org/>. (Besucht am 15.02.2023).
- Prometheus Projekt (2023). *Overview*.
<https://prometheus.io/docs/introduction/overview/>.
 (Besucht am 05.04.2023).
- PushProx Projekt (2023). *PushProx*.
<https://github.com/prometheus-community/PushProx>.
 (Besucht am 20.02.2023).
- Sauerbier, Thomas (1999). *Theorie und Praxis von Simulationssystemen: Eine Einführung für Ingenieure und Informatiker*. Studium Technik. Vieweg+Teubner Verlag. ISBN: 9783528038663.
- Schriftliche Frage an die Bundesregierung im Monat September 2021. Frage Nr. 294* (2021).
https://www.bmwk.de/Redaktion/DE/Parlamentarische-Anfragen/2021/09/9-294.pdf?__blob=publicationFile&v=4.
- Sovity (2023). *Fraunhofer-Ausgründung sovity GmbH startet Data Sovereignty as a Service für Unternehmen*.
<https://internationaldataspaces.org/events/dataspace-protocol-preview-2/>. (Besucht am 20.03.2023).
- SUSE (2023a). *Architecture*.
<https://ranchermanager.docs.rancher.com/v2.6/integrations-in-rancher/logging/logging-architecture>.
 (Besucht am 18.01.2023).
- (2023b). *How Monitoring Works*.
<https://ranchermanager.docs.rancher.com/v2.6/integrations-in-rancher/monitoring-and-alerting/how-monitoring-works>.
 (Besucht am 18.02.2023).
- (2023c). *Monitoring and Alerting*.
<https://ranchermanager.docs.rancher.com/v2.6/pages-for-subheaders/monitoring-and-alerting>. (Besucht am 18.02.2023).
- Symeonides, Moysis u. a. (2020).
 „Fogify: A fog computing emulation framework“.
 In: *2020 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, S. 42–54.
- Tagesspiegel (2022). *Ampelregierung spart bei Digitalprojekten*. <https://background.tagesspiegel.de/digitalisierung/ampelregierung-spart-bei-digitalprojekten>. (Besucht am 01.03.2023).
- Tansley, A. G. (1935).
 „The Use and Abuse of Vegetational Concepts and Terms“.

- In: *Ecology* 16.3, S. 284–307. ISSN: 00129658, 19399170. URL:
<http://www.jstor.org/stable/1930070> (besucht am 28.11.2022).
- Thomas, Llewellyn DW und Aija Leiponen (2016).
„Big data commercialization“.
In: *IEEE Engineering Management Review* 44.2, S. 74–90.
- Woods, Dan (2023). *Big data requires a big architecture*.
(Besucht am 07.03.2023).
- Yilmaz, Onur (2021). *Extending Kubernetes*. Apress Berkeley, CA.
ISBN: 978-1-4842-7094-3. DOI: 10.1007/978-1-4842-7095-0.
- Yourdon, E. und L.L. Constantine (1979). *Structured Design: Fundamentals
of a Discipline of Computer Program and Systems Design*.
Yourdon Press computing series. Prentice Hall. ISBN: 9780138544713.

Akronyme

API Application Programming Interfaces. 16, 27, 46, 49, 57, 62, 65, 66, 69, 71, 76, 77, 85, 88, 97

ASL Attack Specific Language. 39

CNI Container Network Interface. 27, 28, 72, 74, 75, 106

CPU Central Processing Unit. 3, 30, 38, 46, 71, 72

DSC DataSpace Connector. 18, 45, 48, 57, 62, 63, 66, 84, 85, 86, 87, 88, 89, 92, 97

DSL Domain Specific Language. 38, 39, 100

DSSIM Dataspace **S**enario **S**imulation Framework. 2, 25, 41, 46, 49, 51, 53, 59, 61, 66, 75, 76, 78, 79, 84, 87, 97, 100

DSSP DataSpace Support Platform. 12

EDC Eclipse Dataspace Connector. 18, 45, 48, 62, 64, 66, 84, 85, 86, 87, 88, 92, 97

GoF Gang of Four. 22, 62, 69

HTTP Hypertext Transfer Protocol. 28

IaC Infrastructure-as-Code. 39

IDE Integrated development environment. 92, 96

IDS International Data Space. 14, 15

IDSA International Data Spaces Association. 11, 12, 14, 17, 18, 48

JSON JavaScript Object Notation. 38, 44

kbps Kilobit pro Sekunde. 90, 91

MAL Meta Attack Language. 39

ms Millisekunden. 89, 90, 91

ODRL Open Digital Rights Language. 15

PaC Plattform-as-Code. 39

PV Photovoltaik. 83, 84, 85, 86, 87

RAM Reference Architecture Model. 14, 15

REST Representational State Transfer. 63, 66, 69, 87

SDK Software Development Kit. 38

SMGW Smart-Meter-Gateway. 84

SOA serviceorientierte Architektur. 39

tc linux traffic control. 38, 72, 73, 74, 93

TSDB Prometheus Time Series Database. 30, 78

YAML Yet Another Markup Language. 38

Abbildungsverzeichnis

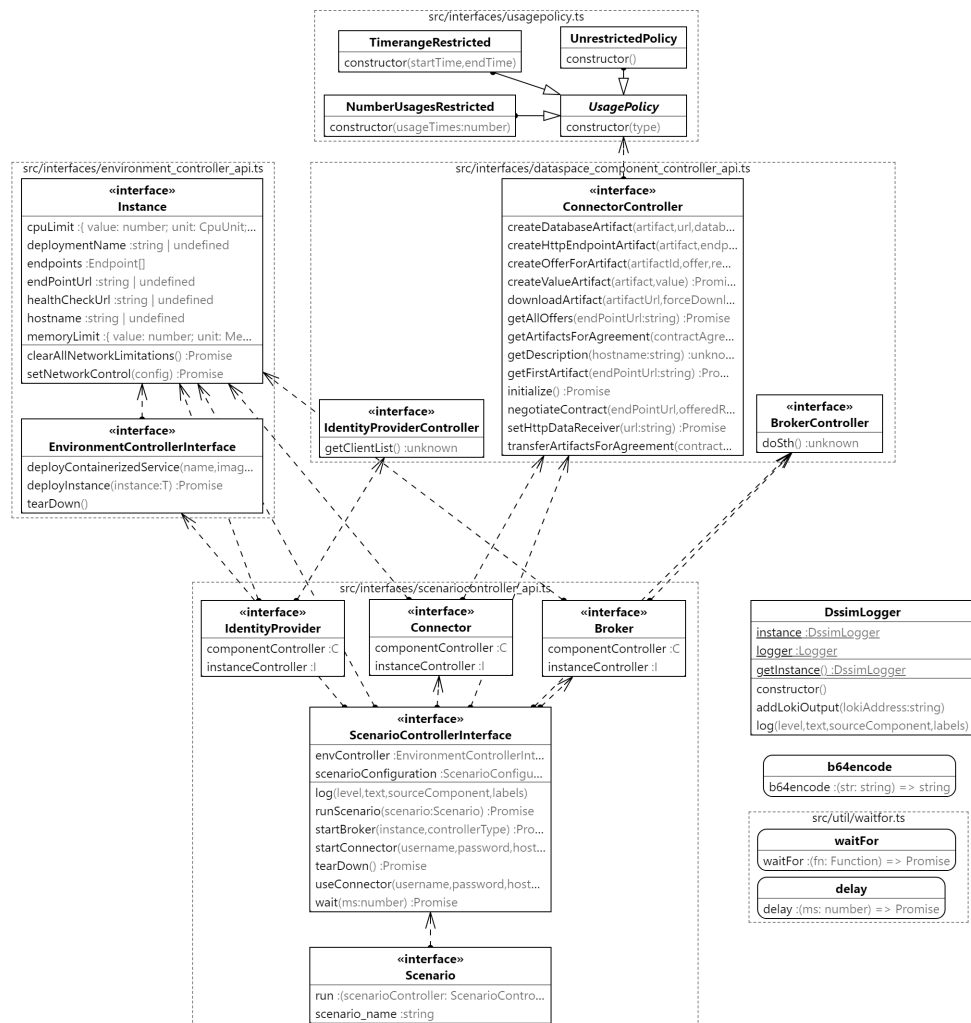
1.1	Lösungsansatz	4
2.1	Service Bereitstellung über Ingress	28
5.1	Systemkontext	42
5.2	Systemübersicht	43
6.1	Struktur dssim-core Projekt	52
6.2	Struktur Scenario-Execution Projekt	54
6.3	Vereinfachte Architektur DSC-Controller	63
6.4	Klassendiagramm Kubernetes-Controller	67
6.5	Logging-Pipeline des Scenario-Monitoring Moduls	77
6.6	Scraping-Pipeline des Scenario-Monitoring Moduls	79
6.7	Beispiel für Dashboard zur Szenarioüberwachung	80
7.1	Schematische Darstellung Energy Data-X	82
7.2	Datenfluss im Evaluationsszenario	86
7.3	Ausführung Szenario ohne Netzwerkeinschränkungen	90
7.4	Ausführung Szenario mit Netzwerkeinschränkungen	91

Quellcodeverzeichnis

5.1	Szenarioeinstellungen mit Umgebungsvariablen	48
6.1	Beispiel für eine Szenariokonfiguration	55
6.2	Erzeugen eines Connectors	56
6.3	Überschreiben der Standardkonfiguration	57
6.4	Methodensignatur zum Starten eines Connectors	61
6.5	Type Predication in TypeScript	68
6.6	Template Methode	70
7.1	Beeinflussung Netzwerkqualität Connectoren	88
1	Evaluationsszenario mit DSC	ii
2	Evaluationsszenario mit EDC	x
3	Evaluationsszenario mit Fassade	xvii

Anhang

1 Klassendiagramm dssim-core



2 Implementierung Evaluationsszenario

2.1 DSC spezifisch

```
1 import {RouteDesc, Rule} from 'dsc-lib';
2 import {Scenario, ScenarioControllerInterface} from 'dssim-
  core';
3 import {DSCController, getIdFromURL, getSelfRefId} from '
  dssim-ids-controller';
4 import {DSCInstance} from 'dssim-kubernetes-controller';
5 import {v4 as uuid} from 'uuid';
6 import {dscFactory, pullSecret} from '../configurations/index
  .js';
7
8 export class SysandukDsc implements Scenario {
9   scenario_name = 'SysAnDuk full szenario';
10  run = async (scenarioController:
    ScenarioControllerInterface) => {
11    // Validierung Umgebungsvariablen
12    if (!process.env.SYSANDUK_USERNAME || !process.env.
      SYSANDUK_PASSWORD)
13      throw new Error('Username/Password Environment
        Variables for SysAnDuk not set.');
```

```
14
15    // Definition Anlagen-IDs
16    const facilities = [
17      'I511234b4-4fc0-479f-950c-db7cd130ef70',
18      'I9397f4f7-3b3b-4487-83a5-443118c9333f',
19      'Ib6975fd1-8504-4b28-b3f7-f409af9ea046',
20      'I2ddc0612-fd72-4d27-9b79-360b5085437c',
21      'I2167e489-94ee-4672-b84f-263072d1d22d',
22    ];
23
24    // Initialisierung der Anbieter
25    const connectorPromises = facilities.map((facility, index
    ) =>
26      scenarioController.startConnector<DSCInstance,
        DSCController>(
27        'admin',
28        'password',
29        'provider' + index,
30        dscFactory('provider' + index),
31        DSCController
32      )
33    );
34
35    // Start aller Connectoren abwarten
36    const providers = await Promise.all(connectorPromises);
```

```
37     scenarioController.log(  
38         'info',  
39         'finished startup of all providers',  
40         SysandukDsc.name,  
41         {}  
42     );  
43  
44     // Datenangebote bei Anbietern anlegen und konfigurieren  
45     await Promise.all(  
46         providers.map(async (connector, index) => {  
47             const endpoint = await connector.componentController.  
48                 connectorApi.endpointsService.createEndpoint(  
49                 {  
50                     location: 'https://iee-bundle-sysanduk-vpp.rke1.  
51                         iee.fraunhofer.de/RESTfulService/TimeSeries/?  
52                         href=false&invalidateImplausibleValues=true&  
53                         timeseriesView=LATEST_VALUE&interpolationMode=  
54                         STEP&facilityMrid=${facilities[index]}' ,  
55                     type: 'GENERIC',  
56                 }  
57             );  
58             const dataSource = await connector.  
59                 componentController.connectorApi.  
60                 dataSourceService.createDataSource(  
61                 {  
62                     basicAuth: {  
63                         key: process.env.SYSANDUK_USERNAME ,  
64                         value: process.env.SYSANDUK_PASSWORD ,  
65                     },  
66                     type: 'REST',  
67                 }  
68             );  
69             await connector.componentController.connectorApi.  
70                 endpointsService.linkDataSource(  
71                 getSelfRefId(endpoint),  
72                 getSelfRefId(dataSource)  
73             );  
74             const route = await connector.componentController.  
75                 connectorApi.routesService.createRoute(  
76                 {  
77                     title: 'Sysanduk Route',  
78                     deploy: RouteDesc.deploy.CAMEL ,  
79                 }  
80             );  
81             const routeId = getSelfRefId(route);  
82             await connector.componentController.connectorApi.  
83                 routesService.createStartEndpoint(  
84                 routeId,
```

```
76     endpoint._links!.self.href!
77   );
78   const createdArtifact = await connector.
    componentController.connectorApi.artifactsService.
    createArtifact(
79     {
80       title: 'Sysanduk artifact',
81       description: 'PV Data',
82       accessUrl: route._links!.self.href!,
83     }
84   );
85
86   const createdCatalog = await connector.
    componentController.connectorApi.catalogsService.
    createCatalog(
87     {
88       title: 'Sysanduk Catalog',
89       description: 'Data Offerings of Sysanduk',
90     }
91   );
92
93   const createdOffer = await connector.
    componentController.connectorApi.
    offeredResourcesService.createOffer(
94     {
95       title: 'Latest Values',
96       description: 'Get most current data',
97       keywords: ['PV', 'energy'],
98       publisher: 'http://fraunhofer.de',
99       language: 'English',
100      license: 'FAIR',
101      sovereign: 'http://fraunhofer.de',
102      samples: [],
103    }
104  );
105
106  const createdRepresentation = await connector.
    componentController.connectorApi.
    representationsService.createRepresentation(
107    {
108      title: 'Custom Data',
109      mediaType: 'application/xml',
110      standard: '',
111    }
112  );
113
114  const contract = await connector.componentController.
    connectorApi.contractsService.createContract(
115    {
```

```
116         title: 'Contract Offer',
117         description: 'created for offer sysanduk',
118         start: new Date().toISOString(),
119         end: new Date(new Date().getFullYear() + 1, 1, 1)
            .toISOString(),
120     }
121 );
122
123 const rule = await connector.componentController.
    connectorApi.rulesService.createRule(
124     {
125         value: JSON.stringify({
126             '@context': {
127                 ids: 'https://w3id.org/idsa/core/',
128                 idsc: 'https://w3id.org/idsa/code/',
129             },
130             '@type': 'ids:Permission',
131             '@id':
132                 'https://w3id.org/idsa/autogen/permission/
                    cf1cb758-b96d-4486-b0a7-f3ac0e289588',
133             'ids:action': [
134                 {
135                     '@id': 'idsc:USE',
136                 },
137             ],
138             'ids:description': [
139                 {
140                     '@value': 'provide-access',
141                     '@type': 'http://www.w3.org/2001/XMLSchema#
                        string',
142                 },
143             ],
144             'ids:title': [
145                 {
146                     '@value': 'Example Usage Policy',
147                     '@type': 'http://www.w3.org/2001/XMLSchema#
                        string',
148                 },
149             ],
150         }
151     },
152 );
153
154 await connector.componentController.connectorApi.
    catalogsService.addOfferedRessourceToCatalog(
155     getSelfRefId(createdCatalog),
156     [getSelfRefId(createdOffer)]
157 );
158
```

```

159         await connector.componentController.connectorApi.
            offeredResourcesService.
            addRepresentationToOfferedResource(
160             getSelfRefId(createdOffer),
161             [getSelfRefId(createdRepresentation)]
162         );
163         await connector.componentController.connectorApi.
            representationsService.addRepresentationArtifact(
164             getSelfRefId(createdRepresentation),
165             [getSelfRefId(createdArtifact)]
166         );
167         await connector.componentController.connectorApi.
            offeredResourcesService.addContractToResource(
168             getSelfRefId(createdOffer),
169             [getSelfRefId(contract)]
170         );
171         await connector.componentController.connectorApi.
            contractsService.addRuleToContract(
172             getSelfRefId(contract),
173             [getSelfRefId(rule)]
174         );
175     })
176 );
177 scenarioController.log(
178     'info',
179     'finished provisioning of all providers',
180     SysandukDsc.name,
181     {}
182 );
183
184 // Initialisierung des Consumer-Connectors
185 const consumer = await scenarioController.startConnector<
186     DSCInstance,
187     DSCController
188 >('admin', 'password', 'consumer', dscFactory('consumer')
    , DSCController);
189
190 // Initialisierung Datensenke
191 const datasink =
192     await scenarioController.envController.
        deployContainerizedService(
193         'dataservice',
194         {
195             image:
196                 'registry.gitlab.cc-asp.fraunhofer.de/dssim/dssim
                    -kubernetes-controller/dataservice:autopull',
197             pullSecret: pullSecret,
198         },
199         [{port: 4000, name: 'default', path: '/'}]

```



```
200     );
201     scenarioController.log('info', 'started a consumer',
        SysandukDsc.name, {});
202
203     // Route zu Datensenke
204     const endpoint =
205         await consumer.componentController.connectorApi.
            endpointsService.createEndpoint(
206         {
207             location: 'http://${datasink.hostname}:${datasink.
                endpoints[0].port}/sink',
208             type: 'GENERIC',
209         }
210     );
211     const route =
212         await consumer.componentController.connectorApi.
            routesService.createRoute(
213         {
214             title: 'Datasink route',
215             deploy: RouteDesc.deploy.CAMEL,
216         }
217     );
218     await consumer.componentController.connectorApi.
        routesService.createLastEndpoint(
219         getSelfRefId(route),
220         getSelfRefId(endpoint)
221     );
222
223     scenarioController.log(
224         'info',
225         'setup complete. Starting negotiation',
226         SysandukDsc.name,
227         {}
228     );
229
230     const contracts: string[] = [];
231
232     // Nutzungsvereinbarungen treffen
233     await Promise.all(
234         providers.map(async (provider, index) => {
235             const connectorDescription = JSON.parse(
236                 await consumer.componentController.connectorApi.
                    messagingService.sendIdsDescription(
237                     'https://${provider.instanceController.hostname}/
                        api/ids/data'
238                 )
239             );
240
241             const catalogDescription = JSON.parse(
```

```

242         await consumer.componentController.connectorApi.
           messagingService.sendIdsDescription(
243             'https://${provider.instanceController.hostname}/
           api/ids/data',
244             connectorDescription['ids:resourceCatalog'][0]['
           @id']
245         )
246     );
247
248     const offers = catalogDescription['ids:
           offeredResource'].map(
249         (e: any) => {
250             return {
251                 offerId: getIdFromURL(e['@id']),
252                 contractOfferId: getIdFromURL(e['ids:
           contractOffer'][0]['@id']),
253                 assetId: getIdFromURL(
254                     e['ids:representation'][0]['ids:instance'
                       ][0]['@id']
255                 ),
256                 assetName: e['ids:title'][0]['@value'],
257             };
258         }
259     );
260
261     const offeredRessource = offers[0];
262
263     const contractOfferDescription = JSON.parse(
264         await consumer.componentController.connectorApi.
           messagingService.sendIdsDescription(
265             `${provider.instanceController.hostname}/api/ids/
           data`,
266             `${provider.instanceController.hostname}/api/
           contracts/${offeredRessource.contractOfferId}`
267         )
268     );
269
270     const contract: Rule = contractOfferDescription['ids:
           permission'][0];
271     contract[
272         'ids:target'
273     ] = `${provider.instanceController.hostname}/api/
           artifacts/${offeredRessource.assetId}`;
274
275     const contractResponse = JSON.parse(
276         await consumer.componentController.connectorApi.
           messagingService.sendContractRequestMessage(
277             `${provider.instanceController.hostname}/api/ids
           /data`,

```

```

278         [
279             `${provider.instanceController.hostname}/api/
                offers/${offeredRessource.offerId}`,
280         ],
281         [
282             `${provider.instanceController.hostname}/api/
                artifacts/${offeredRessource.assetId}`,
283         ],
284         false,
285         [contract]
286     )
287 );
288
289     contracts[index] = getIdFromURL(getSelfRefId(
        contractResponse));
290 })
291 );
292
293 // Initial 0-5 Sekunden warten, dann 100 Mal
    Datenuebertragung alle 15 Sekunden
294 await Promise.all(
    providers.map(async (provider, index) => {
295         await scenarioController.wait(Math.random() * 5000);
296         for (let x = 0; x < 100; x++) {
297             const id: string = uuid();
298             scenarioController.log(
299                 'info',
300                 `starting to query ${provider.instanceController.
                    endPointUrl}`,
301                 SysandukDsc.name,
302                 {
303                     requestId: id,
304                     participant: `${provider.instanceController.
                        endPointUrl}`,
305                     szenarioEvent: 'startMeasure',
306                 }
307             );
308             const startTime = Date.now();
309             const agreementArtifacts =
310                 await consumer.componentController.connectorApi.
                    contractsService.getArtifactsForAgreement(
311                     contracts[index]
312                 );
313             await consumer.componentController.connectorApi.
                    artifactsService.getDataQuery(
314                 getIdFromURL(
315                     agreementArtifacts!._embedded!.artifacts![0].
                        _links!.self.href!
316                 ),
317

```

```

318         true,
319         undefined, //contracts[index],
320         [getSelfRefId(route)]
321     );
322
323     scenarioController.log(
324         'info',
325         'results from ${
326             provider.instanceController.hostname
327         } are in after ${Date.now() - startTime} ms.',
328         SysandukDsc.name,
329         {
330             requestId: id,
331             participant: `${provider.instanceController.
332                 hostname}`,
333             szenarioEvent: 'endMeasure',
334         }
335     );
336
337     // wait some time
338     await scenarioController.wait(15000);
339 }
340 })
341 };
342 }

```

Quellcode 1: Evaluationsszenario mit DSC

2.2 EDC spezifisch

```

1 import {b64encode,Scenario,ScenarioControllerInterface,
   waitFor} from 'dssim-core';
2 import {EDCController} from 'dssim-edc-controller';
3 import {EDCInstance} from 'dssim-kubernetes-controller';
4 import {edcFactory} from '../configurations/edcFactory.js';
5 import {v4 as uuid} from 'uuid';
6 import {pullSecret} from '../configurations/index.js';
7
8 export class SysandukEdc implements Scenario {
9     scenario_name = 'SysAnDUk with EDC';
10    run = async (scenarioController:
11        ScenarioControllerInterface) => {
12        // Validierung Umgebungsvariablen
13        if (!process.env.SYSANDUK_USERNAME || !process.env.
14            SYSANDUK_PASSWORD)
15            throw new Error('Username/Password Environment
16                Variables for SysAnDuk not set.');
```

```
14
15 // Definition Anlagen-IDs
16 const facilities = [
17   'I511234b4-4fc0-479f-950c-db7cd130ef70',
18   'I9397f4f7-3b3b-4487-83a5-443118c9333f',
19   'Ib6975fd1-8504-4b28-b3f7-f409af9ea046',
20   'I2ddc0612-fd72-4d27-9b79-360b5085437c',
21   'I2167e489-94ee-4672-b84f-263072d1d22d',
22 ];
23
24 // Initialisierung der Anbieter
25 const connectorPromises = facilities.map((facility, index) =>
26   scenarioController.startConnector<EDCInstance,
27     EDCController>(
28     'admin',
29     'password',
30     'provider' + index,
31     edcFactory('provider' + index),
32     EDCController
33   )
34 );
35
36 // Start aller Connectoren abwarten
37 const providers = await Promise.all(connectorPromises);
38 scenarioController.log(
39   'info',
40   'finished startup of all providers',
41   SysandukEdc.name,
42   {}
43 );
44
45 // Datenangebote bei Anbietern anlegen und konfigurieren
46 await Promise.all(
47   providers.map(async (connector, index) => {
48     await connector.componentController.connectorApi.
49       assetService.createAsset({
50         asset: {
51           properties: {
52             'asset:prop:id': 'assetId',
53             'asset:prop:name': 'product description',
54             'asset:prop:contenttype': 'application/json',
55           },
56         },
57         dataAddress: {
58           properties: {
59             name: 'Sysanduk Data',
60             baseUrl: 'https://iee-bundle-sysanduk-vpp.rke1.iee.fraunhofer.de/RESTfulService/TimeSeries/?href='
61           }
62         }
63       })
64   })
65 );
```

```

        false&invalidateImplausibleValues=true&
        timeseriesView=LATEST_VALUE&interpolationMode=
        STEP&facilityMrid=${facilities[index]}',
59     type: 'HttpData',
60     authKey: 'Authorization',
61     authCode: 'Basic ${b64encode(
62         '${process.env.SYSANDUK_USERNAME}:${process.env.
        SYSANDUK_PASSWORD}'}',
63     )}',
64 },
65 },
66 });
67
68     await connector.componentController.connectorApi.
        policyService.createPolicy({
69         id: 'aPolicy',
70         policy: {
71             uid: '231802-bb34-11ec-8422-0242ac120002',
72             permissions: [
73                 {
74                     target: 'assetId',
75                     action: {
76                         type: 'USE',
77                     },
78                     edctype: 'dataspaceconnector:permission',
79                 },
80             ],
81             '@type': {
82                 '@policytype': 'set',
83             },
84         },
85     });
86
87     await connector.componentController.connectorApi.
        contractDefinitionService.createContractDefinition({
88         id: '1',
89         accessPolicyId: 'aPolicy',
90         contractPolicyId: 'aPolicy',
91         criteria: [],
92     });
93 });
94     scenarioController.log(
95         'info',
96         'finished provisioning of all providers',
97         SysandukEdc.name,
98         {}
99     );
100
101     // Initialisierung des Consumer-Connectors

```

```
102     const consumer = await scenarioController.startConnector<  
        EDCInstance,EDCController>(  
103         'admin',  
104         'password',  
105         'edcconsumer',  
106         edcFactory('edcconsumer'),  
107         EDCController  
108     );  
109  
110     // Initialisierung Datensinke  
111     const datasink =  
112     await scenarioController.envController.  
        deployContainerizedService(  
113         'dataservice',  
114         {  
115             image: 'registry.gitlab.cc-asp.fraunhofer.de/dssim/  
                dssim-kubernetes-controller/dataservice:autopull',  
116             pullSecret: pullSecret,  
117         },  
118         [{port: 4000, name: 'default', path: '/'}]  
119     );  
120  
121     await consumer.componentController.connectorApi.  
        dataplaneSelectorService.addEntry(  
122     {  
123         // "edctype": "dataspaceconnector:dataplaneinstance",  
124         id: 'http-pull-consumer-dataplane',  
125         url: 'http://${consumer.instanceController.hostname}/  
            control/transfer',  
126         allowedSourceTypes: ['HttpData'],  
127         allowedDestTypes: ['HttpProxy', 'HttpData'],  
128         properties: {  
129             publicApiUrl: 'http://${consumer.instanceController.  
                hostname}:${EDCInstance.PublicEndpoint.port}}${  
                EDCInstance.PublicEndpoint.path}/',  
130         },  
131     }  
132     );  
133  
134     scenarioController.log(  
135         'info',  
136         'setup complete. Starting negotiation',  
137         SysandukEdc.name,  
138         {}  
139     );  
140  
141     const contracts: string[] = [];  
142  
143     // Nutzungsvereinbarungen treffen
```

```

144     await Promise.all(
145     providers.map(async (provider, index) => {
146         const description =
147         await consumer.componentController.connectorApi.
            catalogService.requestCatalog(
148             {
149             providerUrl: 'http://${provider.instanceController.
                hostname}:8282/api/v1/ids/data',
150             }
151         );
152         console.log(description);
153
154         const nego =
155         await consumer.componentController.connectorApi.
            contractNegotiationService.
                initiateContractNegotiation({
156             connectorId: 'http-pull-provider',
157             connectorAddress: 'http://${provider.
                instanceController.hostname}:8282/api/v1/ids/data
                ',
158             protocol: 'ids-multipart',
159             offer: {
160                 offerId: '1:50f75a7a-5f81-4764-b2f9-ac258c3628e2'
161                 ,
162                 assetId: 'assetId',
163                 policy: {
164                     uid: '231802-bb34-11ec-8422-0242ac120002',
165                     permissions: [
166                         {
167                             target: 'assetId',
168                             action: {
169                                 type: 'USE',
170                             },
171                             edctype: 'dataspaceconnector:permission',
172                         },
173                     ],
174                     '@type': {
175                         '@policytype': 'set',
176                     },
177                 },
178             });
179         console.log(nego);
180
181         let contractAgreementId = '';
182         await waitFor(async () => {
183             const negoState = await consumer.componentController.
                connectorApi.contractNegotiationService.
                    getNegotiation(nego.id!);

```



```
184     if (
185         negoState.state === 'CONFIRMED' &&
186         negoState.contractAgreementId
187     ) {
188         scenarioController.log(
189             'info',
190             'Contract negotiation ${negoState.id} is confirmed
191             with contract agreement id ${negoState.
192                 contractAgreementId}',
193             'EDCScenario',
194             {}
195         );
196         contractAgreementId = negoState.contractAgreementId;
197         return true;
198     } else {
199         scenarioController.log(
200             'info',
201             'Contract negotiation ${negoState.id} is not yet
202             confirmed',
203             'EDCScenario',
204             {}
205         );
206         return false;
207     }
208 }));
209
210 contracts[index] = contractAgreementId;
211
212 // Initial 0-5 Sekunden warten, dann 100 Mal
213 // Datenuebertragung alle 15 Sekunden
214 await Promise.all(
215     providers.map(async (provider, index) => {
216         await scenarioController.wait(Math.random() * 5000);
217         for (let x = 0; x < 100; x++) {
218             const id: string = uuid();
219             scenarioController.log(
220                 'info',
221                 'starting to query ${provider.instanceController.
222                     endPointUrl}',
223                 SysandukEdc.name,
224                 {
225                     requestId: id,
226                     participant: `${provider.instanceController.
227                         endPointUrl}`,
228                     szenarioEvent: 'startMeasure',
229                 }
230             );
```

```

227     const startTime = Date.now();
228
229     const transfer = await consumer.componentController.
        connectorApi.transferProcessService.initiateTransfer
        ({
230         connectorId: 'http-pull-provider',
231         connectorAddress: 'http://${provider.
            instanceController.hostname}:8282/api/v1/ids/data
            ',
232         contractId: contracts[index],
233         assetId: 'assetId',
234         managedResources: false,
235         dataDestination: {
236             properties: {
237                 baseUrl: 'http://${datasink.hostname}:${
                    datasink.endpoints[0].port}/sink',
238                 type: 'HttpData',
239             },
240         },
241         protocol: 'ids-multipart',
242         transferType: {},
243     });
244
245     console.log(transfer);
246
247     await waitFor(async () => {
248         const transferStatus =
249             await consumer.componentController.connectorApi.
                transferProcessService.getTransferProcessState(
                    transfer.id!
250                );
251         if (transferStatus.state === 'COMPLETED') {
252             scenarioController.log(
253                 'info',
254                 'Transfer complete!',
255                 'EDCScenario',
256                 {}
257             );
258             return true;
259         } else {
260             scenarioController.log(
261                 'info',
262                 'Transfer in status ${transferStatus.state}!',
263                 'EDCScenario',
264                 {}
265             );
266             return false;
267         }
268     });
269

```

```

270     scenarioController.log(
271         'info',
272         'results from ${provider.instanceController.hostname}
           are in after ${Date.now() - startTime} ms.',
273         SysandukEdc.name,
274         {
275             requestId: id,
276             participant: `${provider.instanceController.
               hostname}`,
277             szenarioEvent: 'endMeasure',
278         }
279     );
280
281     // wait some time
282     await scenarioController.wait(15000);
283 }
284 }));
285 };
286 }

```

Quellcode 2: Evaluationsszenario mit EDC

2.3 Mit Fassade

```

1  import {Scenario, ScenarioControllerInterface} from 'dssim-
   core';
2  import {v4 as uuid} from 'uuid';
3  import {pullSecret} from '../configurations/index.js';
4
5  export class SysAnDukLatestValueScenario implements Scenario
   {
6      scenario_name = 'SysAnDuk full szenario';
7      run = async (scenarioController:
         ScenarioControllerInterface) => {
8          // Validierung Umgebungsvariablen
9          if (!process.env.SYSANDUK_USERNAME || !process.env.
              SYSANDUK_PASSWORD)
10             throw new Error('Username/Password Environment
                Variables for SysAnDuk not set.');
```

```

20
21 // Initialisierung der Anbieter
22 const connectorPromises = facilities.map((facility, index
    ) =>
23     scenarioController.startConnector('admin', 'password',
        'provider' + index)
24 );
25
26 // Start aller Connectoren abwarten
27 const providers = await Promise.all(connectorPromises);
28 scenarioController.log(
29     'info',
30     'finished startup of all providers',
31     SysAnDukLatestValueScenario.name,
32     {}
33 );
34
35 // Datenangebote bei Anbietern anlegen und konfigurieren
36 await Promise.all(
37     providers.map(async (connector, index) => {
38         const artifact = await connector.componentController.
            createHttpEndpointArtifact(
39             { name: 'SysAnDUk VPP - Latest Values' },
40             'https://iee-bundle-sysanduk-vpp.rke1.iee.fraunhofer.
                de/RESTfulService/TimeSeries/?href=false&
                invalidateImplausibleValues=true&timeseriesView=
                LATEST_VALUE&interpolationMode=STEP&facilityMrid=$
                {facilities[index]}',
41             'application/xml',
42             undefined,
43             {
44                 username: process.env.SYSANDUK_USERNAME!,
45                 password: process.env.SYSANDUK_PASSWORD!,
46             });
47
48         await connector.componentController.
            createOfferForArtifact(
49             artifact,
50             {
51                 name: 'SysAnDUk VPP - Latest Values of ' + facilities
                    [index],
52                 start: new Date(),
53                 end: new Date(new Date().getFullYear() + 1, 1, 1),
54             },
55             { mediaType: 'plain/text' },
56             { name: 'Solar Energy Info Catalog' }
57         );
58     })
59 );

```

```
60     scenarioController.log(  
61         'info',  
62         'finished provisioning of all providers',  
63         SysAnDukLatestValueScenario.name,  
64         {}  
65     );  
66  
67     // Netzwerkeinschraenkungen (nur fuer zweiten Teil der  
68         Validierung)  
69     await providers[0].instanceController.setNetworkControl({  
70         bandwidth: {  
71             value: 20,  
72             unit: 'kbit',  
73         },  
74     });  
75     await providers[1].instanceController.setNetworkControl({  
76         delay: {value: 200, unit: 'ms'},  
77     });  
78     await providers[2].instanceController.setNetworkControl({  
79         delay: {value: 50, unit: 'ms'},  
80     });  
81     await providers[3].instanceController.setNetworkControl({  
82         bandwidth: {  
83             value: 20,  
84             unit: 'kbit',  
85         },  
86         delay: {value: 200, unit: 'ms'},  
87     });  
88  
89     // Initialisierung des Consumer-Connectors  
90     const consumer = await scenarioController.startConnector(  
91         'admin',  
92         'password',  
93         'consumer'  
94     );  
95  
96     scenarioController.log(  
97         'info',  
98         'started a consumer',  
99         SysAnDukLatestValueScenario.name,  
100         {}  
101     );  
102  
103     // Initialisierung Datensenke  
104     const datasink = await scenarioController.envController.  
105         deployContainerizedService(  
106             'dataservice',  
107             {  
108                 image:
```

```
107         'registry.gitlab.cc-asp.fraunhofer.de/dssim/dssim-
108             kubernetes-controller/dataservice:autopull',
109         pullSecret: pullSecret,
110     },
111     [{port: 4000, name: 'default', path: '/'}]
112 );
113
114 await consumer.componentController.setHttpDataReceiver(
115     'http://${datasink.hostname}:${datasink.endpoints[0].
116         port}/sink'
117 );
118
119 scenarioController.log(
120     'info',
121     'setup complete. Starting scenario in 10 seconds',
122     SysAnDukLatestValueScenario.name,
123     {
124         szenarioEvent: 'startScenario',
125     }
126 );
127
128 const contracts: string[] = [];
129
130 // Nutzungsvereinbarungen treffen
131 await Promise.all(
132     providers.map(async (provider, index) => {
133         const offers = await consumer.componentController.
134             getAllOffers(
135                 `${provider.instanceController.endPointUrl}`
136             );
137         contracts[index] = (
138             await consumer.componentController.negotiateContract(
139                 `${provider.instanceController.endPointUrl}`,
140                 offers[0]
141             ).contractId;
142         })
143 );
144
145 // Initial 0-5 Sekunden warten, dann 100 Mal
146 // Datenuebertragung alle 15 Sekunden
147 await Promise.all(
148     providers.map(async (provider, index) => {
149         await scenarioController.wait(Math.random() * 5000);
150         for (let x = 0; x < 100; x++) {
151             const id: string = uuid();
152             scenarioController.log(
153                 'info',
```

```
151     'starting to query ${provider.instanceController.  
152         endPointUrl}',  
153     SysAnDukLatestValueScenario.name,  
154     {  
155         requestId: id,  
156         participant: '${provider.instanceController.  
157             hostname}',  
158         szenarioEvent: 'startMeasure',  
159     }  
160 );  
161 const startTime = Date.now();  
162  
163 try {  
164     await consumer.componentController.  
165         transferArtifactsForAgreement(  
166             contracts[index]  
167         );  
168     scenarioController.log(  
169         'info',  
170         'results from ${  
171             provider.instanceController.hostname  
172             } are in after ${Date.now() - startTime} ms.',  
173         SysAnDukLatestValueScenario.name,  
174         {  
175             requestId: id,  
176             participant: '${provider.instanceController.  
177                 hostname}',  
178             status: 'success',  
179             szenarioEvent: 'endMeasure',  
180         }  
181     );  
182 } catch (error) {  
183     console.error('Error during Transfer:', error);  
184     scenarioController.log(  
185         'info',  
186         'error during data transfer from ${  
187             provider.instanceController.hostname  
188             } are in after ${Date.now() - startTime} ms.',  
189         SysAnDukLatestValueScenario.name,  
190         {  
191             requestId: id,  
192             participant: '${provider.instanceController.  
193                 hostname}',  
194             status: 'error',  
195             szenarioEvent: 'endMeasure',  
196         }  
197     );  
198 }
```

```
195         // wait some time
196         await scenarioController.wait(15000);
197     }
198 })
199 );
200
201 scenarioController.log(
202     'info',
203     'scenario completed.',
204     SysAnDukLatestValueScenario.name,
205     {scenarioEvent: 'endScenario'}
206 );
207 };
208 }
```

Quellcode 3: Evaluationsszenario mit Fassade